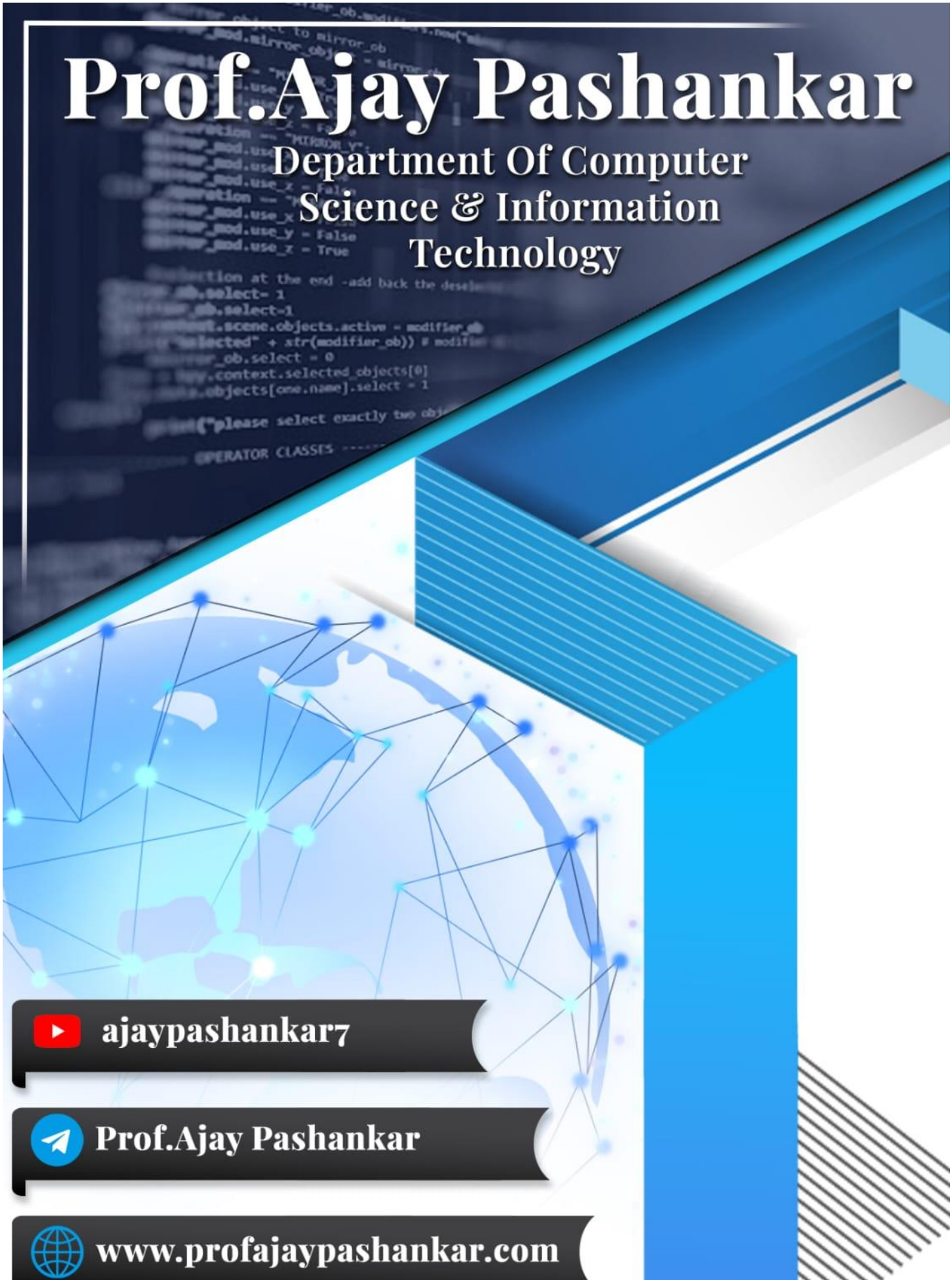

Prof.Ajay Pashankar
Department Of Computer Science & Information Technology
ajaypashankar7
Prof.Ajay Pashankar
www.profajaypashankar.com

INDEX

Sr. No.	Practical	Topic
1	Practical 1(a)	Create an MVC Application to Perform Basic Arithmetic Operations
2	Practical 1(b)	Create an MVC Application to Perform Basic Arithmetic Operations using Console Application
3	Practical 1(c)	Arrange Names of Cities in Sorted Order using C#
4	Practical 2(a)	Function Overloading in C#
5	Practical 2(b)	Inheritance (All Types) in C#
6	Practical 2(c)	Constructor Overloading in C#
7	Practical 2(d)	Interfaces in C#
8	Practical 2(e)	Delegates and Events in C#
9	Practical 2(f)	Exception Handling in C#
10	Practical 3(a)	Methods using C# Delegates
11	Practical 3(b)	Multicast Delegate
12	Practical 3(c)	BankAccount Class with Property Validation
13	Practical 4(a)	ASP.NET Web Forms: Student Registration Form
14	Practical 4(b)	Registration Form using Validation Controls
15	Practical 4(c)	AdRotator Control
16	Practical 5(a)	Different Types of Cookies
17	Practical 5(b)	Product Table and Insert Operation using Entity Framework Core
18	Practical 5(c)	CRUD Operations on Employee Table using EF Core
19	Practical 5(d)	Authentication and Authorization in ASP.NET Core
20	Practical 6(a)	College Information Portal and Progressive Web Application
21	Practical 6(b)	PWA with Custom Offline Page
22	Practical 7(a)	PWA Working in Offline Mode using Service Worker
23	Practical 7(b)	Student Record Management PWA
24	Practical 8(a)	Basic PWA with HTML Starting Page and Manifest
25	Practical 8(b)	Web API Returning Student Data in JSON Format
26	Practical 9(a)	CRUD Operations using ASP.NET Core Web API and Entity Framework Core
27	Practical 9(b)	Web API for Factorial of a Number and Postman Testing
28	Practical 10(a)	PWA Manifest and Push Notifications
29	Practical 10(b)	E-Commerce PWA with Product Listing and Offline Cart

Module-1 : Introduction to C#.NET

PRACTICAL – 1

(A) Create an MVC Application to perform basic Arithmetic Operations.

To create an ASP.NET Core MVC application that performs basic arithmetic operations (Addition, Subtraction, Multiplication, Division) using C#.

Software Required

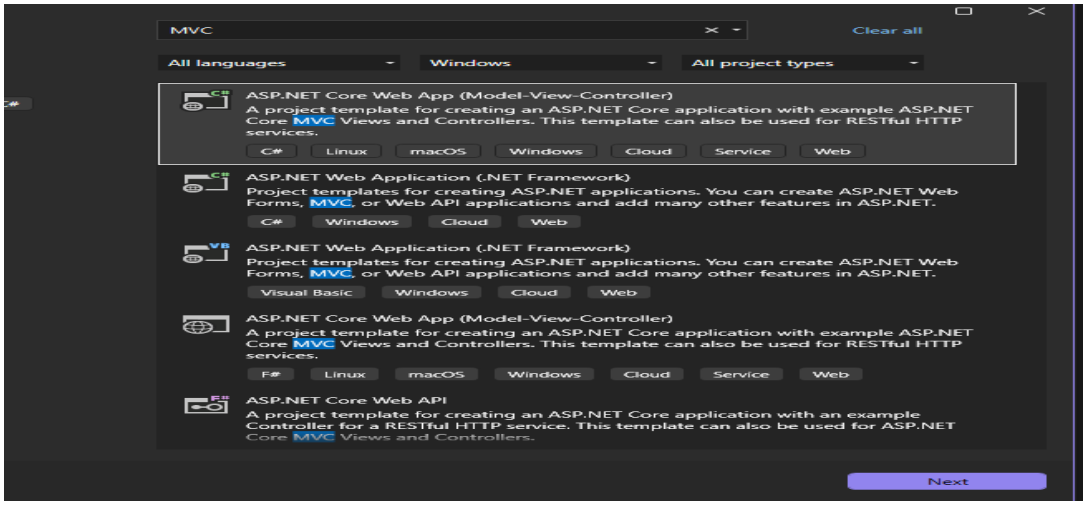
IDE	Microsoft Visual Studio
Framework	ASP.NET Core MVC(.NET 6/ .NET 8)
Language	C#

Steps

1: Open visual studio 2022/2026

2: Create new project

- Click on create a new project
- Search for MVC
- Select project type as ASP.NET Core Web App(Model-View-controller)
- Then click next



3: Configure Project

Enter Details

Field	Value
Project Name	ArithmeticMvc
Location	Choose desired folder
Solution Name	ArithmeticMVC

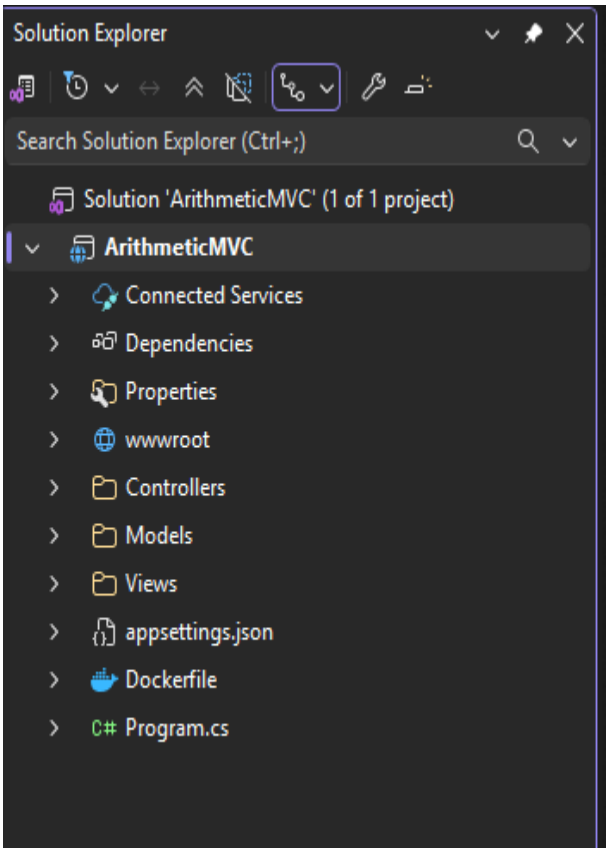
4: Additional Information

Option	Value
Framework	.NET 8 (or latest available)
Authentication Type	None
Configure HTTPS	checked

5: understanding Project Structure

On right side, you will see:

Solution Explorer



Folder	Purpose
Controllers	Contains controller files
Models	Contains data classes
View	Contains UI pages
wwwroot	CSS,Js, Images
Program.cs	Main startup file

6: Create Model File

Model stores data

- In solution Explorer(If this window is not seen,go to view Menu and select it.)
- Right-click: Models →Add→Class
- Enter File name as Calculator.cs and click Add

7: Write Model Code(Delete existing code and paste)

```
namespace ArithmeticMVC.Models
{
   public class Calculator
   {
      public double Number1 { get; set; }
      public double Number2 { get; set; }
      public required string Operation { get; set; }
      public double Result { get; set; }
   }
}
```

- Press Ctrl+ S to save

8: Open Controller

In solution explorer: open Controllers => Double click HomeController.cs

9: Modify Controller code:

Remove existing code inside HomeController and replace with:

```
Using  Microsoft.AspNetCore.Mvc;
Using  ArithmeticMVC.Models;

namespace ArithmeticMVC.Controllers
```

```
{
   public class HomeController:Controller
   {

         //GET Method
         public IActionResult Index()
         {
            return View();
         }
         //POST Method
         [HttpPost]
      public IActionResult Index(Calculator calc)
         {
           switch(calc.Operation)
         {
            case "+":
               calc.Result = calc.Number1 + calc.Number2;
               break;

            case "-":
               calc.Result = calc.Number1 - calc.Number2;
               break;

            case "*":
               calc.Result = calc.Number1 * calc.Number2;
               break;

            case "/":
               if (calc.Number2!=0)
               {
                  calc.Result = calc.Number1 / calc.Number2;
               }
               break;
         }
         return View(calc);
      }
    }
   }
```

Save the file

10: Open views Folder

In solution explorer : Open: => Views -> Home

Double-click:=> Index.cshtml

11: Replace Existing View Code : Delete all existing code and paste:

```
@model ArithmeticMVC.Models.Calculator
```

```
<h2>Arithmetic Operations</h2>
<form method="post">
        <label>Enter Number1</label><br/>
        <input type="number" step="any"asp-for="Number1"/>
        <br/><br/>
        <label>Enter Number2</label><br/>
        <input type="number" step="any"asp-for="Number2"/>
        <br/><br/>
        <label>Select Operation</label><br/>
        <select asp-for="Operation">
                <option value="+">Addition</option>
                <option value="-">Subtraction</option>
                <option value="*">Multiplication</option>
                <option value="/">Division</option>
                </select>
                <br/><br/>
                <input type="submit" value="Calculate"/>
        </form>
        <br/>

        @if(Model!=null)
{
        <h3>Result=@Model.Result</h3>
}
```

12: Build the Project

At top menu click:

Build → Build Solution (Ctrl+Shift+B)

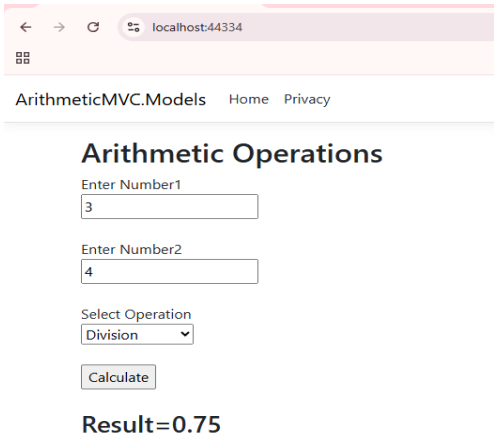
If build succeeds :

- No error will appear

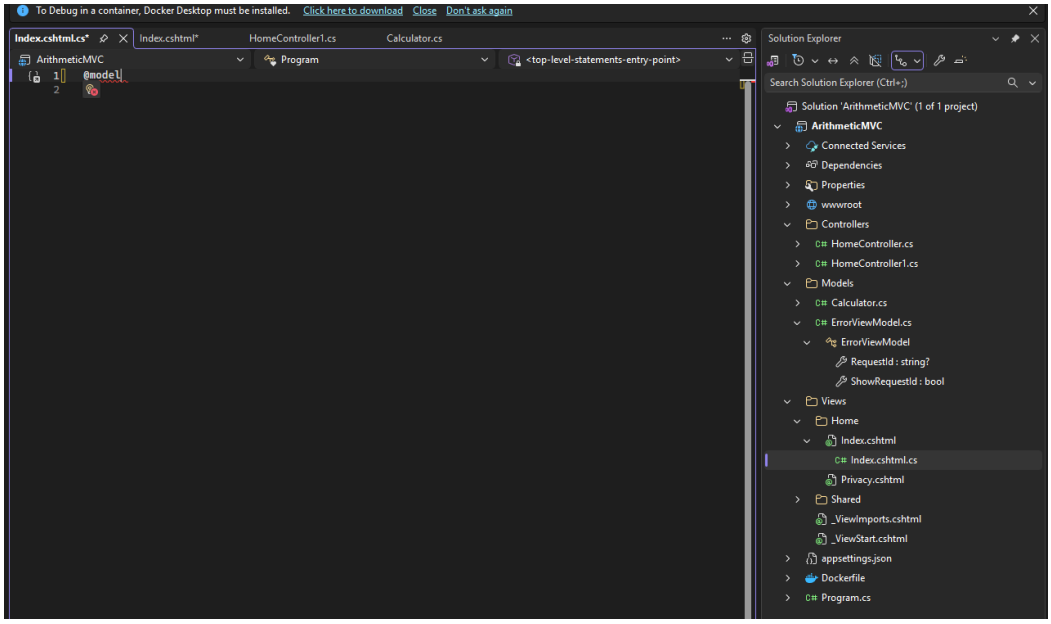
Step 13 : Run the Application

Click : IIS Express(Ctrl+F5)

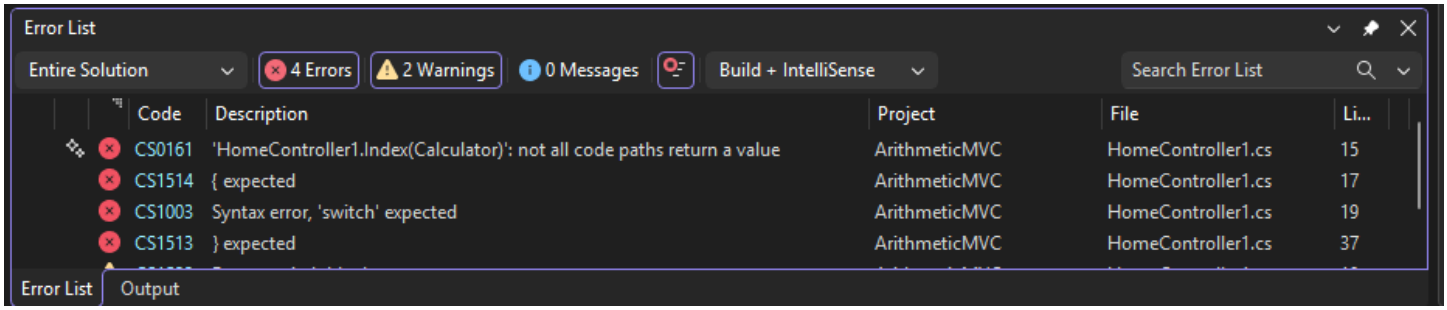
Output



Errors you could face :



Make sure you click on Index.cshtml and not c# intex.cshtml



Important Methods / Functions :

1. `public IActionResult Index()` - GET Method

   Used to load the home page. Returns the view using `return View();`

2. `[HttpPost]` - Attribute

   Specifies that this method will be called when the form is submitted. Used to handle POST requests.

3. `public IActionResult Index(Calculator calc)` - POST Method

   Receives the `Calculator` model data from the form. Performs the arithmetic calculation here.

4. `return View(calc);` - Method

   Returns the view along with the model so that the result can be displayed.

5. `switch (calc.Operation)` - Control Statement

   Performs +, -, *, / operations based on the selected operation.

6. `@model ArithmeticMVC.Models.Calculator` - Razor Directive

   Declares which model class the view will use.

7. `asp-for="Number1"` - Tag Helper

   Binds the input field to a property of the Model.

8. `method="post"` - HTML Form Attribute

   Sends form data to the Controller using the POST method.

Important Properties in Model:

1. `public double Number1 { get; set; }` - Stores the first number input

2. `public double Number2 { get; set; }` - Stores the second number input

3. `public required string Operation { get; set; }` - Stores the operation: +, -, *, /

4. `public double Result { get; set; }` - Stores the calculated result

(B) Floyd's Triangle till N Rows in C#

To write a c# program to print Floyd's triangle up to N rows, where N is input by the user.

Method: Console Application (Recommended new students as beginner)

1: Create New project

2: Select Project type

In search box type: Console C#

Select:

Console App(.NET Framework):C#

3: Configure Project

Enter:

Field	Value
Project Name	FloydTriangle
Location	Choose your folder

4: Additional Information

Choose latest .NET version.

Click: Create

5: Open Program.cs File

In Solution Explorer, double-click:

Program.cs

You will see default code, remove Existing code

6: Write Floyd's Triangle Program (Delete all existing code and Paste the following code)

```
using System;

class Program
{
```

```
    static void Main()
    {
        int n, number = 1;

        Console.Write("Enter number of rows:");
        n = Convert.ToInt32(Console.ReadLine());

        for(int i =1;i <=n;i++)
        {
            for(int j=1;j <=i;j++)
            {
                Console.Write(number + " ");
                number++;
            }

            Console.WriteLine();
        }
        Console.ReadLine();
    }
}
```

7: Save the file

8: Build the Project

At top menu:

Build → Build solution (Ctrl+shift+B) **Ensure no error appears**

9: Run the program

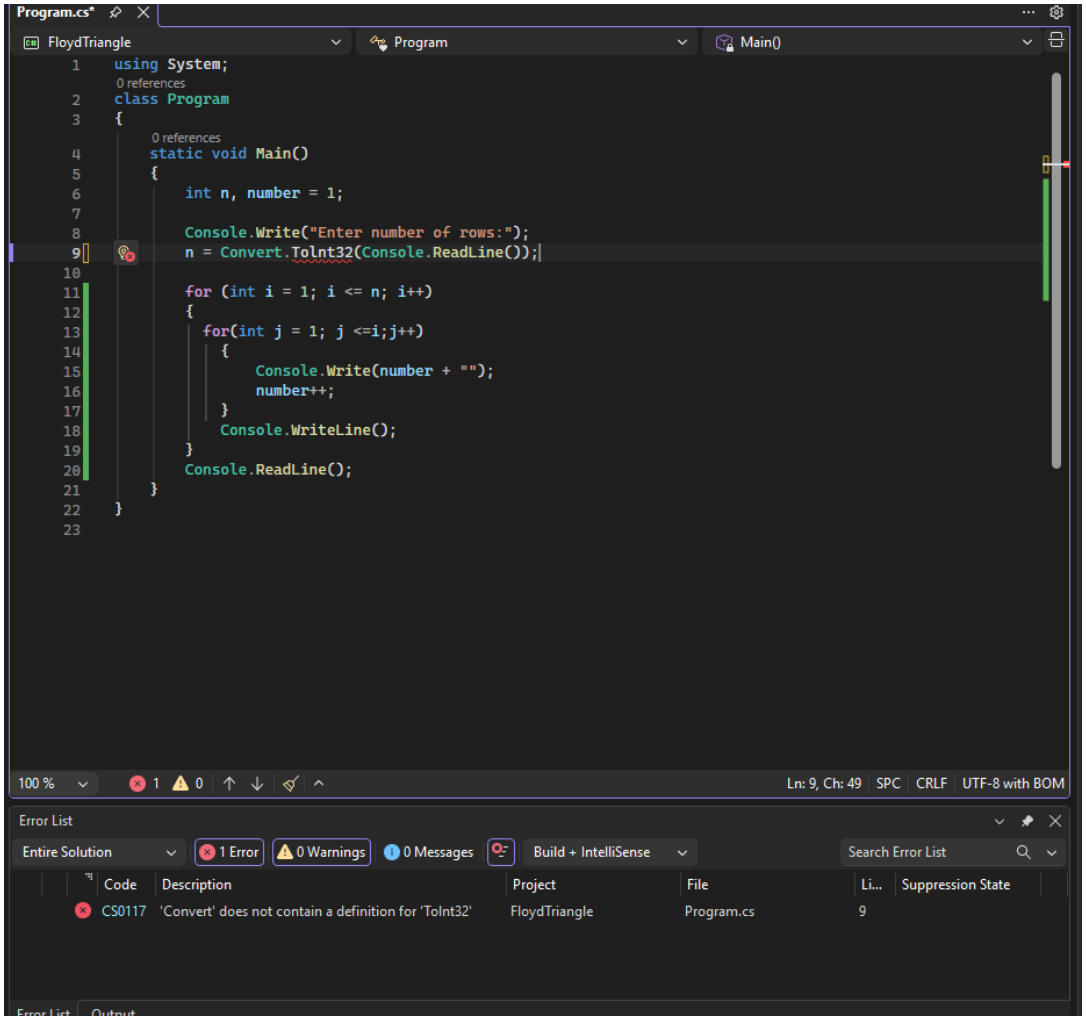
Press Ctrl + F5

Or click on Start

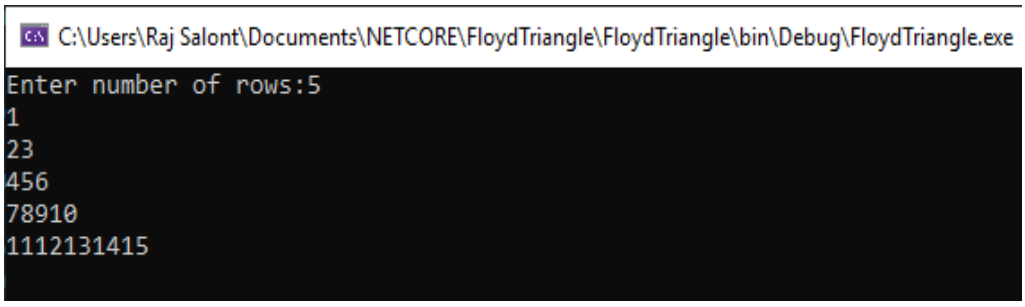
10: Output:

```
Enter number of rows:6
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
16 17 18 19 20 21
```

Errors



There was only spelling mistake in “ToInt” in line no. 9



Issue: The output was displaying all the numbers together without spaces.

Cause: In the Console.Write() statement, there was no space inside the double quotation marks after the variable.

Methods used:

1. Main()

- Entry point of the program.
- Execution starts from here

2. Console.Write()

- Displays text on the console without moving to the next line.

3. Console.WriteLine()

- Displays text or moves the cursor to the next line.

4. Console.ReadLine()

- Takes input from the user
- Also used at the

Difference between 1(a) and 1(b)

(a) MVC: Web application with browser UI, input boxes and buttons

(b) Console: Runs in black command window, takes input from keyboard

(C) Arrange names of cities in sorted orders

To write a c# program that accepts names of 10 cities from the user and displays them in sorted (alphabetical) order.

Steps:

1. Open Visual Studio 2026.
Start Visual Studio 2026 from the Start Menu.

2. Create a New Project.
Click on Create a new project. Select Console App (.NET). Give a logical name to the project. Click Next and then Create.

3. Open Program.cs File.
In the Solution Explorer, double-click on Program.cs to open the file.

4. Delete Existing Code.
Remove all the default code present in Program.cs.

5. Write / Paste the Program Code.
Write the C# code to accept 10 city names from the user, store them in an array, sort the array in ascending order, and display the sorted city names.
Save the file by pressing Ctrl + S.

6. Build the Application.
Go to Build -> Build Solution to check for any compilation errors.

7. Run the Application.
Press Ctrl + F5 to run the program.
Enter 10 city names when prompted.
The program will display the city names in sorted order.

Program:

```
using System;

class Program
{
   static void Main()
   {
      string[] cities = new string[10];

      Console.WriteLine("Enter 10 city names:");

      for(int i=0;i<10;i++)
```

```
        {
            cities[i] = Console.ReadLine();
        }

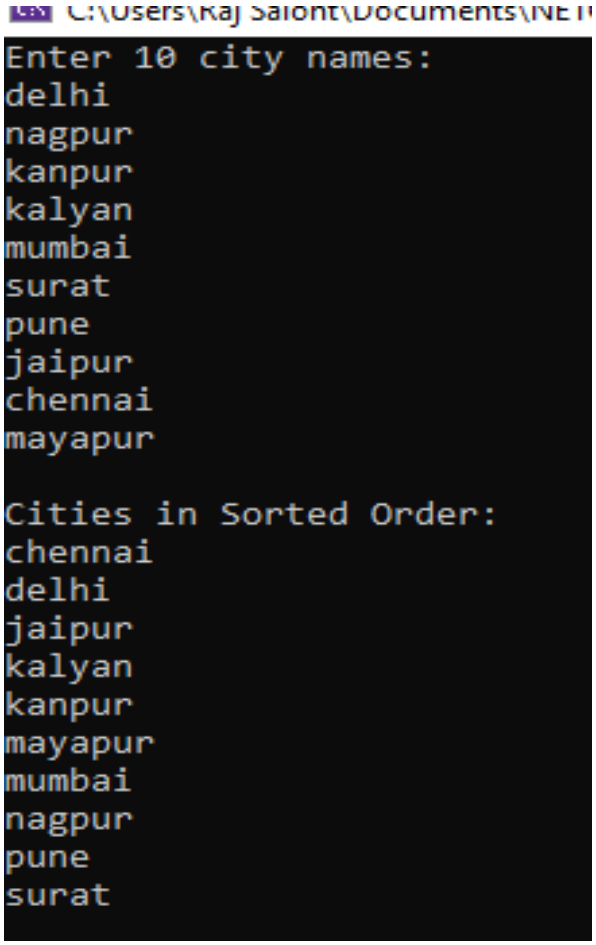
        Array.Sort(cities);

        Console.WriteLine("\nCities in Sorted Order:");

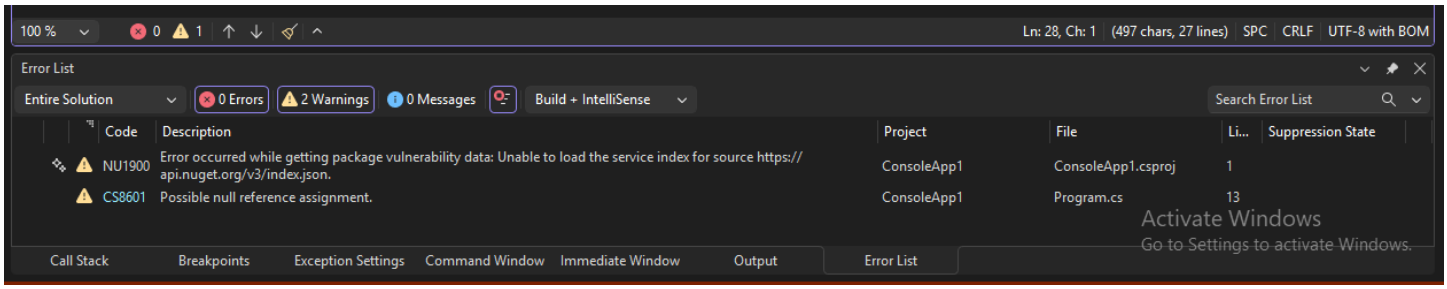
        foreach (string city in cities)
        {
            Console.WriteLine(city);
        }

        Console.ReadLine();
    }
}
```

8. Output



Error Faced:



Important Methods / Functions:

1. `static void Main()` - Entry Point Method

   The starting point of the program.

2. `string[] cities = new string[10];` - Array Declaration

   Creates an array to store 10 city names.

3. `Console.WriteLine("Enter 10 city names:")` - Method

   Displays a message to the user with a new line.

4. `cities[i] = Console.ReadLine();` - Method

   Stores each city name entered by the user into the array.

5. `Array.Sort(cities);` - Most Important Method

   Sorts the array elements in alphabetical order.

6. `foreach (string city in cities)` - Loop

   Iterates through the sorted array and prints each city.

7. `Console.WriteLine(city);` - Method

   Prints each city name on a new line.

PRACTICAL-2

(A) Write the program to use Function overloading feature of C#

To demonstrate function overloading in C# by creating multiple methods with the same name but different parameters.

Steps:

1. Open Visual Studio 2026.

2. Create -> Console App (.NET). Give logical Name to Project.

3. Open Program.cs.

4. Delete existing code.

5. Write the program code for Function Overloading.

6. Save the file by pressing Ctrl + S.

7. Run using Ctrl + F5.

Program.cs :

```
using System;
class Addition
{
    //Function with 2 parameters
    public void Add(int a, int b)
    {
        Console.WriteLine("Addition of 2 numbers=" + (a + b));
    }

    //Overloaded function with 3 Parameters
    public void Add(int a, int b, int c)
    {
        Console.WriteLine("Addition of 3 numbers=" + (a + b + c));
    }
}

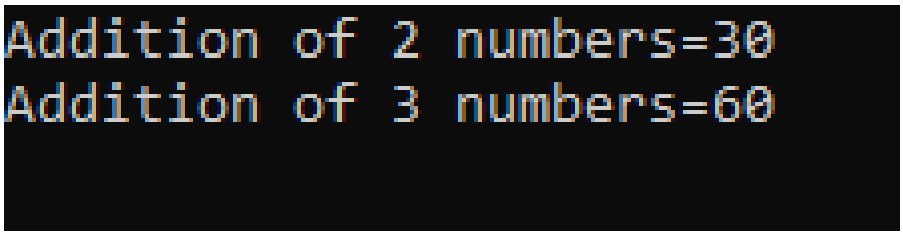
class Program
{
    static void Main()
    {
        Addition obj = new Addition();

        obj.Add(10, 20);
```

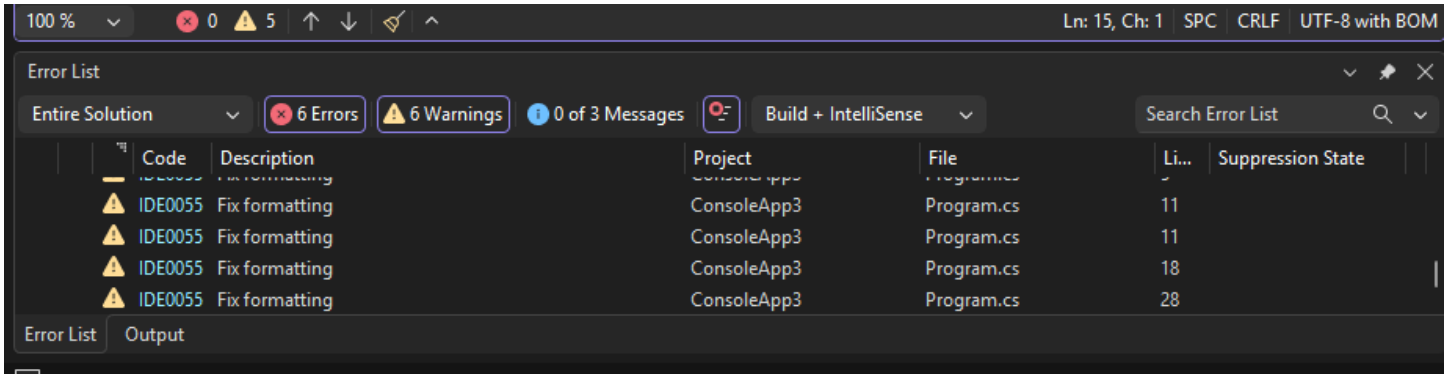
```
        obj.Add(10, 20, 30);

        Console.ReadLine();
    }
}
```

8. Output :



Errors Faced:



Important Methods / Functions:

1. `public void Add(int a, int b)` - Overloaded Method 1

   Takes 2 integer parameters and prints the sum of 2 numbers.

2. `public void Add(int a, int b, int c)` - Overloaded Method 2

   Takes 3 integer parameters and prints the sum of 3 numbers.

   Note: Same method name with different parameters demonstrates function overloading.

3. `static void Main()` - Entry Point Method

   Creates object of `Addition` class and calls both overloaded methods.

4. `obj.Add(10, 20);` - Method Call

   Calls the 2-parameter version of Add.

5. `obj.Add(10, 20, 30);` - Method Call

   Calls the 3-parameter version of Add.

6. `Console.WriteLine()` - Method

   Displays output on console.

7. `Console.ReadLine()` - Method

   Pauses the console screen until user presses Enter.

(B) Write the program to use Inheritance (all types) feature of C#.

Steps:

1. Open Visual Studio 2026.

2. Create → Console App (.NET). Give logical Name to Project.

3. Open Program.cs.

4. Delete existing code.

5. Write the program code for Single, Multilevel and Hierarchical Inheritance.

6. Save the file by pressing Ctrl + S.

7. Run using Ctrl + F5.

Program.cs

```
using System;

// Single Inheritance
class Animal
{
    public void Eat()
    {
        Console.WriteLine("Animal can Eat");
    }
}

class Dog : Animal
{
    public void Bark()
    {
        Console.WriteLine("Dog can Bark");
    }
}

// Multilevel Inheritance
class Puppy : Dog
{
    public void Weep()
    {
        Console.WriteLine("Puppy can Weep");
    }
}
```

```
// Hierarchical Inheritance
class Cat : Animal
{
   public void Meow()
   {
      Console.WriteLine("Cat can Meow");
   }
}

class Program
{
   static void Main()
   {
      // Single Inheritance
      Dog d = new Dog();
      d.Eat();
      d.Bark();

      // Multilevel Inheritance
      Puppy p = new Puppy();
      p.Eat();
      p.Bark();
      p.Weep();

      // Hierarchical Inheritance
      Cat c = new Cat();
      c.Eat();
      c.Meow();

      Console.ReadLine();
   }
}
```

8. Output :

```
Animal can Eat
Dog can Bark
Animal can Eat
Dog can Bark
Puppy can Weep
Animal can Eat
Cat can Meow
```

Important Methods / Functions :

1. `class Dog : Animal` - Single Inheritance

   Dog class inherits from Animal class.

2. `class Puppy : Dog` - Multilevel Inheritance

   Puppy inherits from Dog, which inherits from Animal.

3. `class Cat : Animal` - Hierarchical Inheritance

   Cat and Dog both inherit from Animal.

4. `public void Eat()` - Base Class Method

   Defined in Animal class. Inherited by Dog, Puppy, Cat.

5. `public void Bark()` - Derived Class Method

   Defined in Dog class.

6. `public void Weep()` - Derived Class Method

   Defined in Puppy class.

7. `public void Meow()` - Derived Class Method

   Defined in Cat class.

8. `Dog d = new Dog();` - Object Creation

   Creates object to demonstrate single inheritance.

9. `Puppy p = new Puppy();` - Object Creation

   Creates object to demonstrate multilevel inheritance.

10. `Cat c = new Cat();` - Object Creation

    Creates object to demonstrate hierarchical inheritance.

(C) Write the program to use Constructor overloading feature of C#

Steps:

1. Open Visual Studio 2026.

2. Create -> Console App (.NET). Give logical Name to Project.

3. Open Program.cs.

4. Delete existing code.

5. Write the program code for Constructor Overloading.

6. Save the file by pressing Ctrl + S.

7. Run using Ctrl + F5.

Program.cs

```
using System;

class Student
{
    //Default Constructor
    public Student()
    {
        Console.WriteLine("Default Constructor Called");
    }

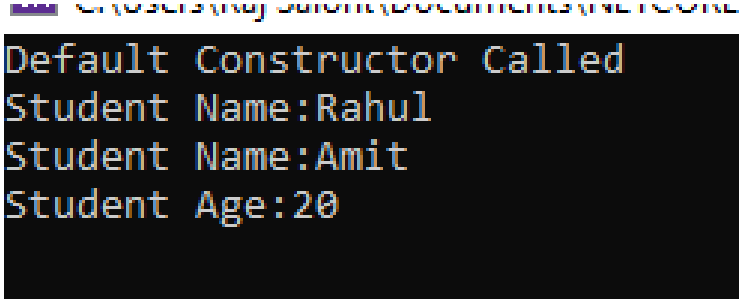
    //Parameterized Constructor
    public Student(string name)
    {
        Console.WriteLine("Student Name:" + name);
    }
    //Another Parameterized Constructor
    public Student(string name, int age)
    {
        Console.WriteLine("Student Name:" + name);
        Console.WriteLine("Student Age:" + age);
    }
}

class Program
{
    static void Main()
    {
        Student s1 = new Student();
```

```
        Student s2 = new Student("Rahul");
        Student s3 = new Student("Amit", 20);

        Console.ReadLine();
    }
}
```

8. Output



Important Methods / Functions:

1. `public Student()` - Default Constructor

   Constructor with no parameters. Prints "Default Constructor Called".

2. `public Student(string name)` - Parameterized Constructor

   Constructor with 1 parameter. Prints student name.

3. `public Student(string name, int age)` - Parameterized Constructor 2

   Constructor with 2 parameters. Prints student name and age.

   Note: Same constructor name with different parameters demonstrates constructor overloading.

4. `Student s1 = new Student();` - Object Creation

   Calls default constructor.

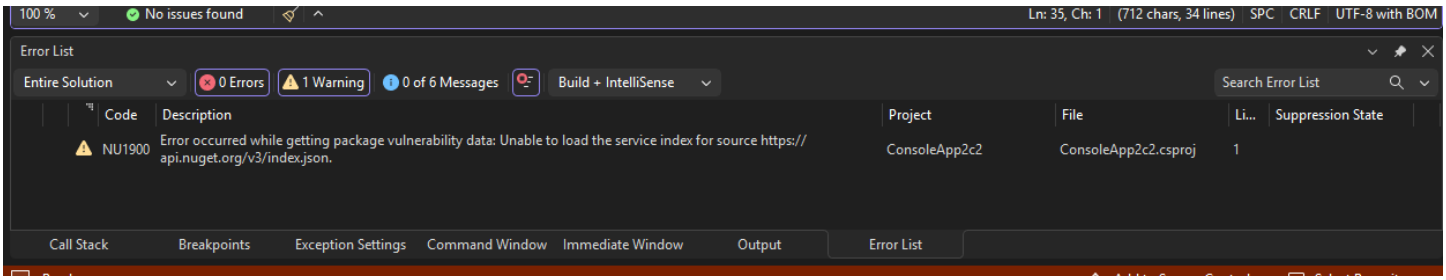
5. `Student s2 = new Student("Rahul");` - Object Creation

   Calls 1-parameter constructor.

6. `Student s3 = new Student("Amit", 20);` - Object Creation

   Calls 2-parameter constructor.

Error Faced:



(D) Write a program to use Interfaces feature of C#

Steps:

1. Open Visual Studio 2026.

2. Create →Console App (.NET). Give logical Name to Project.

3. Open Program.cs.

4. Delete existing code.

5. Write the program code for Interface.

6. Save the file by pressing Ctrl + S.

7. Run using Ctrl + F5.

Program.cs.

```
using System;

// Interface
interface IShape
{
   void Area();
}

// Class implementing interface
class Square : IShape
{
   int side = 5;

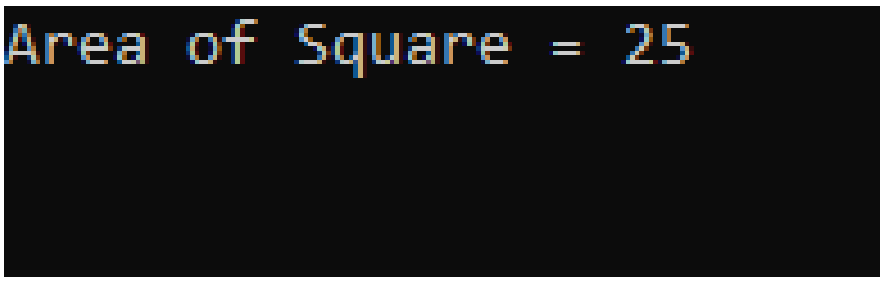
   public void Area()
   {
      Console.WriteLine("Area of Square = " + (side* side));
   }
}

class Program
{
   static void Main()
   {
      Square s = new Square();

      s.Area();
```

```
        Console.ReadLine();
    }
}
```

8. Output :



Important Methods / Functions:

1. `interface IShape` - Interface Declaration

   Declares an interface with method signature `void Area();`

2. `void Area();` - Interface Method

   Abstract method that must be implemented by classes.

3. `class Square : IShape` - Interface Implementation

   Square class implements IShape interface.

4. `public void Area()` - Method Implementation

   Provides implementation for the Area method from IShape.

5. `Square s = new Square();` - Object Creation

   Creates object of Square class.

6. `s.Area();` - Method Call

   Calls the implemented Area method.

(E) Write a program to use Delegates and events feature of C#

Steps:

1. Open Visual Studio 2026.

2. Create → Console App (.NET). Give logical Name to Project.

3. Open Program.cs.

4. Delete existing code.

5. Write the program code for Delegates and Events.

6. Save the file by pressing Ctrl + S.

7. Run using Ctrl + F5.

Program.cs.

```
using System;

// Delegate Declaration
public delegate void Message();

class Test
{
   // Event Declaration
   public event Message Notify;

   public void Show()
   {
      Console.WriteLine("Method inside Test class");

      // Raising Event
      Notify();
   }
}

class Program
{
   // Event Handling Method
   public static void Display()
   {
      Console.WriteLine("Event Executed Successfully");
   }
```

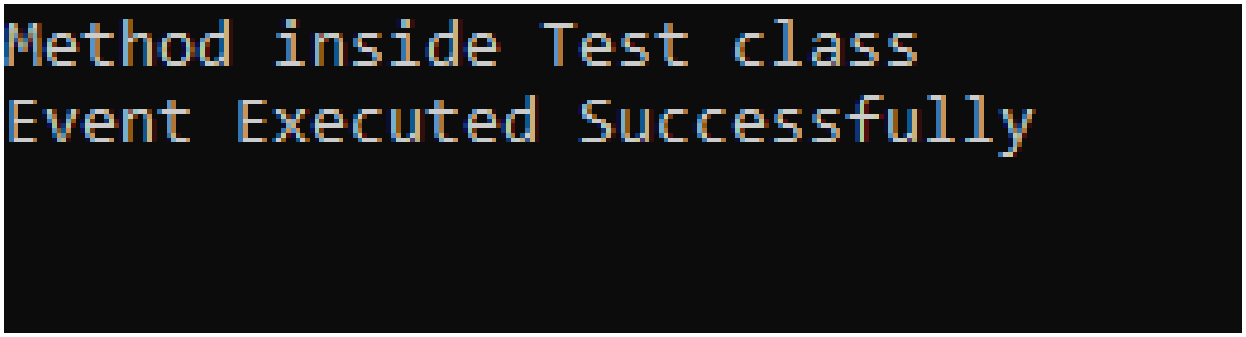
```
    static void Main()
    {
        Test t = new Test();

        // Attaching method to event
        t.Notify += Display;

        t.Show();

        Console.ReadLine();
    }
}
```

8. Output :



Important Methods / Functions:

1. `public delegate void Message();` - Delegate Declaration

   Declares a delegate that can reference methods with void return type and no parameters.

2. `public event Message Notify;` - Event Declaration

   Declares an event based on Message delegate.

3. `public void Show()` - Method

   Contains business logic and raises the event by calling `Notify();`

4. `public static void Display()` - Event Handler Method

   Method that gets executed when event is raised.

5. `t.Notify += Display;` - Event Subscription

   Attaches Display method to Notify event using += operator.

6. `t.Show();` - Method Call

   Calls method which triggers the event.

(F) Write a program to use Exception handling feature of C#

Steps:

1. Open Visual Studio 2026.

2. Create → Console App (.NET). Give logical Name to Project.

3. Open Program.cs.

4. Delete existing code.

5. Write the program code for Exception Handling.

6. Save the file by pressing Ctrl + S.

7. Run using Ctrl + F5.

Program.cs.

```
using System;

class Program
{
   static void Main()
   {
      int a, b, result;

      try
      {
         Console.Write("Enter First Number: ");
         a = Convert.ToInt32(Console.ReadLine());

         Console.Write("Enter Second Number: ");
         b = Convert.ToInt32(Console.ReadLine());

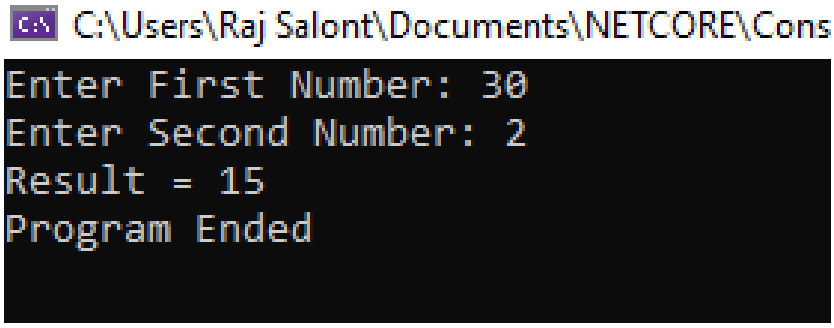
         result = a / b;

         Console.WriteLine("Result = " + result);
      }
      catch (DivideByZeroException)
      {
         Console.WriteLine("Cannot divide by zero");
      }
      catch (FormatException)
      {
         Console.WriteLine("Invalid Input");
```

```
        }
        finally
        {
            Console.WriteLine("Program Ended");
        }

        Console.ReadLine();
    }
}
```

8. Output :



Important Methods / Functions / Keywords:

1. `try` - Block

   Contains code that may cause an exception.

2. `catch (DivideByZeroException)` - Exception Handler

   Catches division by zero error.

3. `catch (FormatException)` - Exception Handler

   Catches invalid input format error.

4. `finally` - Block

   Code that always executes regardless of exception.

5. `Convert.ToInt32(Console.ReadLine())` - Method

   Converts user input string to integer.

6. `result = a / b;` - Operation

   Division operation that may throw DivideByZeroException.

7. `Console.Write()` - Method

   Takes input from user.

PRACTICAL-3

(A) Create methods add(), multiply(),subtract(),divide() with suitable parameters and call these methods using concept of C# delegate.

Steps:

1. Open Visual Studio 2026.

2. Create → Console App (.NET). Give logical Name to Project.

3. Open Program.cs.

4. Delete existing code.

5. Write the program to create a class with two methods having same name but different number of parameters.

6. In Main method, create object of the class and call both overloaded methods.

7. Save the file by pressing Ctrl + S.

8. Run using Ctrl + F5.

9. Observe the output on console screen.

Program.cs

```
using System;

// Delegate Declaration
public delegate void Calculation(int a, int b);

class Arithmetic
{
   public void Add(int a, int b)
   {
      Console.WriteLine("Addition = " + (a + b));
   }

   public void Subtract(int a, int b)
   {
      Console.WriteLine("Subtraction = " + (a - b));
   }

   public void Multiply(int a, int b)
   {
```

```
        Console.WriteLine("Multiplication = " + (a* b));
    }

    public void Divide(int a, int b)
    {
        Console.WriteLine("Division = " + (a / b));
    }
}

class Program
{
    static void Main()
    {
        Arithmetic obj = new Arithmetic();

        Calculation c;

        c = obj.Add;
        c(20, 10);

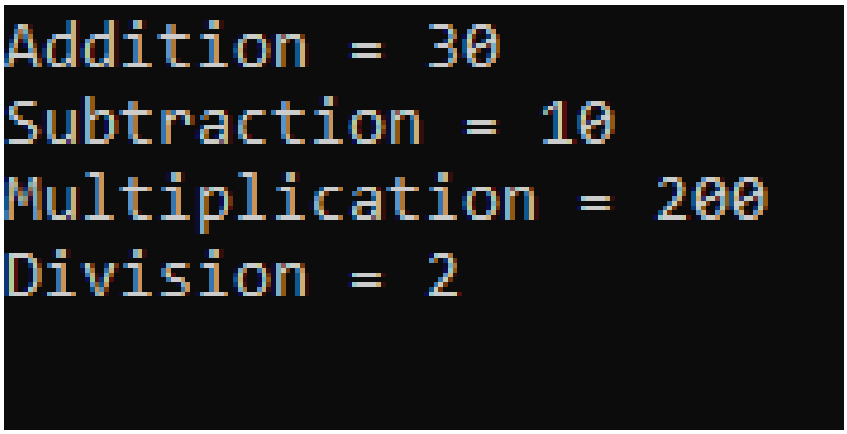
        c = obj.Subtract;
        c(20, 10);

        c = obj.Multiply;
        c(20, 10);

        c = obj.Divide;
        c(20, 10);

        Console.ReadLine();
    }
}
```

10. Output



Important Methods / Functions:

1. public delegate void Calculation(int a, int b)
2. public void Add(int a, int b)
3. public void Subtract(int a, int b)
4. public void Multiply(int a, int b)
5. public void Divide(int a, int b)
6. static void Main()
7. Console.WriteLine()
8. Console.ReadLine()

(B) Write a program using multicast delegate.

Steps:

1. Open Visual Studio 2026.

2. Create -> Console App (.NET). Give logical Name to Project.

3. Open Program.cs.

4. Delete existing code.

5. Write the program to declare a delegate and create a class with multiple methods.

6. In Main method, create object and multicast delegate instance. Add multiple methods to the delegate and invoke it to call all methods together.

7. Save the file by pressing Ctrl + S.

8. Run using Ctrl + F5.

9. Observe the output on console screen.

Program.cs.

```
using System;

// Delegate Declaration
public delegate void Demo();

class Test
{
    public void Show()
    {
        Console.WriteLine("Show Method Called");
    }

    public void Display()
    {
        Console.WriteLine("Display Method Called");
    }

    public void Print()
    {
        Console.WriteLine("Print Method Called");
    }
```

```
}

class Program
{
   static void Main()
   {
      Test t = new Test();

      // Creating Multicast Delegate
      Demo d;

      d = t.Show;
      d += t.Display;
      d += t.Print;

      // Calling all methods together
      d();

      Console.ReadLine();
   }
}
```

Output

```
Show Method Called
Display Method Called
Print Method Called
```

Important Methods / Functions:

1. public delegate void Demo()
2. public void Show()
3. public void Display()
4. public void Print()
5. static void Main()
6. Console.WriteLine()
7. Console.ReadLine()

(C) Create a class Bank Account with Account Number and Balance. Implement property for Balance with validation (Balance cannot be negative).

Steps:

1. Open Visual Studio 2026.

2. Create -> Console App (.NET). Give logical Name to Project.

3. Open Program.cs.

4. Delete existing code.

5. Write the program to create a BankAccount class with private field and public properties.

6. Implement validation in set property of Balance so that negative value cannot be assigned.

7. In Main method, create object, set AccountNumber and Balance, and try to set negative balance to test validation.

8. Save the file by pressing Ctrl + S.

9. Run using Ctrl + F5.

10. Observe the output on console screen.

Program.cs.

```
using System;

class BankAccount
{
    // Field
    private double balance;

    // Property
    public double Balance
    {
        get
        {
            return balance;
        }

        set
        {
```

```
            if (value >= 0)
            {
                balance = value;
            }
            else
            {
                Console.WriteLine("Balance cannot be negative");
            }
        }
    }

    // Property for Account Number
    public int AccountNumber { get; set; }
}

class Program
{
    static void Main()
    {
        BankAccount b = new BankAccount();

        b.AccountNumber = 101;

        b.Balance = 5000;

        Console.WriteLine("Account Number: " + b.AccountNumber);

        Console.WriteLine("Balance: " + b.Balance);

        // Negative Balance Test
        b.Balance = -2000;

        Console.ReadLine();
    }
}
```

Output:

```
Account Number: 101
Balance: 5000
Balance cannot be negative
```

Important Methods / Functions:

1. public int AccountNumber { get; set; }
2. public double Balance { get; set; }
3. static void Main()
4. Console.WriteLine()
5. Console.ReadLine()

PRACTICAL-4

(A) Create an ASP.NET Web Form application using simple server controls.

Controls Used:

1. Label and TextBox for Student Name
2. RadioButton for Gender (Male/Female)
3. DropDownList for Course (BSc IT, BSc CS, BCA)
4. CheckBox for Terms & Conditions
5. Calendar control for Date
6. Button for Submit
7. Label for Output

Steps:

1. Open Visual Studio 2026.

2. Create -> ASP.NET Web Application (.NET Framework).

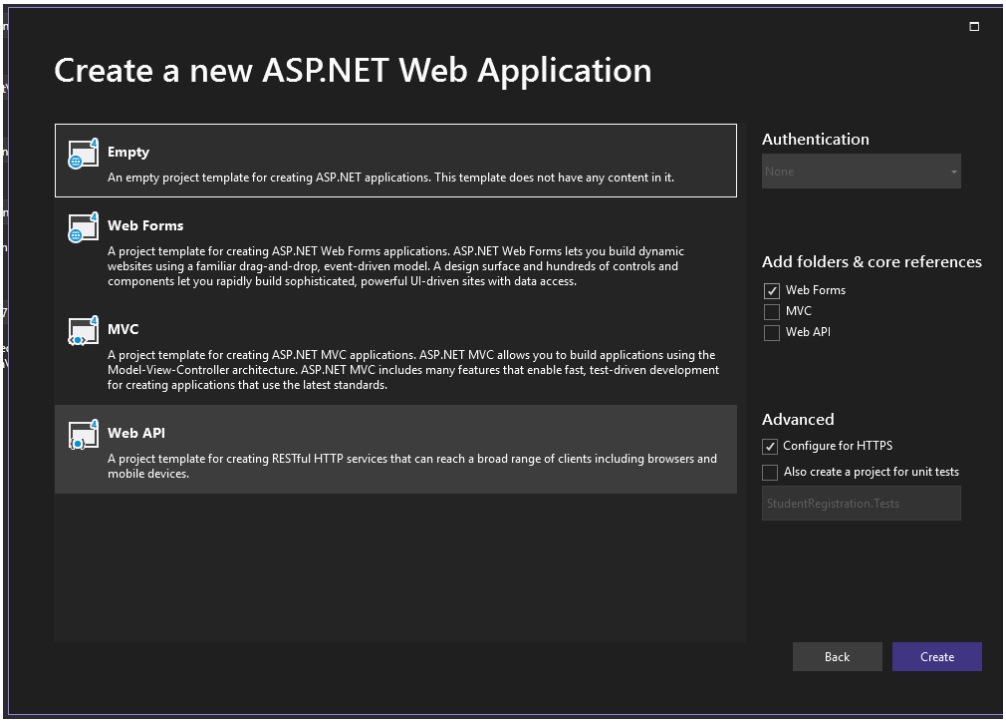
3. Configure Project

Enter:

Field	**Value**
Project Name	StudentRegistration
Location	Select folder

Click: Create

4. Select Template: Empty. Check Web Forms. Click Create.



5. Right-click Project -> Add -> Web Form.

Name: `StudentForm.aspx`.

Click: Add

Visual Studio creates:

- StudentForm.aspx
- StudentForm.aspx.cs

6. Open `StudentForm.aspx` and click Design View.

7. Open Toolbox -> View Menu -> Toolbox.

8. Add Controls from Toolbax

Select Standard Tab in toolbox and Drag controls one by one.

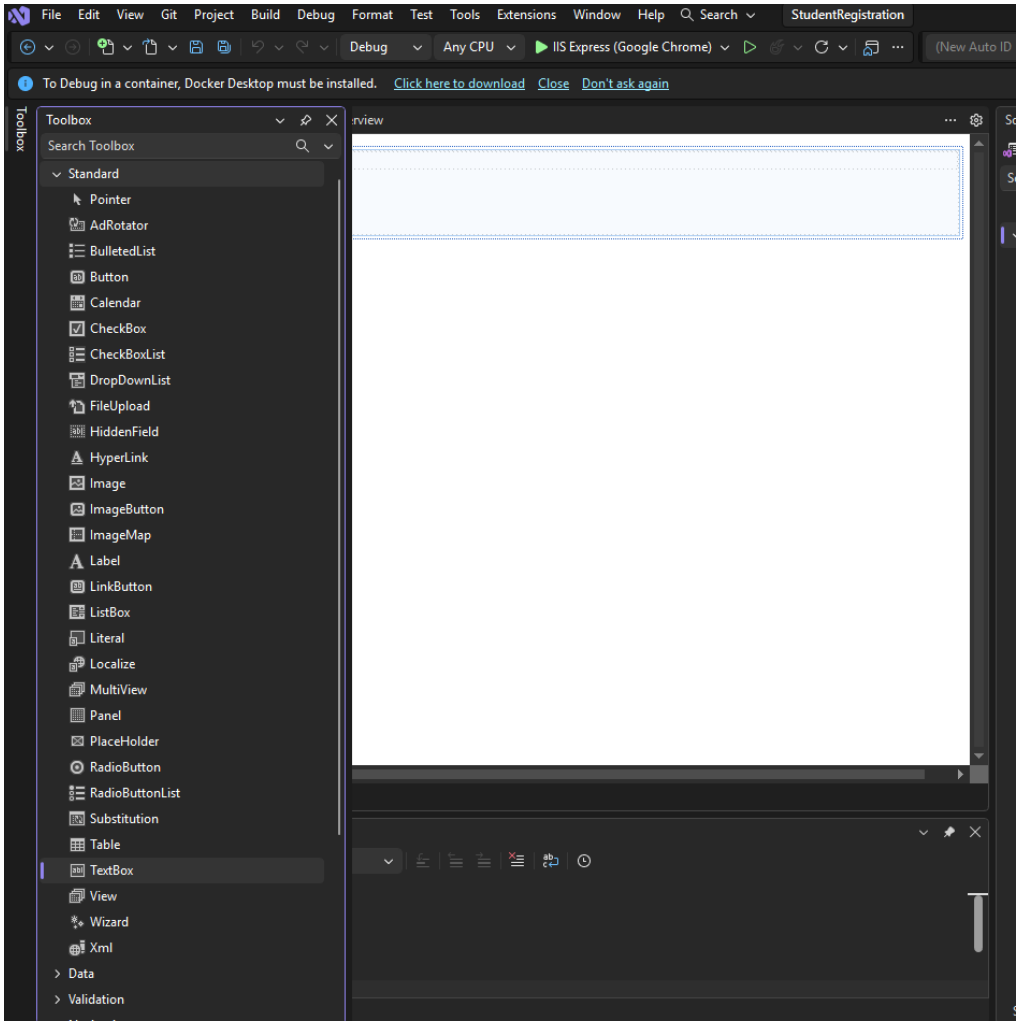
9. Add Student Name Controls

Drag:

- Label
- TextBox

After dragging the control on web page Select the Control and Right Click on it, Select

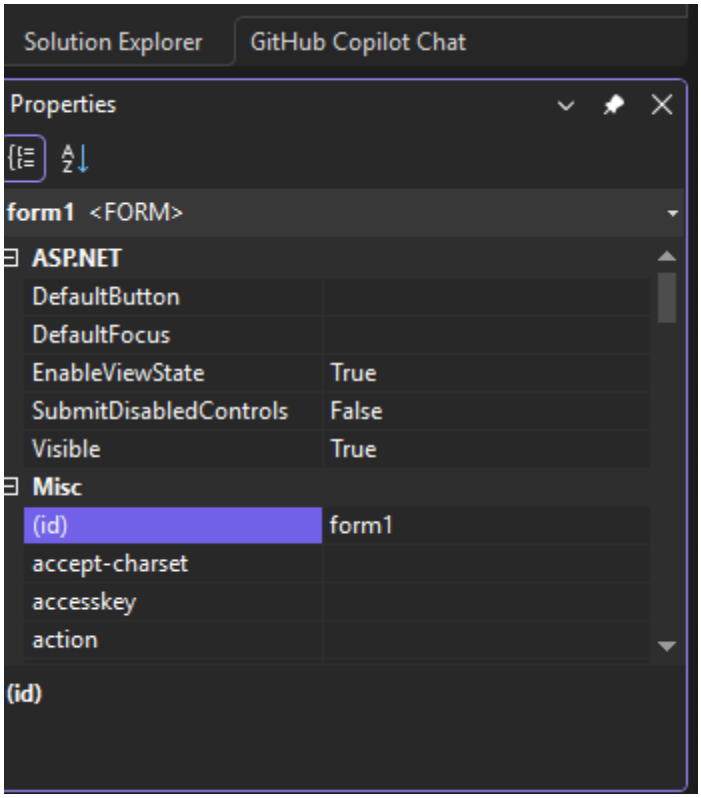
Properties Option.



In Properties window give following values:

Label: (Property: Text ->Value: Student Name)

TextBox: (Property:ID->Value: txtName)



10. Add Gender Controls

Drag

- Label
- RadioButton
- RadioButton

Label: (Property: Text -> Value: Gender)

RadioButton 1: (Property: ID -> rbFemale, Text -> Female, GroupName ->Gender)

RadioButton 2 : (Property: ID -> rbMale , Text -> male, GroupName ->Gender)

11. Add Course DropDownList

Drag :

- Label
- DropDownList

Label: (Text->Course)

DropDownList : (Id->ddlCourse, Text-> Female, GroupName->Gender)

12. Add items to DropDownList

Click DropDownList-> Click on Small Arror on Right Top Corner of DropDown Box -> Select Edit Items -> In new Window ->....Add items:

- BSc IT
- BSc CS
- BCA

13. Add CheckBox

Drag CheckBox

Set:

CheckBox: (ID->chkTerms,Text->Accept Terms & Conditions)

14. Add Calender

Drag:

Calendar

(Text->calDate)

15: Add Button
Drag Button.
Set:

Label: (: ID → btnSubmit, Text → Submit)

16: Add Output Label
Drag Label.
Set:

Label: (ID → lblOutput, Text → (empty))

17: Double Click Submit Button
Double-click Submit Button.
Visual Studio opens C# Code Editor.

18: Write Code
Inside btnSubmit_Click() write:

```
Protected Sub btnSubmit_Click(sender As Object, e As EventArgs) Handles btnSubmit.Click

    Dim gender As String = ""
```

```
    If RbMale.Checked Then
        gender = "Male"
    ElseIf rbFemale.Checked Then
        gender = "Female"
    End If

    If chkTerms.Checked Then
        lblOutput.Text = "Student Name : " & txtName.Text &
                 "<br/>Gender : " & gender &
                 "<br/>Course : " & ddlCourse.SelectedValue &
                 "<br/>Date : " & calDate.SelectedDate.ToShortDateString()
    Else
        lblOutput.Text = "Please accept Terms & Conditions"
    End
```

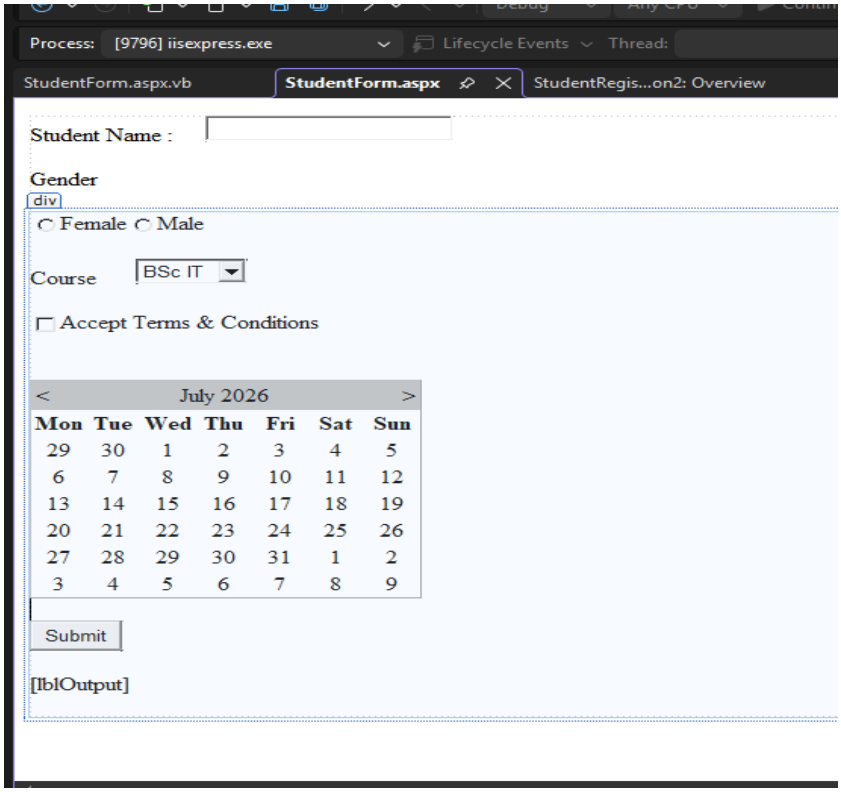
19: Save Project

20: Run Application

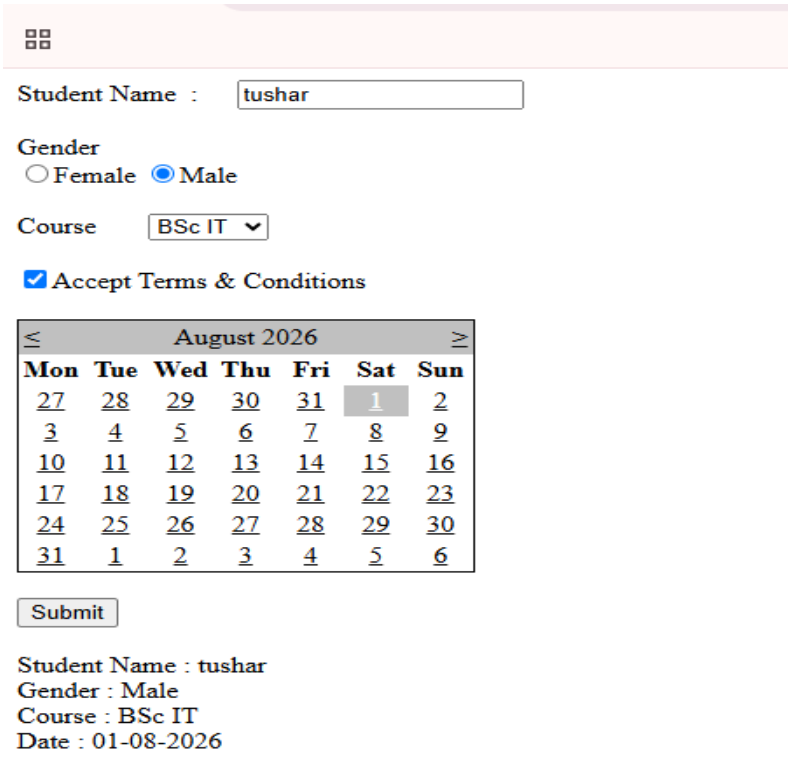
Ctrl + F5

Or Click:

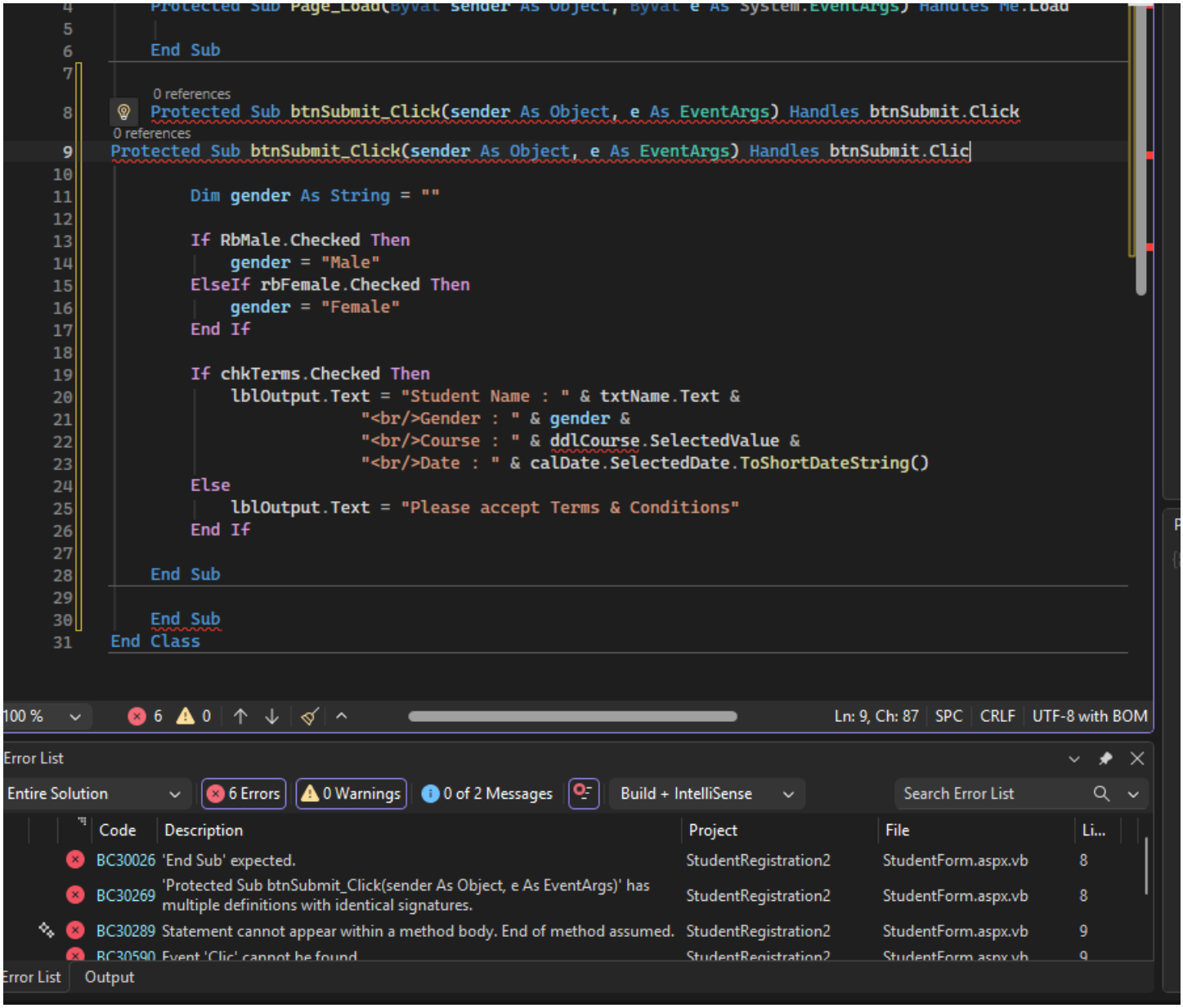
IIS Express



OutPut


Student Name : tushar
Gender
Female Male
Course BSc IT
Accept Terms & Conditions
August 2026
Mon Tue Wed Thu Fri Sat Sun
27 28 29 30 31 1 2
3 4 5 6 7 8 9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30
31 1 2 3 4 5 6
Submit
Student Name : tushar
Gender : Male
Course : BSc IT
Date : 01-08-2026

Error faced:


End Sub
0 references
Protected Sub btnSubmit_Click(sender As Object, e As EventArgs) Handles btnSubmit.Click
0 references
Protected Sub btnSubmit_Click(sender As Object, e As EventArgs) Handles btnSubmit.Clic
Dim gender As String = ""
If RbMale.Checked Then
gender = "Male"
ElseIf rbFemale.Checked Then
gender = "Female"
End If
If chkTerms.Checked Then
lblOutput.Text = "Student Name : " & txtName.Text &
"
Gender : " & gender &
"
Course : " & ddlCourse.SelectedValue &
"
Date : " & calDate.SelectedDate.ToShortDateString()
Else
lblOutput.Text = "Please accept Terms & Conditions"
End If
End Sub
End Sub
End Class
100 %
6
0
Ln: 9, Ch: 87
SPC
CRLF
UTF-8 with BOM
Error List
Entire Solution
6 Errors
0 Warnings
0 of 2 Messages
Build + IntelliSense
Search Error List
Code
Description
Project
File
Li...
BC30026 'End Sub' expected.
StudentRegistration2
StudentForm.aspx.vb
8
BC30269 'Protected Sub btnSubmit_Click(sender As Object, e As EventArgs)' has multiple definitions with identical signatures.
StudentRegistration2
StudentForm.aspx.vb
8
BC30289 Statement cannot appear within a method body. End of method assumed.
StudentRegistration2
StudentForm.aspx.vb
9
Error List
Output

(B) Create a Registration form to demonstrate use of various validation controls.

Steps:

1. Open Visual Studio 2026.

2. Create ->ASP.NET Web Application (.NET Framework).

3. Select Template: Empty. Check Web Forms. Click Create.

4. Right-click Project -> Add -> Web Form. Name: `Registration.aspx`.

5. Open `Registration.aspx` in Design View.

6. Add TextBox, Button and Label controls from Toolbox for Name, Email, Age, Password, Confirm Password and Output.

Add Label and Text	Control Type	ID
Name:	TextBox	txtName
Emai:l	TextBox	txtEmail
Age:	TextBox	txtAge
Password:	TextBox	txtPassword
Confirm Password:	TextBox	txtConfirmPassword
Register	Button	btnSubmit
Output	Label	lbloutput

7. Add Validation Controls

Add these Validators from Toolbox under Validation Arrow

RequiredFieldValidator

Add for Name TextBox

Property	Value
ControlToValidate	txtName
ErrorMessage	Name Required

RegularExpressionValidator

Property	Value
ControlToValidate	**txtEmail**
ErrorMessage	**Invalid Email**
ValidationExpression	**\\w+([_+.']\w+)@\w+([-.]\w+)\.\w+([-.]\w+)***

RangeValidator

Add for Age TextBox

Property	**Value**
ControlToValidate	txtAge
Minimum value	18
Maxmimum Value	60
Type	Integer
ErrorMessage	Age must be between 18 and 60

CompareValidator

Add for Confirm Password TextBox

Property	**Value**
ControlToValidate	txtConfirmPassword
ControlToCompare	txtPassword
ErrorMessage	Password does not match

8. Double Click Register Button

Write code

```
protected void btnSubmit_Click(object sender, EventArgs e)
{
    if (Page.IsValid)  // Agar saare validation pass ho gaye
    {
        lblOutput.Text = "Registration Successful";
    }
}
```

Note: Before running the programme open the web.config file from file solution explorer and add the following code

Inside and immediately the <**Configuration**>Tag add follow lines.

```
<appSettings>
  <add key="ValidationSettings:UnobtrusiveValidationMode" value="None" />
</appSettings>
```

9. Run Program:

10. Output

Name : simi

Email : Invalid Email

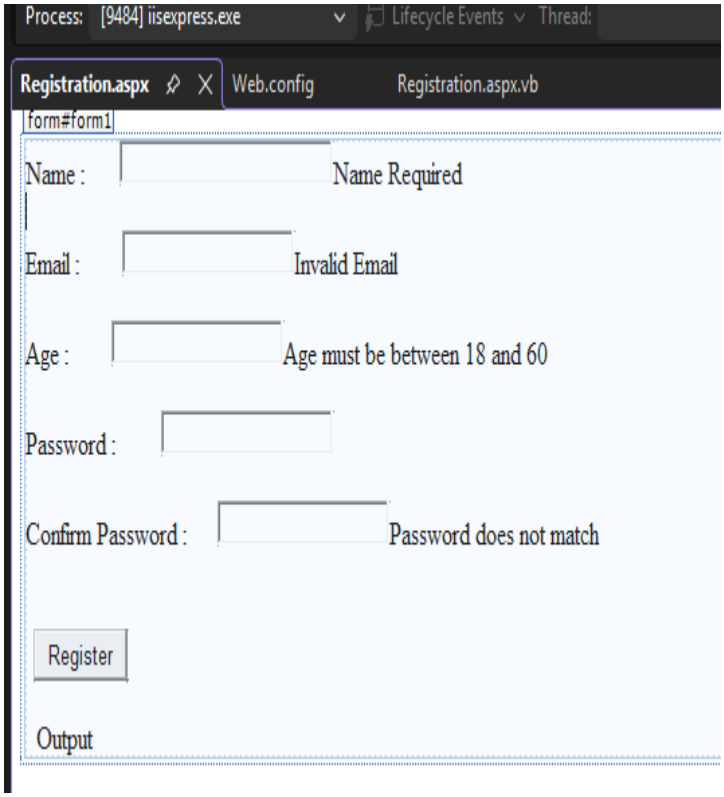
Age : 20

Password : 12345678

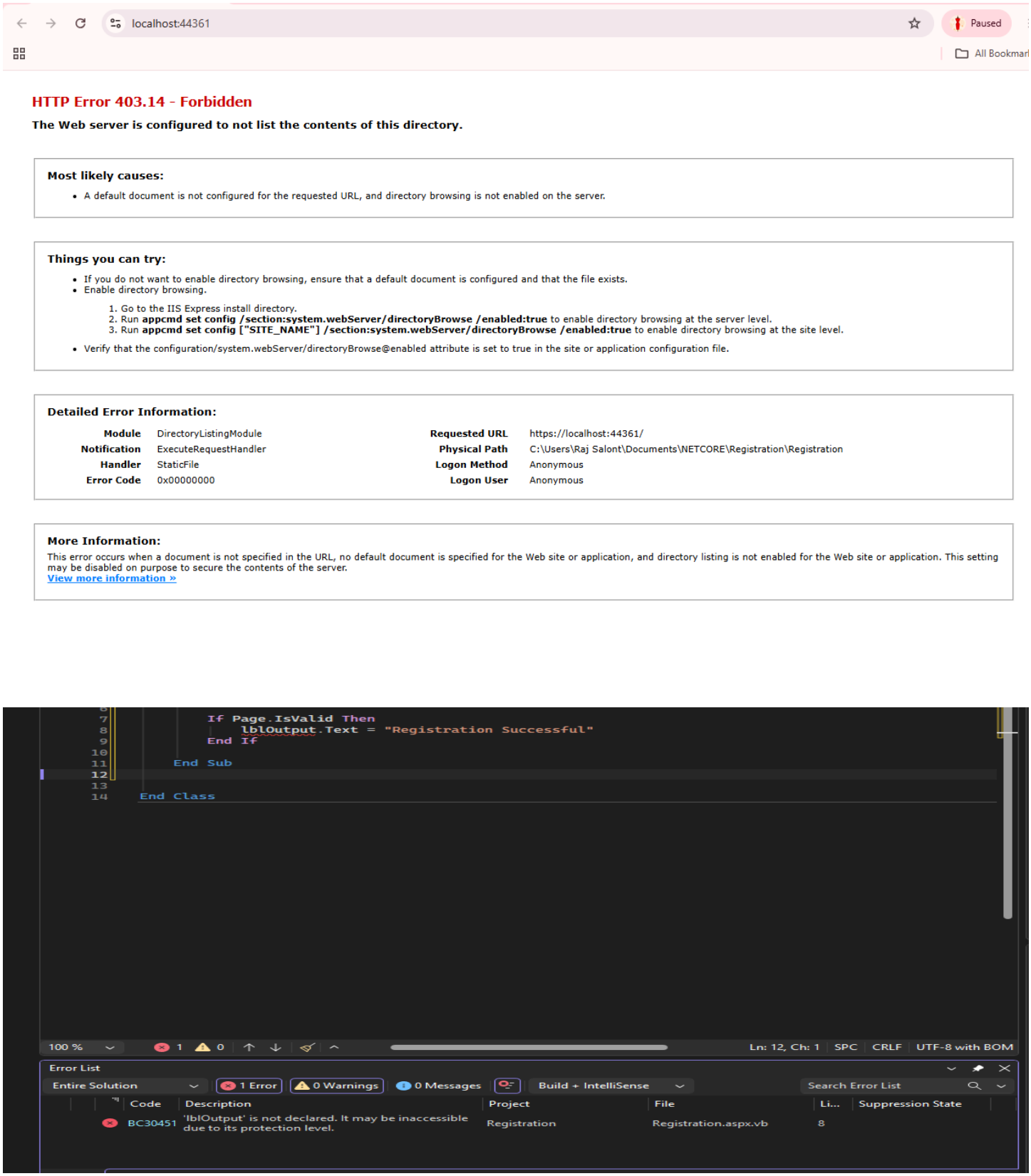
Confirm Password : 12345678

Register

Registration Successful



Error Faced:


localhost:44361
Paused
All Bookmarl
HTTP Error 403.14 - Forbidden
The Web server is configured to not list the contents of this directory.
Most likely causes:
A default document is not configured for the requested URL, and directory browsing is not enabled on the server.
Things you can try:
If you do not want to enable directory browsing, ensure that a default document is configured and that the file exists.
Enable directory browsing.
1. Go to the IIS Express install directory.
2. Run appcmd set config /section:system.webServer/directoryBrowse /enabled:true to enable directory browsing at the server level.
3. Run appcmd set config ["SITE_NAME"] /section:system.webServer/directoryBrowse /enabled:true to enable directory browsing at the site level.
Verify that the configuration/system.webServer/directoryBrowse@enabled attribute is set to true in the site or application configuration file.
Detailed Error Information:
Module DirectoryListingModule
Notification ExecuteRequestHandler
Handler StaticFile
Error Code 0x00000000
Requested URL https://localhost:44361/
Physical Path C:\Users\Raj Salont\Documents\NETCORE\Registration\Registration
Logon Method Anonymous
Logon User Anonymous
More Information:
This error occurs when a document is not specified in the URL, no default document is specified for the Web site or application, and directory listing is not enabled for the Web site or application. This setting may be disabled on purpose to secure the contents of the server.
View more information »

If Page.IsValid Then
lblOutput.Text = "Registration Successful"
End If
End Sub
End Class
100 %
1
0
Ln: 12, Ch: 1
SPC
CRLF
UTF-8 with BOM
Error List
Entire Solution
1 Error
0 Warnings
0 Messages
Build + IntelliSense
Search Error List
Code
Description
Project
File
Li...
Suppression State
BC30451
'lblOutput' is not declared. It may be inaccessible due to its protection level.
Registration
Registration.aspx.vb
8

(C) Create a Web Form to demonstrate use of Adrotator Control.

Steps:

1. Create Project

1. Open Visual Studio 2026.
2. Create → **ASP.NET Web Application (.NET Framework)**.
3. Select **Empty** and check **Web Forms**.
4. Click **Create**.

2. Add Web Form

1. Right-click Project → Add → **Web Form**.
2. Name: AdRotatorDemo.aspx

3. Add XML File

1. Right-click Project → Add → **New Item**.
2. Select **XML File**.
3. Name: Ads.xml.

4. Write XML Code

- Open Ads.xml.
- Add the given AdRotator XML code.

```
<Advertisements>

  <Ad>

    <ImageUrl>Images/ad1.jpg</ImageUrl>

    <NavigateUrl>https://www.google.com/</NavigateUrl>

    <AlternateText>Google Ad</AlternateText>

    <Impressions>50</Impressions>

  </Ad>


  <Ad>

    <ImageUrl>Images/ad2.jpg</ImageUrl>

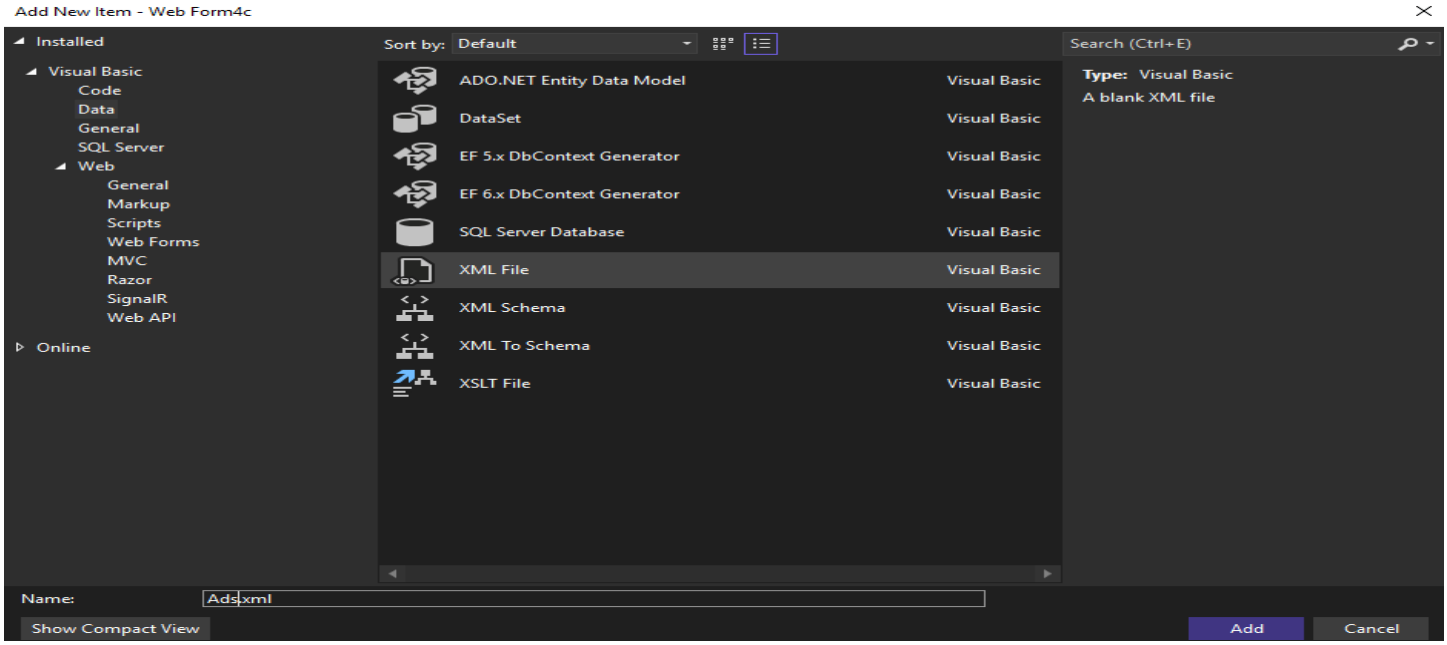
    <NavigateUrl>https://www.microsoft.com/</NavigateUrl>
```

```
  <AlternateText>Microsoft Ad</AlternateText>
  <Impressions>50</Impressions>
 </Ad>
</Advertisements>
```

5. Create Images Folder

1. Right-click Project → Add → **New Folder**.
2. Name: Images.
3. Add:
   - ad1.jpg
   - ad2.jpg

(Download from internet and then add inside folder or copy and paste)



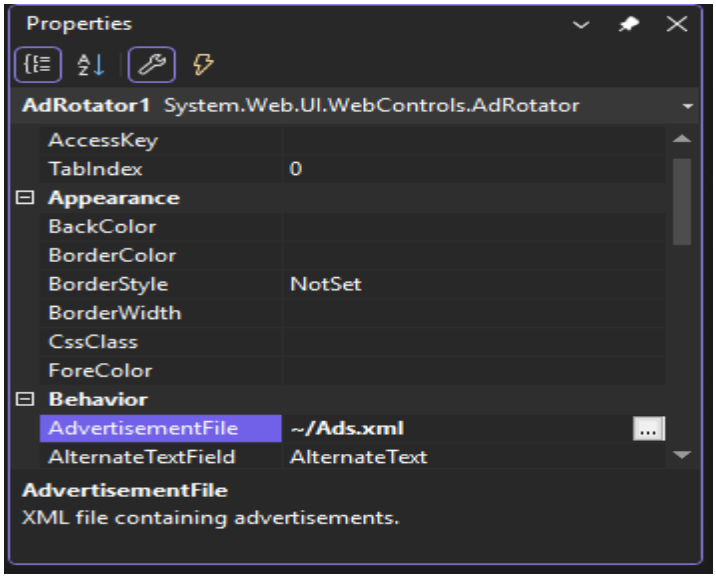
6. Open Web Form Design

- Open AdRotatorDemo.aspx from Solution Explorer.

7. Add AdRotator Control

1. From Toolbox, drag **AdRotator** onto the Web Form.
2. Set **AdvertisementFile** to ~/Ads.xml.

Property	Value
AdvertisementFile	~/Ads.xml

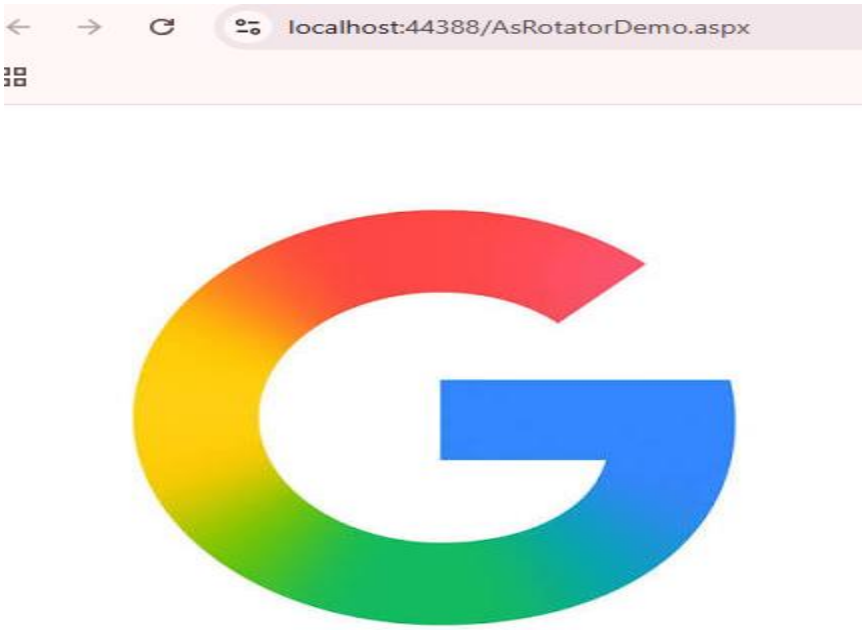


8. Save All Files

9. Run Application

- Run using **IIS Express**.
- The advertisement should appear on the Web Form.

10. Output



PRACTICAL-5

(A) Create a web application to demonstrate the use of different types of cookies.

Steps:

Steps:

1. Create Project

Step 1: Open Visual Studio 2026

Step 2: Create → `ASP.NET Web Application (.NET Framework)`

Step 3: Select: `Empty` and `Check Web Forms`

Step 4: Click Create

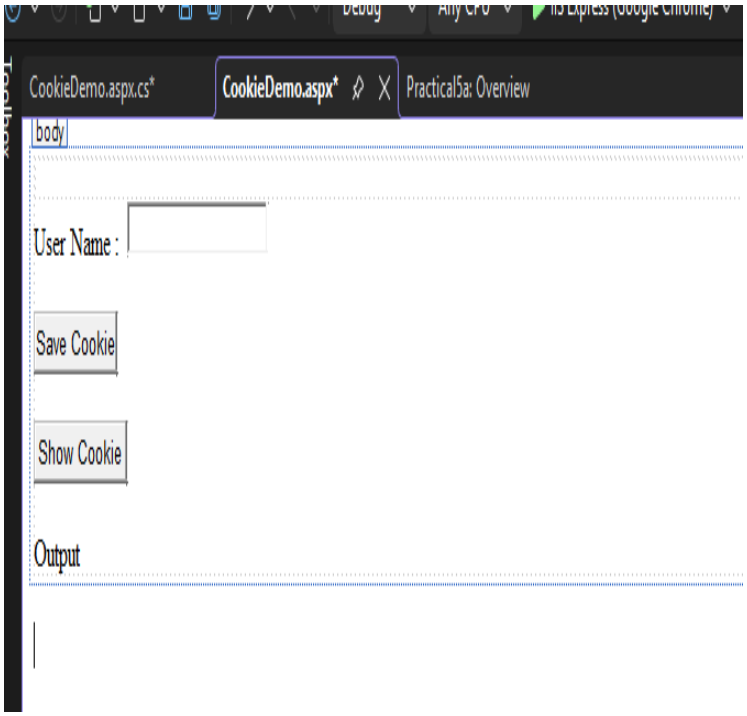
2. Add Web Form

Right-click Project → `Add` → `Web Form`

Name: `CookieDemo.aspx`

3. Add Controls

Label Text	Control Type	ID
User Name	TextBox	txtName
Save Cookie	Button	btnSave
Show Cookies	Button	btnShow
Output	Label	lblOutput



4. Write Code for Save Cookie Button

Double Click `btnSave` Button

```
protected void btnSave_Click(object sender, EventArgs e)

{
    // Session Cookie - Browser band hone par delete ho jayega
    HttpCookie sessionCookie = new HttpCookie("SessionUser");
    sessionCookie.Value = txtName.Text;
    Response.Cookies.Add(sessionCookie);

    // Persistent Cookie - 7 din tak rahega
    HttpCookie persistentCookie = new HttpCookie("PersistentUser");
    persistentCookie.Value = txtName.Text;
    persistentCookie.Expires = DateTime.Now.AddDays(7);
    Response.Cookies.Add(persistentCookie);

    lblOutput.Text = "Cookies Saved Successfully";
}
}
```

5. Write Code for Show Cookie Button

```
protected void btnShow_Click(object sender, EventArgs e)

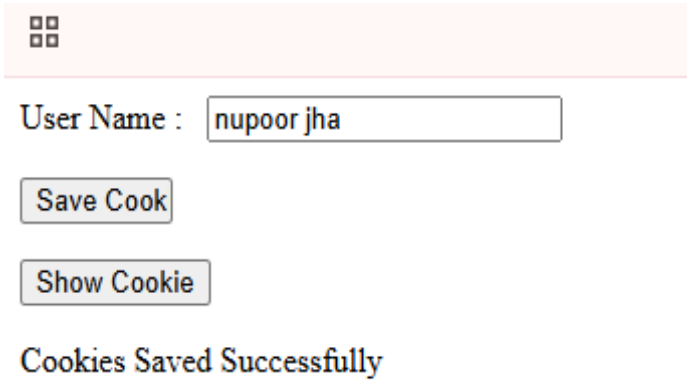
{
    string sessionData = Request.Cookies["SessionUser"].Value;
    string persistentData = Request.Cookies["PersistentUser"].Value;

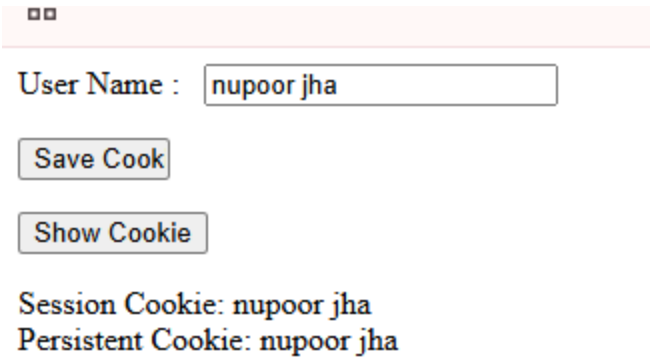
    lblOutput.Text = "Session Cookie: " + sessionData +
                "<br/>Persistent Cookie: " + persistentData;
}
```

6. Run Application

Press `F5` to run

7. Output





Important Methods/Properties/Classes Used:

1. `new HttpCookie("CookieName")` - To create a new cookie object
2. `cookie.Value` - To set or get the value of cookie
3. `cookie.Expires` - To set expiration date for persistent cookie
4. `Response.Cookies.Add(cookie)` - To add cookie to response and send to browser
5. `Request.Cookies["CookieName"]` - To read cookie value from request
6. `lblOutput.Text` - To display text on label control
7. `DateTime.Now.AddDays(7)` - To set cookie expiration after 7 days
8. `protected void btnSave_Click(object sender, EventArgs e)` - Event handler for Save Cookie button
9. `protected void btnShow_Click(object sender, EventArgs e)` - Event handler for Show Cookie button

(B) Create a product Table in Built in SQL Server of Visual Studio and perform Insert Operation using Entity Framework Core.

Steps:

1: Open Visual Studio 2026 and create a new project

2: Create MVC Project.

Select ASP.NET Core Web App Model-View-Controller

3: Configure Project. Set Project Name and Framework

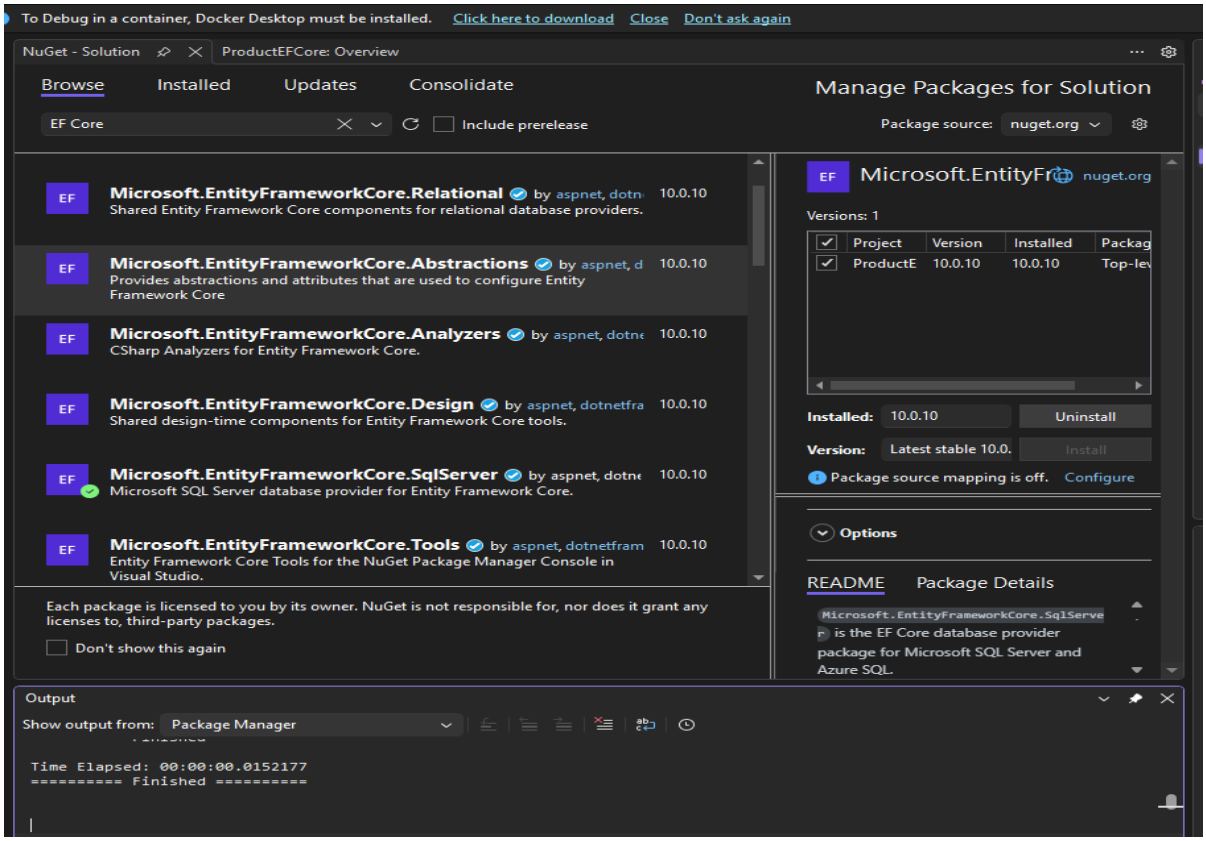
Field	Value
Project Name	ProductEFCore
Framework	.Net 8 or Higher

4: Install EF Core Packages using NuGet Package Manager. Install Microsoft.EntityFrameworkCore.SqlServer,

Microsoft.EntityFrameworkCore

Microsoft.EntityFrameworkCore.Tools

Microsoft.VisualStudio.Web.CodeGeneration.Design



5: Create Product Model

Models->Add-> Class

Name: Products.cs

In Models folder with properties Id, ProductName, Price

```
namespace ProductEFCore.Models
{
   public class Products
   {
      public int Id { get; set; }
      public required string ProductName { get; set; }
      public double Price { get; set; }
   }
}
```

6: Create Data Folder in the project

Right-click on Project name-> Add -> New Folder

Name: Data

7: Create DbContext Class

AppDbContext.cs inside Data folder

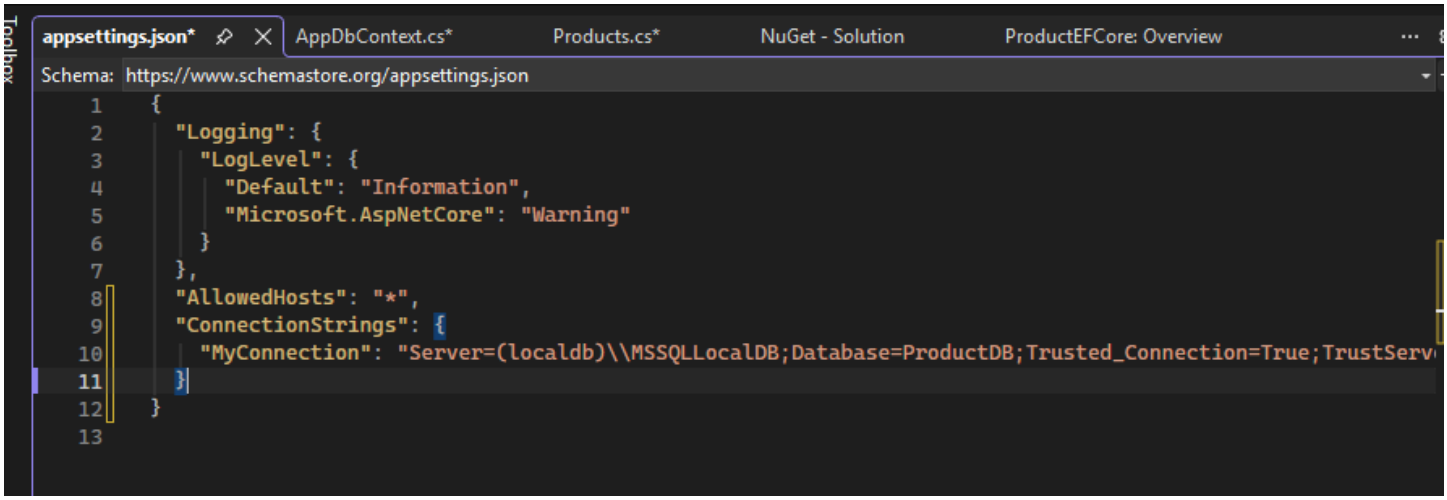
define DbSet for Products

```
using Microsoft.EntityFrameworkCore;
using ProductEFCore.Models;

namespace ProductEFCore.Data
{
   public class AppDbContext : DbContext
   {
      public AppDbContext(DbContextOptions<AppDbContext> options)
         : base(options)
      {
      }
      public DbSet<Products> Products { get; set; }
   }
}
```

8: Add Connection String in appsettings.json file for SQL Server database

Add




```
"AllowedHosts": "",
"ConnectionStrings": {
  "MyConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=ProductDB;Trusted_Connection=True;TrustServer
Certificate=True",
  "AppDbContext":
"Server=(localdb)\\mssqllocaldb;Database=AppDbContext;Trusted_Connection=True;MultipleAc
tiveResultSets=true"
}
```

9: Register DbContext in Program.cs file

using AddDbContext and UseSqlServer

```
using ProductEFCore.Data;
using Microsoft.EntityFrameworkCore;
```

Before:

```
var app = builder.Build();
```

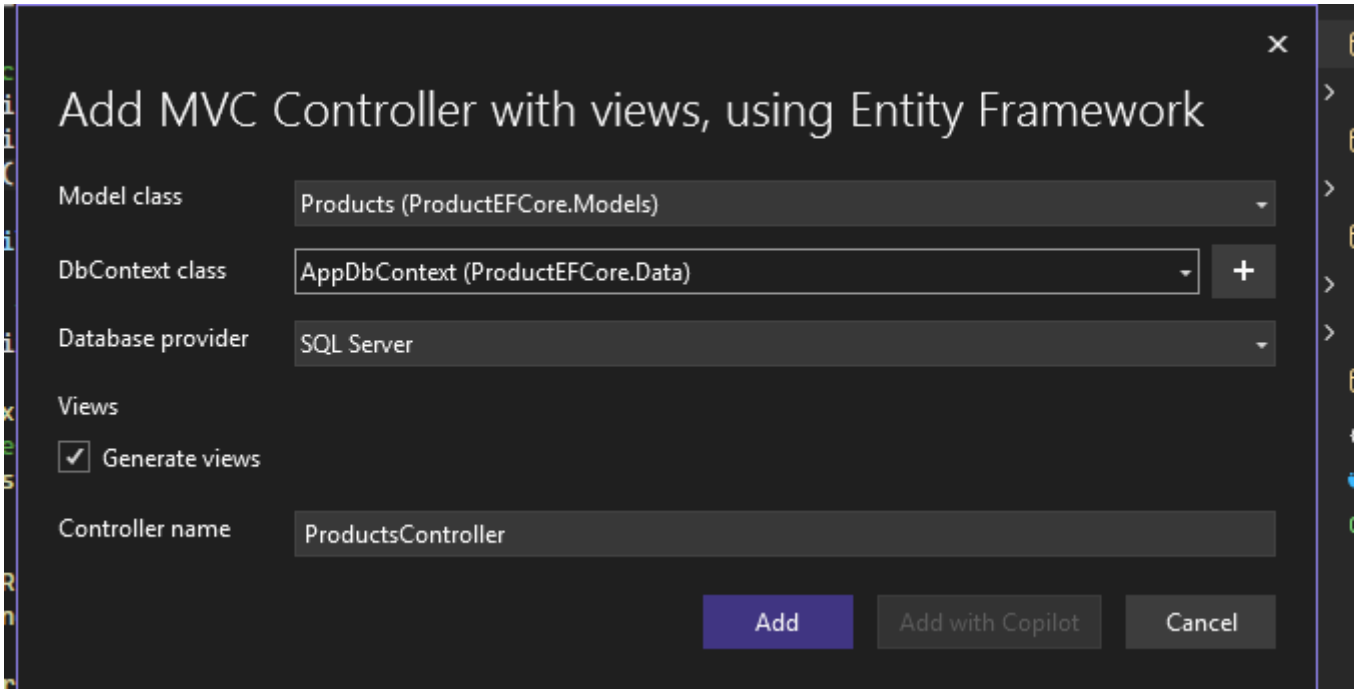
Add:

```
        builder.Services.AddDbContext<AppDbContext>(options =>
options.UseSqlServer(
    builder.Configuration.GetConnectionString("MyConnection"),
    sqlOptions => sqlOptions.EnableRetryOnFailure()
```

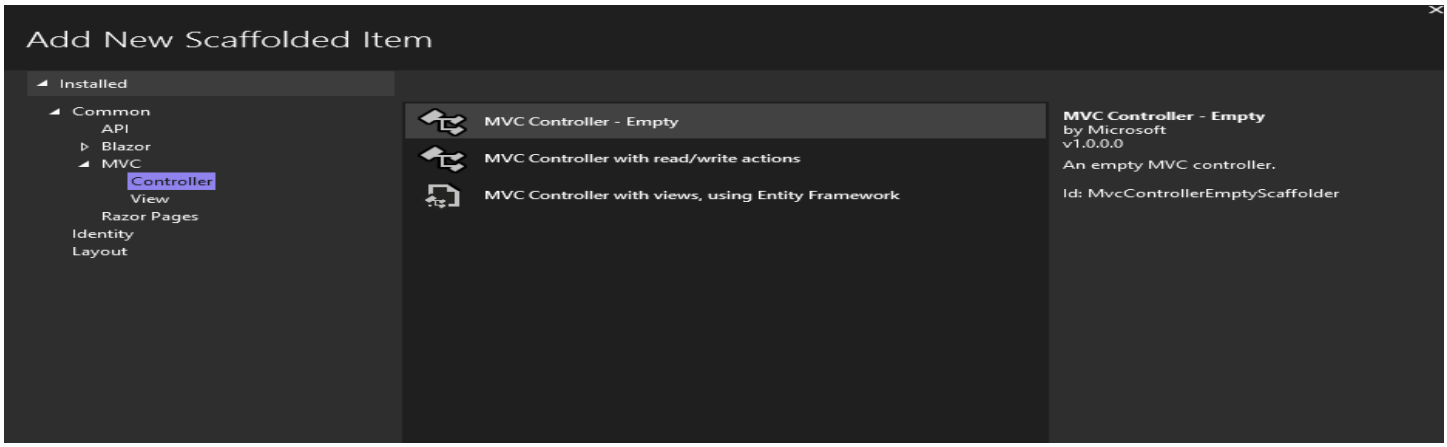
10: Create Products Controller using Scaffold.

Select MVC Controller with views, using Entity Framework

Controllers->Add-> Controller



11: Configure Scaffold by selecting Model class as Products and Data context class as AppDbContext



12: Create Database using Migration.

Tools-> NuGet Package Manager->Package Manager Console

Add-Migration InitialCreate

Update-Database

Note** If the Migration shows Some errors or serious Warning then:

1. Open the File

Locatew the ProductEFCore.csproj file inside your C:\Users\Administrator\source\repos\ProductEFCore\directory.

Open the file with Notepad or any good text editor.

Paste the Code FiX before </Project>tag.

```
<ItemGroup>
  <NuGetAuditSuppress Include="https://github.com/advisories/GHSA-g4vj-cjjj-v7hg" />
</ItemGroup>
```

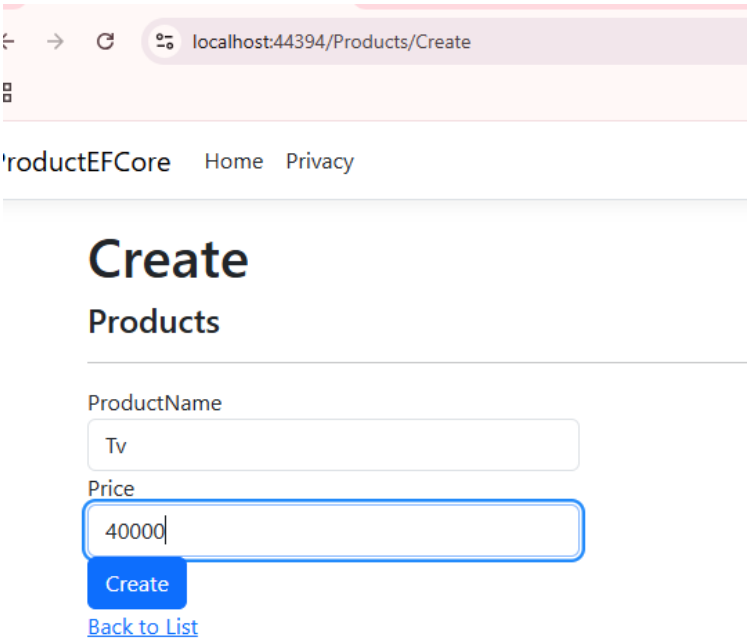
And Bulid and rebuild the project again and run Migration Command again

Once Success

Update-Database

13: Run the Project

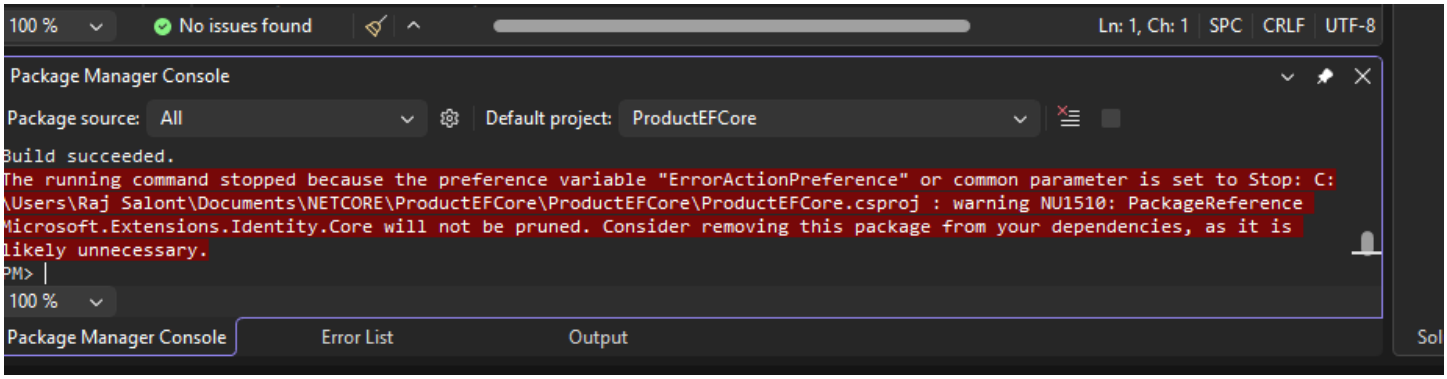
14: Open Insert Page by adding /Products to URL. Enter Product Name and Price and click Create



Important Methods / Functions:

1. AddDbContext - To register DbContext with dependency injection
2. UseSqlServer - To configure DbContext to use SQL Server
3. GetConnectionString - To read connection string from configuration
4. Add-Migration - To create migration files for database
5. Update-Database - To apply migrations and create database and tables
6. IActionResult Index - To display list of products
7. IActionResult Create - To show create form and to insert data
8. DbSet - To represent a collection of entities in DbContext

Error Faced:



(C) Develop a web application to perform CRUD Operations(Create, Read, Update, Delete) on Employee table using EF Core.

Steps:

1: Create MVC Project.

Project Name EmployeeCRUD and Framework .NET 10

2: Install EF Core Packages. Same packages as Practical 5b

3: Create Employee Model class in Models folder with properties Id, Name, Department, Salary

```
{
   public class Employee
   {
      public int Id { get; set; }

      public string Name { get; set; }

      public string Department { get; set; }

      public double Salary { get; set; }
   }
}
```

4: Create Data Folder(same as 5b)

5: Create DbContext Class

AppDbContext.cs inside Data folder and define DbSet for Employees

```
namespace EmployeeCRUD.Models


using EmployeeCRUD.Models;
using Microsoft.EntityFrameworkCore;

namespace EmployeeCRUD.Data
{
   public class AppDbContext : DbContext
   {
      public AppDbContext(DbContextOptions<AppDbContext> options)
         : base(options)
      {
      }
```

```
        public DbSet<Employee> Employees { get; set; }
    }
}
```

6: Add Connection String in appsettings.json file for EmployeeDB

```
"ConnectionStrings": {
  "MyConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=EmployeeDB;Trusted_Connection=True;TrustServerCertificate=True",
  "AppDbContext":
"Server=(localdb)\\mssqllocaldb;Database=AppDbContext;Trusted_Connection=True;MultipleActiveResultSets=true"
}
```

7: Configure DbContext in Program.cs file using AddDbContext and UseSqlServer

```
using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Data;
```

Before:

```
Var app=builder.Bulid();
```

Add:

```
builder.Services.AddDbContext<AppDbContext>(options =>
options.UseSqlServer(
builder.Configuration.GetConnectionString("MyConnection")));
```

8: Create Employee Controller using Scaffold. Select MVC Controller with views, using Entity Framework

9: Configure Scaffold by selecting Model class as Employee and Data context class as AppDbContext

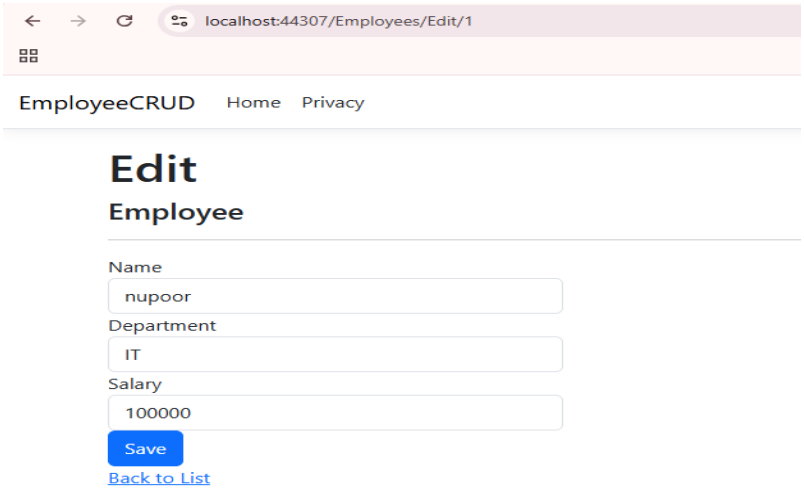
10: Create Database

```
Add-Migration IntialCreate
```

Then:

```
Update-Database
```

Step 11: Run Application and open /Employees URL to perform CRUD operations

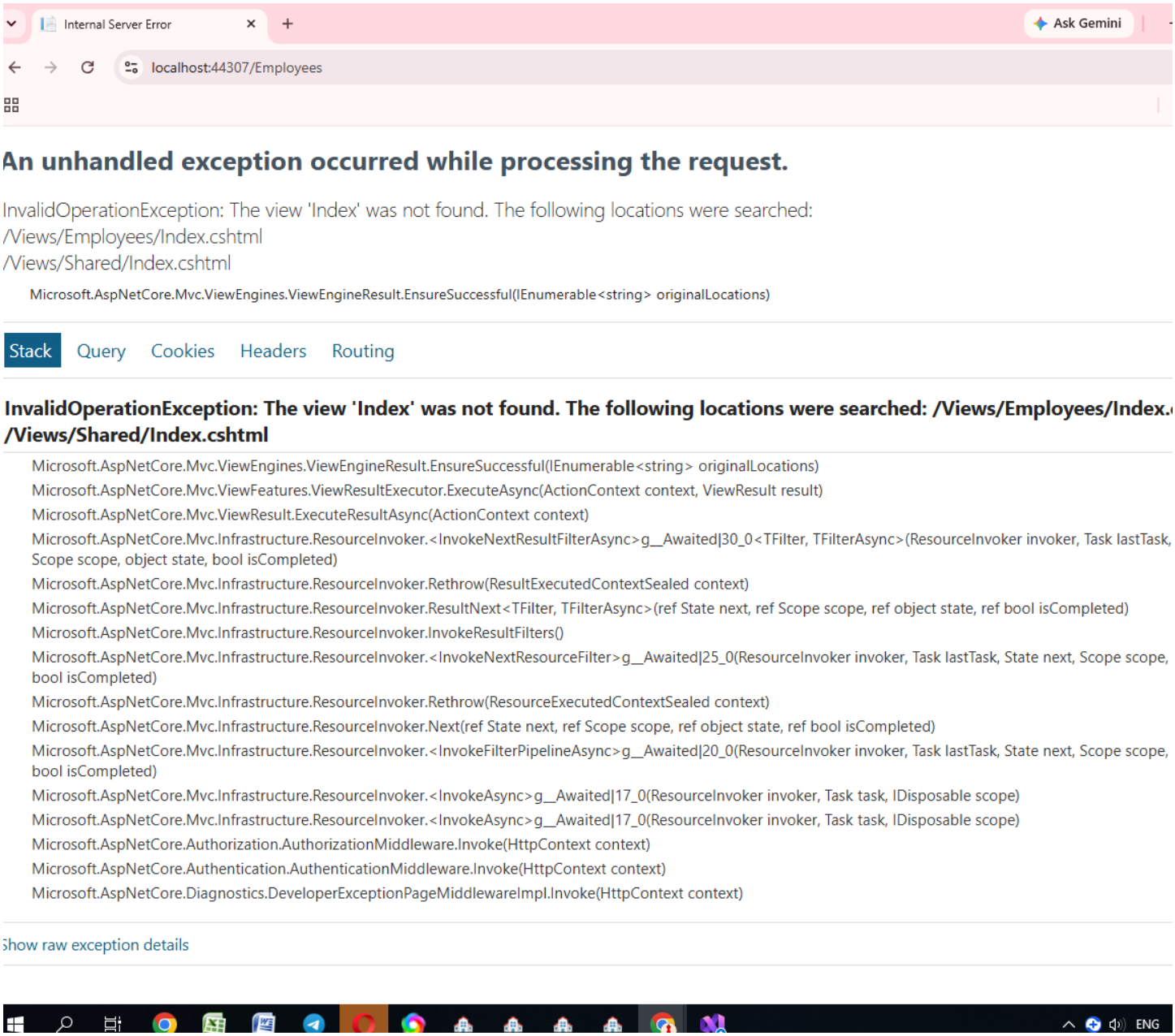




Important Methods / Functions:

1. AddDbContext - To register DbContext
2. UseSqlServer - To configure SQL Server provider
3. GetConnectionString - To get connection string from http://appsettings.json
4. Add-Migration - To add migration
5. Update-Database - To update database
6. IActionResult Index - To display all employees
7. IActionResult Create - To add new employee
8. IActionResult Edit - To update employee details
9. IActionResult Delete - To delete employee
10. IActionResult Details - To show employee details
11. DbSet - To access Employees table

Error Faced:


Internal Server Error
Ask Gemini
localhost:44307/Employees
An unhandled exception occurred while processing the request.
InvalidOperationException: The view 'Index' was not found. The following locations were searched:
/Views/Employees/Index.cshtml
/Views/Shared/Index.cshtml
Microsoft.AspNetCore.Mvc.ViewEngines.ViewEngineResult.EnsureSuccessful(IEnumerable<string> originalLocations)
Stack
Query
Cookies
Headers
Routing
InvalidOperationException: The view 'Index' was not found. The following locations were searched: /Views/Employees/Index.
/Views/Shared/Index.cshtml
Microsoft.AspNetCore.Mvc.ViewEngines.ViewEngineResult.EnsureSuccessful(IEnumerable<string> originalLocations)
Microsoft.AspNetCore.Mvc.ViewFeatures.ViewResultExecutor.ExecuteAsync(ActionContext context, ViewResult result)
Microsoft.AspNetCore.Mvc.ViewResult.ExecuteResultAsync(ActionContext context)
Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.<InvokeNextResultFilterAsync>g__Awaited|30_0<TFilter, TFilterAsync>(ResourceInvoker invoker, Task lastTask, Scope scope, object state, bool isCompleted)
Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.Rethrow(ResultExecutedContextSealed context)
Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.ResultNext<TFilter, TFilterAsync>(ref State next, ref Scope scope, ref object state, ref bool isCompleted)
Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.InvokeResultFilters()
Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.<InvokeNextResourceFilter>g__Awaited|25_0(ResourceInvoker invoker, Task lastTask, State next, Scope scope, bool isCompleted)
Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.Rethrow(ResourceExecutedContextSealed context)
Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.Next(ref State next, ref Scope scope, ref object state, ref bool isCompleted)
Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.<InvokeFilterPipelineAsync>g__Awaited|20_0(ResourceInvoker invoker, Task lastTask, State next, Scope scope, bool isCompleted)
Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.<InvokeAsync>g__Awaited|17_0(ResourceInvoker invoker, Task task, IDisposable scope)
Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.<InvokeAsync>g__Awaited|17_0(ResourceInvoker invoker, Task task, IDisposable scope)
Microsoft.AspNetCore.Authorization.AuthorizationMiddleware.Invoke(HttpContext context)
Microsoft.AspNetCore.Authentication.AuthenticationMiddleware.Invoke(HttpContext context)
Microsoft.AspNetCore.Diagnostics.DeveloperExceptionPageMiddlewareImpl.Invoke(HttpContext context)
Show raw exception details
ENG

(D) Authentication and authorization in ASP.NET Core.

Steps:

Step 1: Create MVC Project. Project (ASP.Net Core Web App(Model-View-Controller))

Name AuthDemo and Framework .NET 10

Step 2: Select Authentication Type as Individual Accounts in Additional Information window



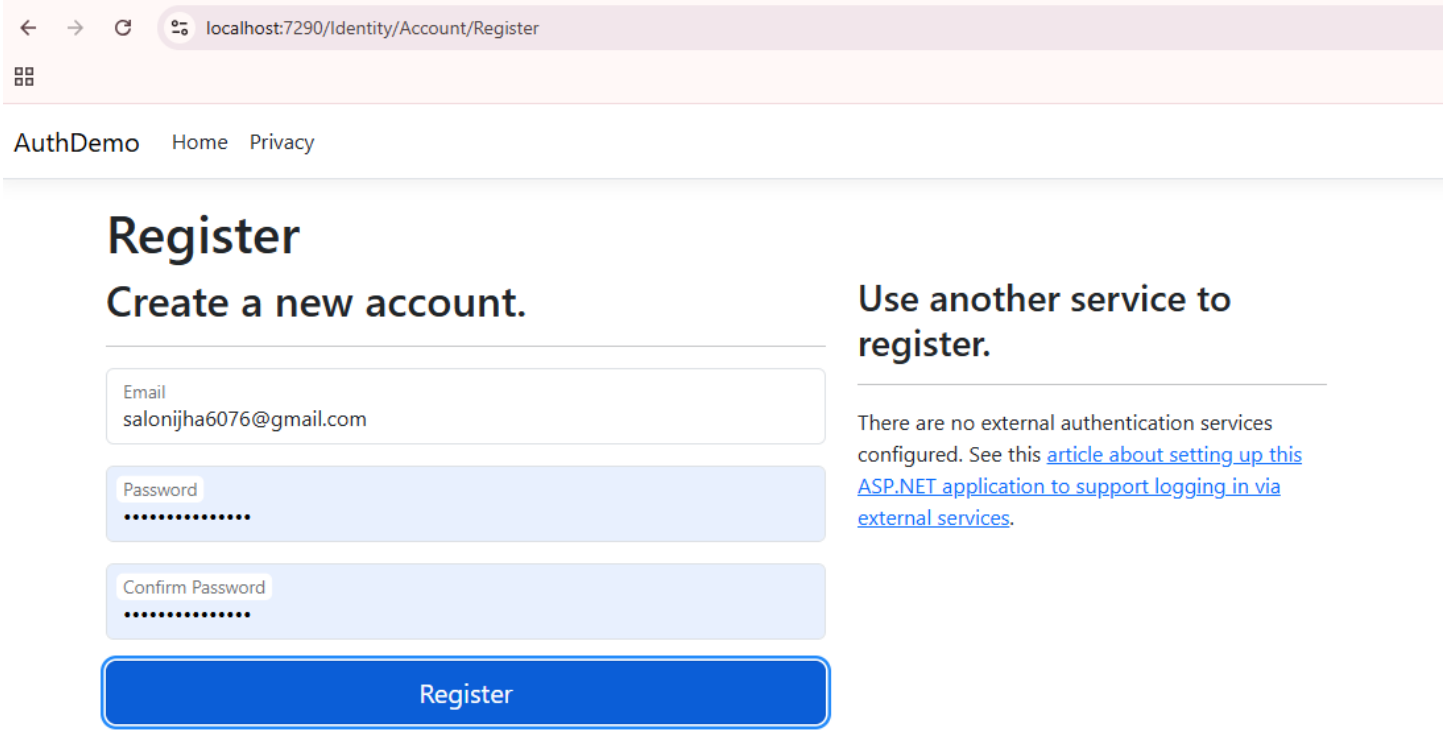
Step 3: Build Project using Rebuild Solution

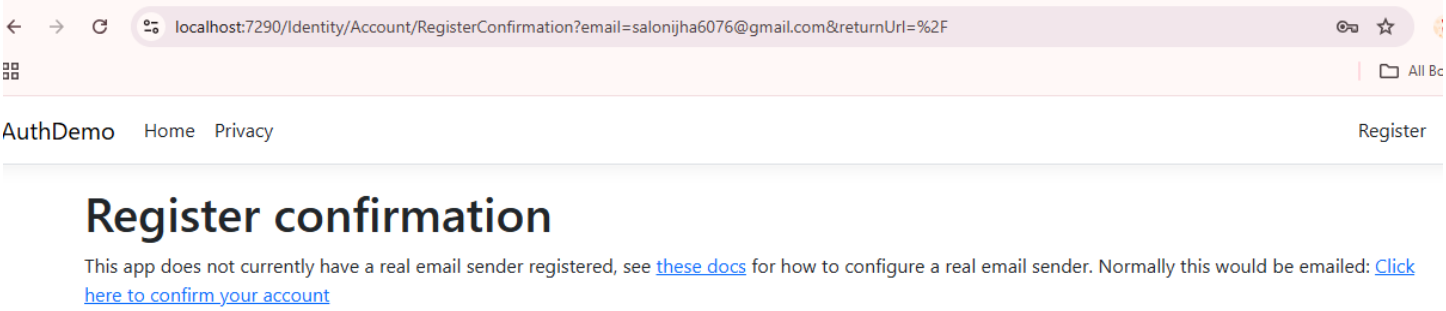
Go to Build-> Rebulid Solution

Step 4: Create Database by running Update-Database command. Identity tables are created automatically

Step 5: Run Application

Step 6: Register User by clicking Register link and entering Email and Password





Step 7: Login by clicking Login link and entering credentials

Step 8: Add Authorized Page. Create new method in HomeController and apply Authorize attribute

```
using Microsoft.AspNetCore.Authorization;
```

Add new Method

```
[Authorize]
 public IActionResult Secret()
 {
    return View();
 }
```

Step 9: Create Secret View in Views/Home folder

Create folder only if needed.

Right-click on **Home** Folder → Add → View → Select **Razor View Empty**

Name: Secret.cshtml

Write in empty area below the prewritten lines:

<h2>Authorized Users Only</h2>

Step 10: Test Authorization by accessing /Home/Secret URL.

If logged in page opens else redirects to Login pageImportant Methods / Functions:Update-Database

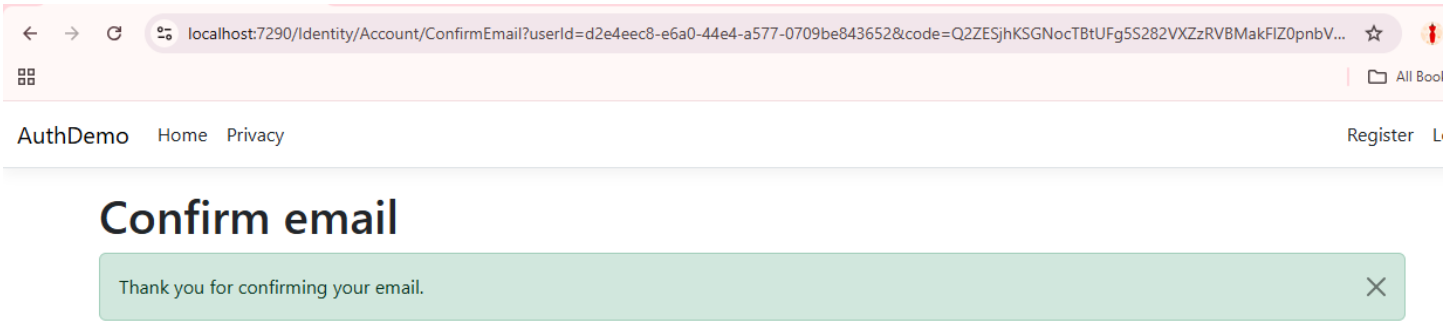
- To create Identity database tablesRegister

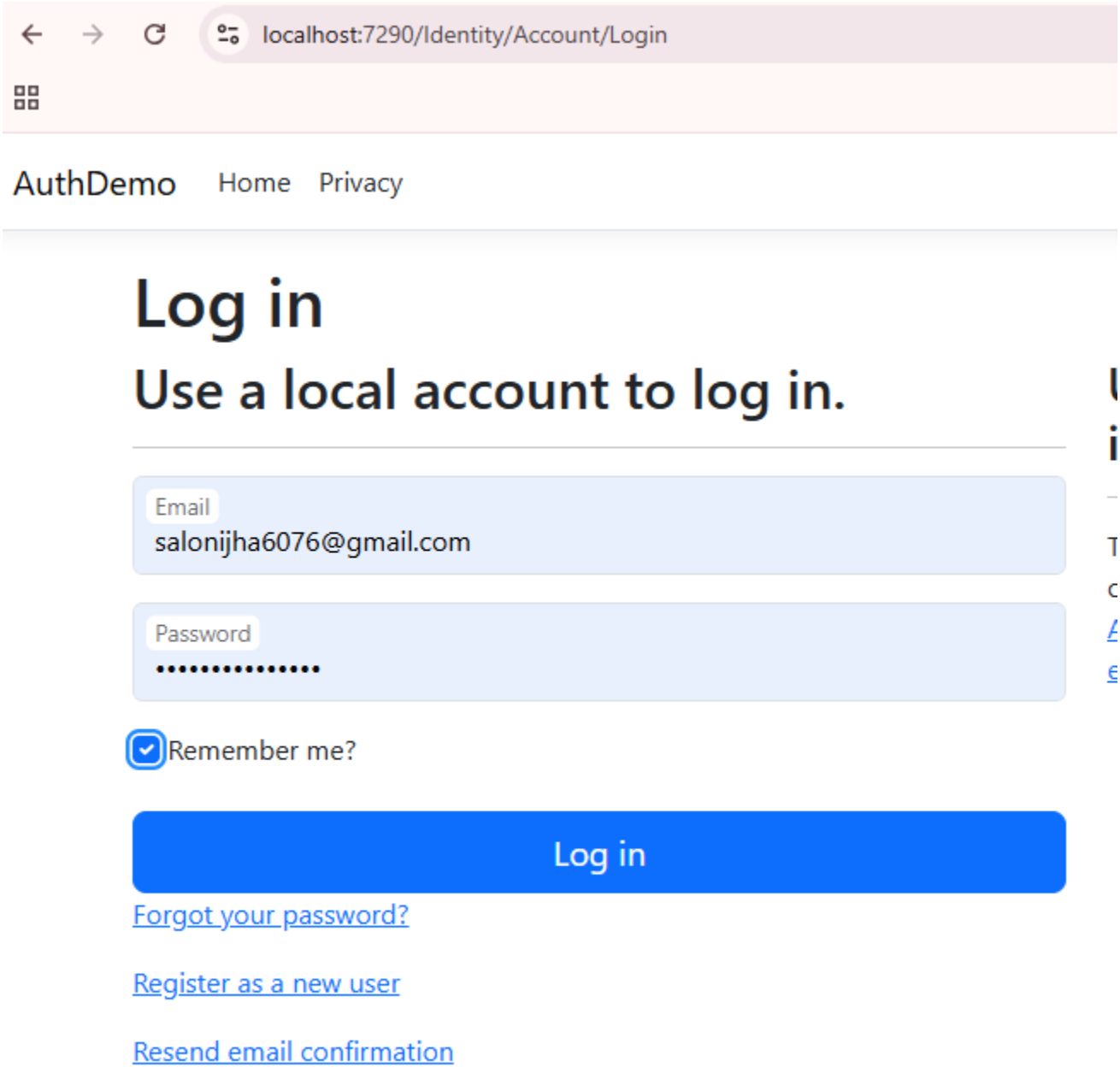
- To create new user accountLogin

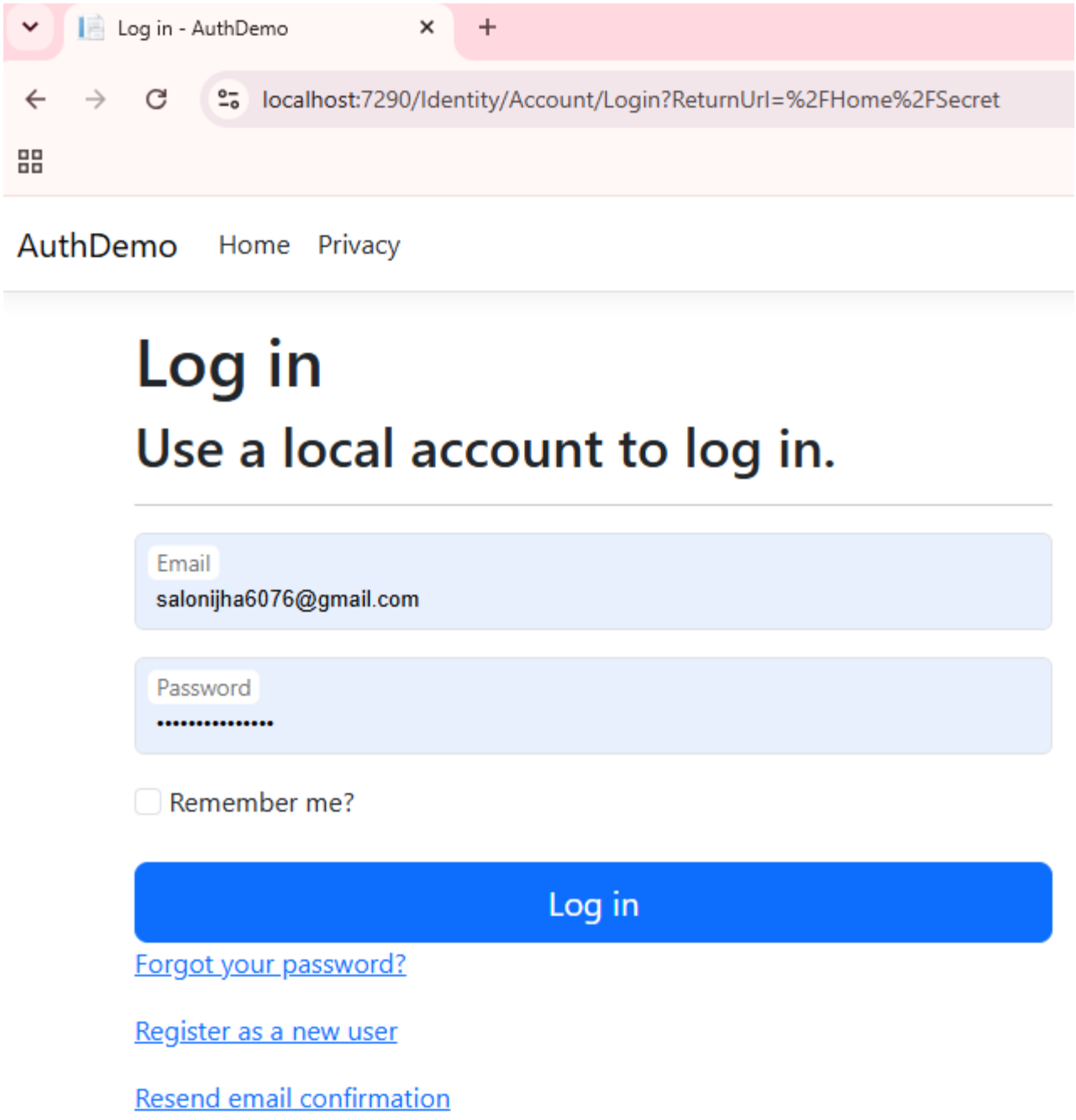
- To authenticate userAuthorize

- Attribute used to restrict access to authenticated users onlyIActionResult Secret

- Action method for authorized usersreturn View - To return view to browser




localhost:7290/Identity/Account/Login
AuthDemo
Home
Privacy
Log in
Use a local account to log in.
Email
salonijha6076@gmail.com
Password
Remember me?
Log in
Forgot your password?
Register as a new user
Resend email confirmation


Log in - AuthDemo
localhost:7290/Identity/Account/Login?ReturnUrl=%2FHome%2FSecret
AuthDemo
Home
Privacy
Log in
Use a local account to log in.
Email
salonijha6076@gmail.com
Password
Remember me?
Log in
Forgot your password?
Register as a new user
Resend email confirmation

Module-2 PWA (Progressive Web Application)

PRACTICAL-6

(A) Create a simple website for a college Information Portal and convert it into a progressive web application.

Steps:

1. Open Visual Studio 2026

Click: Create a new project

2. Create ASP.NET Core Web App

Search: `ASP.NET Core Web App (Model-View-Controller)` → Click: Next

3. Configure Project

Project Name: `CollegePortalPWA`

Framework: `NET 10 or Latest Available` → Click: Create

4. Update Home Page

`Views → Home → Index.cshtml`

Add College Name, Courses Offered list, and Contact Number.

Replace existing Html code with

```
<div class="text-center">
  <h1>College Information Portal</h1>

    <h3>Welcome to xyz College</h3>
    <p>Courses Offered:</p>
    <ul>
        <li>BSC IT</li>
        <li>BSC CS</li>
        <li>BCA</li>
    </ul>
    <p>Contact: 9818763456</p>
</div>
```

5. Add PWA Support - Service Worker

Right-click `wwwroot` → `Add → New Item` → `JavaScript File` →

Name: `service-worker.js`

```
self.addEventListener('install', function (event) {
    console.log('Service Worker Installed');
});

self.addEventListener('fetch', function (event) {
    console.log('Fetching:', event.request.url);
});
```

6. Create Manifest File

: Right-click `wwwroot` → `Add → New Item` → `JSON File` →

Name: `manifest.json`

```
  {
 "name": "College Information Portal",
 "short_name": "CollegePortal",
 "start_url": "/",
 "display": "standalone",
 "background_color": "#ffffff",
 "theme_color": "#0d6efd",
 "icons": [
  {
   "src": "/icons/icon1.png",
   "sizes": "192x192",
   "type": "image/png"
  },
  {
   "src": "/icons/icon2.png",
   "sizes": "512x512",
   "type": "image/png"
  }
 ]

}
```

7. Create Icons Folder

: Right-click `wwwroot` → `Add → New Folder` → Name: `icons`

Add:

`icon1.png`

`icon2.png`

8. Register Manifest

: `Views → Shared → _Layout.cshtml`

Add manifest link tag inside `<head>` section.

```
<link rel="manifest"href="~/manifest.json">
```

9. Register Service Worker

: Same `_Layout.cshtml` file, before `</body>`

```
<script>
if ('serviceWorker' in navigator) {
    navigator.serviceWorker.register('/service-worker.js')
        .then((registration) => {
            console.log('Service Worker Registered!', registration);
        })
        .catch((error) => {
            console.log('Service Worker Registration Failed:', error);
        });
}
</script>
```

10. Verify Static Files Enabled

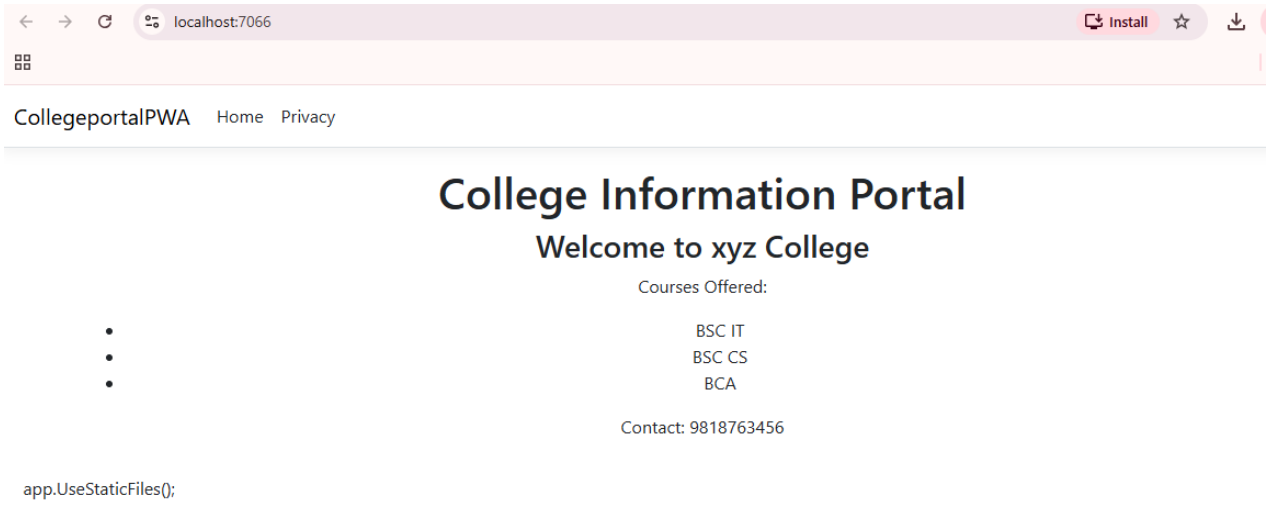
`Program.cs`

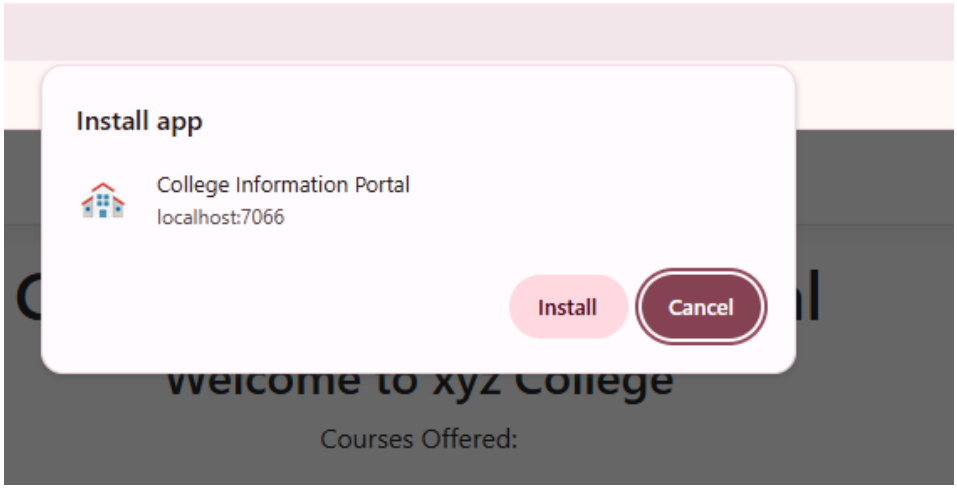
Ensure this line exists, if not then add at appropriate place:

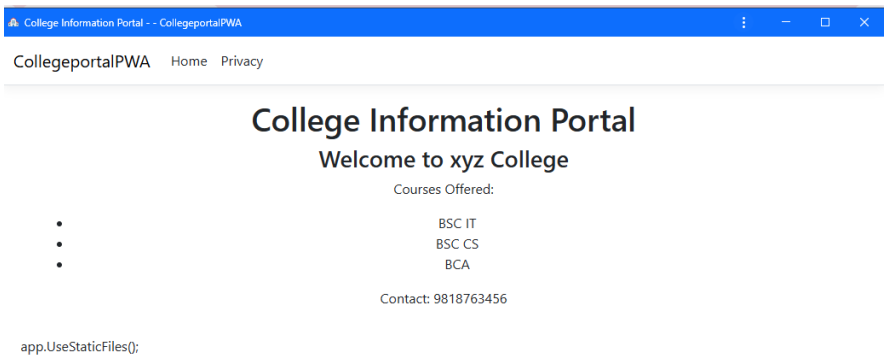
```
app.UseStaticFiles();
```

11. Run Application

Run in Chrome. Click on PWA Install Icon in address bar to install the app.



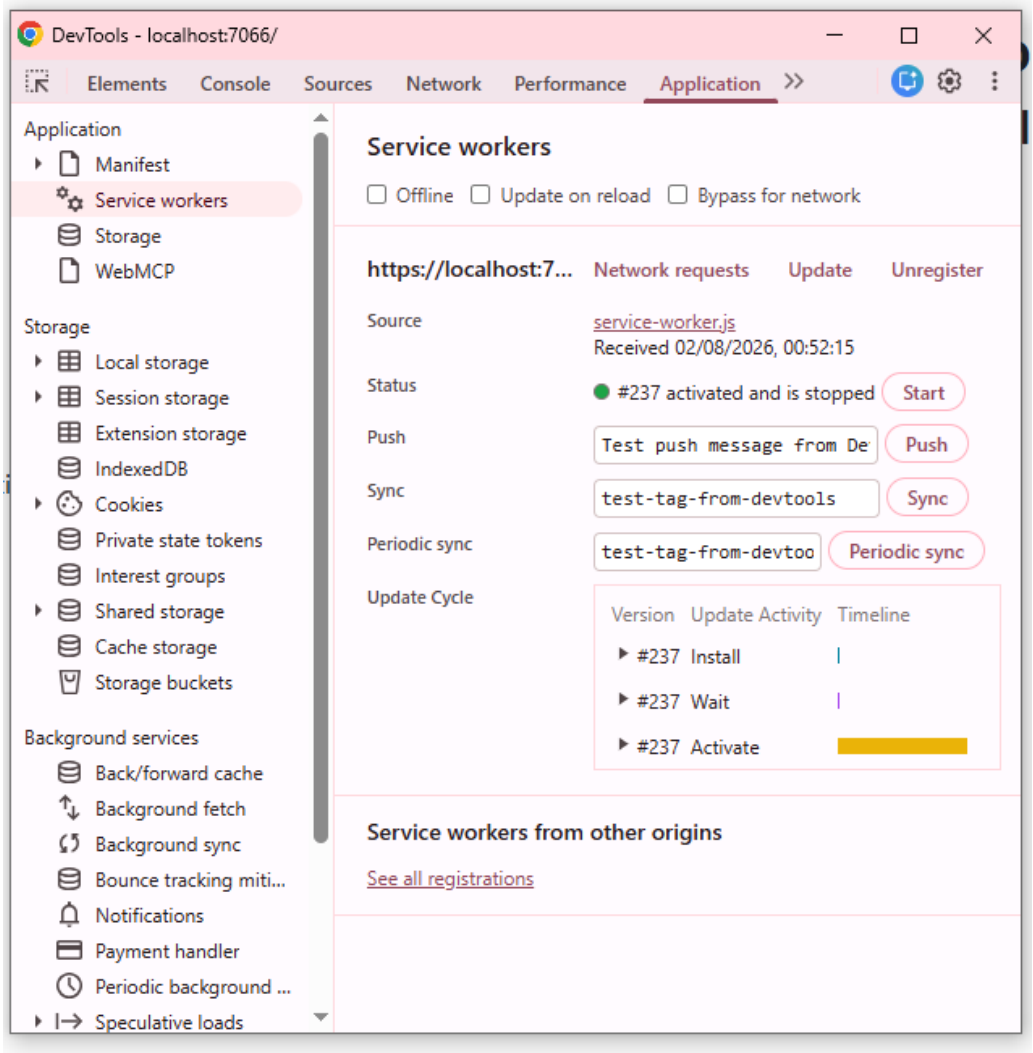




12. Verification of Service Worker

Press `F12` → Go to `Application → Service Workers`

Check if `service-worker.js` status is `activated and running`.



Important Methods/Functions Used:

- `self.addEventListener('install', function(event))` → For Service Worker Installation

- `self.addEventListener('fetch', function(event))` → To handle network requests

- `if ('serviceWorker' in navigator)` → To check browser support

- `navigator.serviceWorker.register('/service-worker.js')` → To register Service Worker

Important Properties Used:

- `name`, `short_name`, `start_url`, `display`, `background_color`, `theme_color`, `icons`

(B) Create a PWA that displays a custom offline page when the user is not connected to the internet.

Steps:

1. Create ASP.NET Core MVC Project

Project Name: `OfflinePWA`

Framework: `.NET 10 or Latest Available` → Click: Create

2. Create Home Page

: `Views → Home → Index.cshtml`

Add message: "This website supports Offline Mode."

```
<div class="text-center">
    <h1>College Information Portal</h1>
    <h3>Welcome to XYZ College</h3>
    <p>This website supports Offline Mode.</p>
</div>
```

3. Create Offline Page

: Right-click `wwwroot` → `Add → New Item → HTML Page` →

Name: `offline.html`

Design page with message: "No Internet Connection. Please check your internet connection."

```
<!DOCTYPE html>
<html>
<head>
    <title>Offline</title>
</head>
<body>
    <h1>No Internet Connection</h1>
    <h3>Please check your internet connection.</h3>
</body>
</html>
```

4. Create Manifest File

: `wwwroot` → `Add → New Item → JSON File` →

Name: `manifest.json`

```
{
  "name": "Offline PWA",
  "short_name": "OfflinePWA",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0d6efd",
  "icons": [
    {
      "src": "/icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

5. Create Icons Folder

: `wwwroot` → `Add → New Folder` → Name: `icons`

Add two icon files inside it.

- icon1.png
- icon2.png

6. Create Service Worker for Offline Caching

: `wwwroot` → `Add → New Item → JavaScript File` →

Name: `service-worker.js`

```
const CACHE_NAME = "offline-cache-v1";

const urlsToCache = [
    "/",
    "/offline.html"
];

self.addEventListener("install", event => {
```

```
    event.waitUntil(
        caches.open(CACHE_NAME)
            .then(cache => {
                return cache.addAll(urlsToCache);
            })
    );
});

self.addEventListener("fetch", event => {
    event.respondWith(
        fetch(event.request)
            .catch(() => {
                return caches.match("/offline.html");
            })
    );
});
```

7. Register Manifest

: `Views → Shared → _Layout.cshtml` → Add link tag in `<head>`

```
<link rel="manifest" href="~/manifest.json">
```

8. Register Service Worker

: Same `_Layout.cshtml` file, before `</body>`

Use `navigator.serviceWorker.register()` method.

```
<script>
        if ('serviceWorker' in navigator) {
            navigator.serviceWorker.register('/service-worker.js')
                .then(() => console.log('Service Worker Registered'));
        }
</script>
```

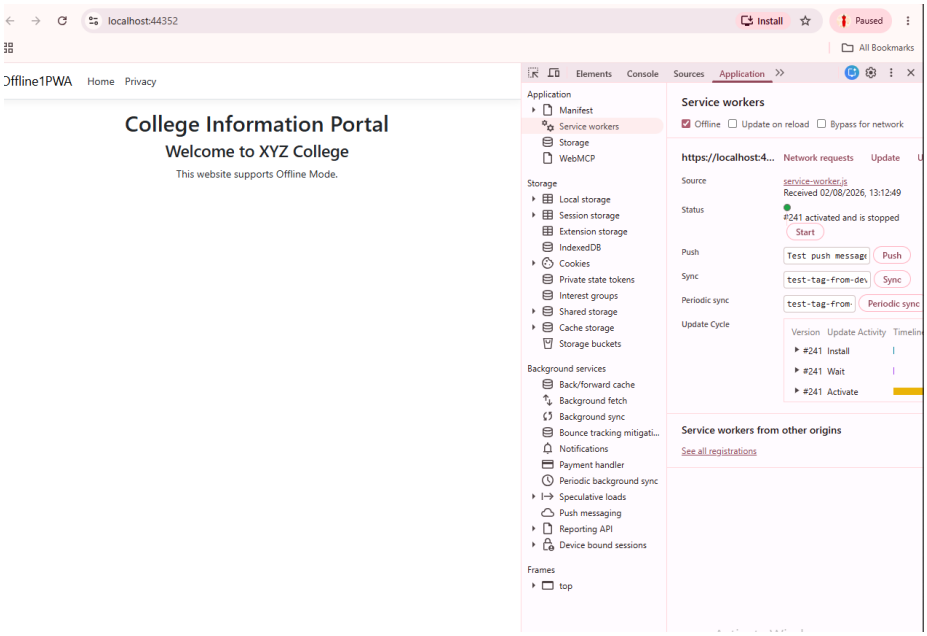
9. Verify Static Files

: `Program.cs` → Ensure `app.UseStaticFiles();` exists.

10. Run Application

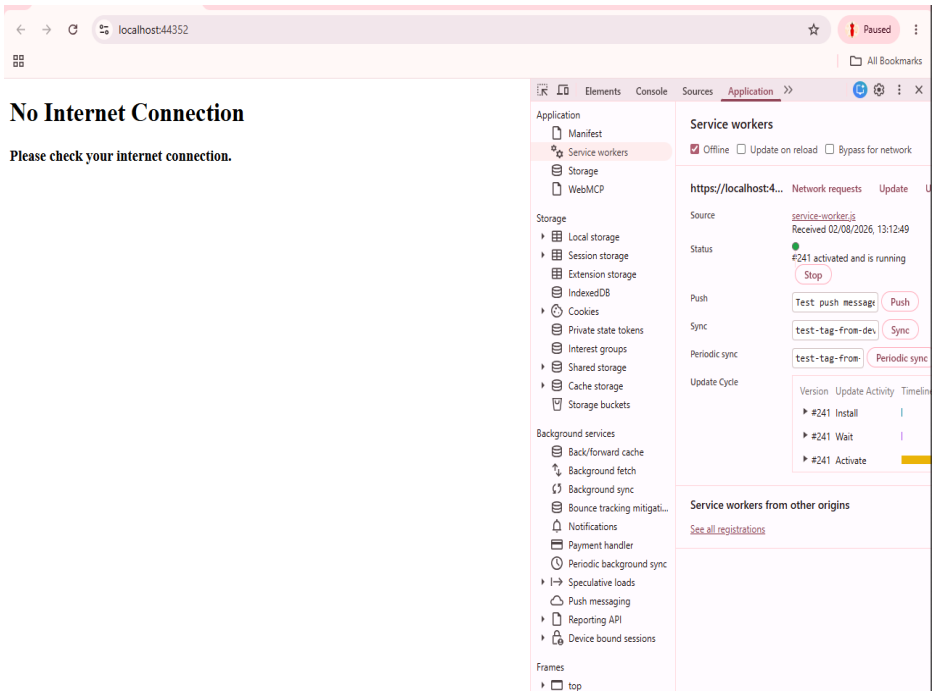
11. Verify Service Worker

Press `F12` → `Application → Service Workers` → Verify status.



12. Test Offline Page

Press `F12` → `Network` tab → Select `Offline` → Refresh page.



Important Methods/Functions Used:

- `caches.open(CACHE_NAME)` → To open cache storage
- `cache.addAll(urlsToCache)` → To cache required files during install
- `self.addEventListener('install', event)` → To cache files
- `self.addEventListener('fetch', event)` → To intercept requests
- `event.respondWith()` → To serve from cache or network
- `caches.match('/offline.html')` → To show offline page when network fails

PRACTICAL-7

(A) Develop a PWA that works in offline mode using service Worker.

Steps:

1. Create ASP.NET Core MVC Project

Open Visual Studio 2026 → Create new project → Select `ASP.NET Core Web App (Model-View-Controller)`

Project Name: `SWOfflinePWA` → Framework: `.NET 10` → Click: Create

2. Create Home Page

: `Views → Home → Index.cshtml`

Replace existing code with

```
<div class="text-center">
  <h1>College Information Portal</h1>
  <h3>PWA Offline Demo</h3>
  <p>This application works without internet connection.</p>
</div>
```

Update content to show: "College Information Portal" and "This application works without internet connection."

3. Create Manifest File

: Right-click `wwwroot` → `Add → New Item` → `JSON File`

Name: `manifest.json`

```
{
 "name": "Offline PWA",
 "short_name": "OfflinePWA",
 "start_url": "/",
 "display": "standalone",
 "background_color": "#ffffff",
 "theme_color": "#0d6efd",
 "icons": [
  {
   "src": "/icons/icon1.png",
   "sizes": "192x192",
   "type": "image/png"
  },
  {
```

```
    "src": "/icons/icon2.png",
    "sizes": "512x512",
    "type": "image/png"
   }
 ]
}
```

4. Create Icons Folder

: Right-click `wwwroot` → `Add → New Folder` → Name: `icons`

Add: `icon1.png` 192x192

`icon2.png` 512x512

5. Create Service Worker

: Right-click `wwwroot` → `Add → New Item` → `JavaScript File`

Name: `service-worker.js`

```
const CACHE_NAME = "pwa-cache-v1";

const urlsToCache = [
   "/",
   "/Home/Index"
];

self.addEventListener("install", event => {
   event.waitUntil(
      caches.open(CACHE_NAME)
         .then(cache => {
            return cache.addAll(urlsToCache);
         })
   );
});

self.addEventListener("fetch", event => {
   event.respondWith(
      caches.match(event.request)
         .then(response => {
            return response || fetch(event.request);
         })
   );
});
```

6. Register Manifest

: `Views → Shared → _Layout.cshtml` → Inside `<head>`

add link tag for `manifest.json`

```
<link rel="manifest" href="/manifest.json">
```

7. Register Service Worker

: Same `_Layout.cshtml` file, before `</body>` tag

```
<script>
      if ('serviceWorker' in navigator) {
          navigator.serviceWorker.register('/service-worker.js')
              .then(() => console.log('Service Worker Registered'));
      }
</script>
```

8. Verify Static Files

: `Program.cs` → Ensure `app.UseStaticFiles();` exists

9. Run Application

Run project and check PWA Install icon in browser

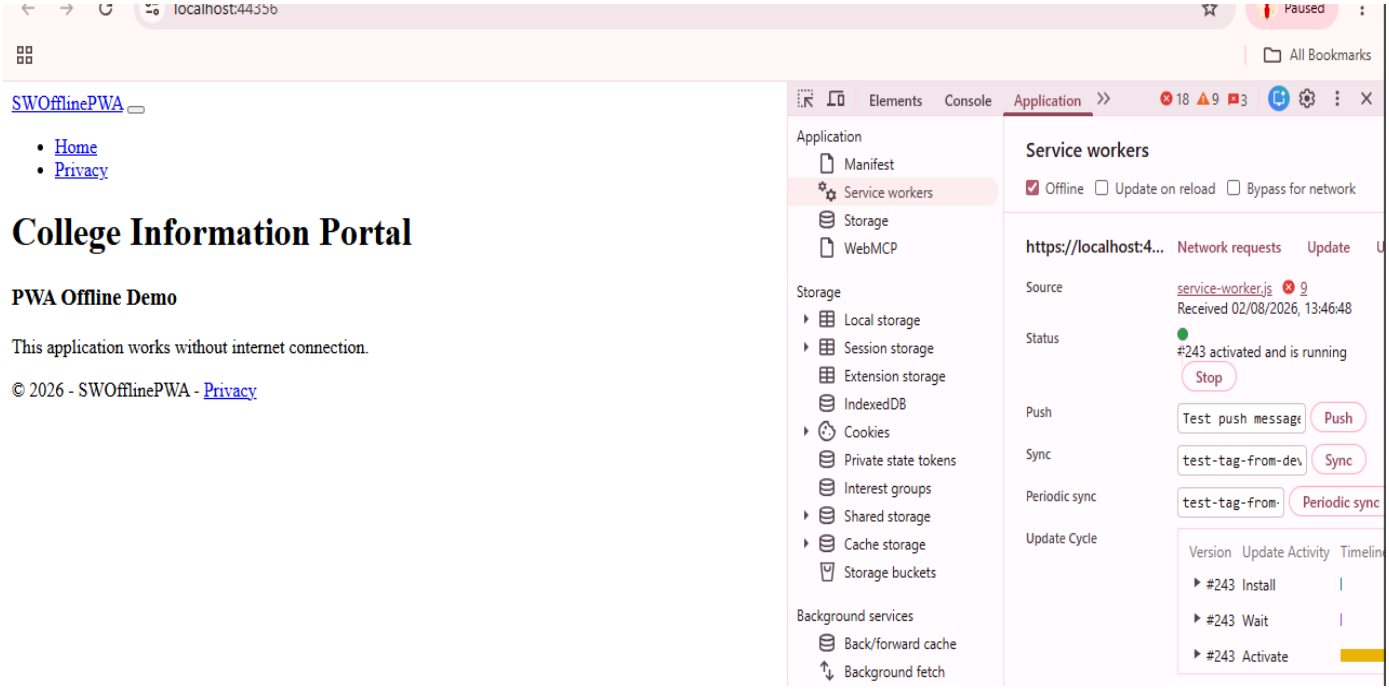
10. Verify Service Worker

Press `F12` → `Application → Service Workers` → Check `service-worker.js` status = `activated and running`

11. Test Offline Mode

Press `F12` → `Network` tab → Select `Offline` → Refresh webpage

Website still works because pages are loaded from Service Worker cache



Important Methods/Functions Used:

1. `caches.open(CACHE_NAME)` - To open cache storage
2. `cache.addAll(urlsToCache)` - To cache files during install event
3. `self.addEventListener('install', event)` - To handle service worker installation
4. `self.addEventListener('fetch', event)` - To intercept network requests
5. `caches.match(event.request)` - To check if request exists in cache
6. `fetch(event.request)` - To fetch resource from network
7. `navigator.serviceWorker.register('/service-worker.js')` - To register service worker
8. `app.UseStaticFiles()` - To enable serving static files

(B) Create a Student Record Management PWA.

Steps:

1. Create ASP.NET Core MVC Project

Project Name: `StudentRecordPWA`

Framework: `.NET 10 or Latest` → Click: Create

2. Create Student Model

: Right-click `Models` → `Add → Class` →

Name: `Student.cs`

Write:

```
namespace StudentRecordPWA.Models
{
    public class Student
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Course { get; set; }
        public int Age { get; set; }
    }
}
```

Add properties: `Id`, `Name`, `Course`, `Age`

3. Create Student Controller

: Right-click `Controllers` → `Add → Controller` → Choose: `MVC Controller - Empty`

Name: `StudentsController`

4. Write Controller Logic

: `StudentsController.cs`

```
using Microsoft.AspNetCore.Mvc;
using StudentRecordPWA.Models;
using System.Collections.Generic;

namespace StudentRecordPWA.Controllers
{
    public class StudentsController : Controller
    {
        public IActionResult Index()
        {
            List<Student> students = new List<Student>()
            {
                new Student { Id = 1, Name = "Rahul", Course = "BSc IT", Age = 20 },
                new Student { Id = 2, Name = "Amit", Course = "BCA", Age = 21 },
                new Student { Id = 3, Name = "Sneha", Course = "BSc CS", Age = 19 }
            };
            return View(students);
        }
    }
}
```

Create list of student objects and pass it to View

5. Create View Folder

: `Views → Students`

6. Create Index View

: Right-click `Views/Students` → `Add → View` → `Razor View - Empty` → Name: `Index.cshtml`

Write:

```
@model IEnumerable<StudentRecordPWA.Models.Student>


<h2>Student Records</h2>
<table border="1" cellpadding="10">
    <tr>
        <th>Id</th>
        <th>Name</th>
        <th>Course</th>
        <th>Age</th>
    </tr>

    @foreach (var item in Model)
    {
        <tr>
            <td>@item.Id</td>
            <td>@item.Name</td>
            <td>@item.Course</td>
            <td>@item.Age</td>
        </tr>
    }
</table>
```

Display student data in HTML table

7. Create Manifest File

: `wwwroot` → `Add → New Item` → `JSON File` →

Name: `manifest.json`

Write:

```
{
  "name": "Student Record PWA",
  "short_name": "StudentPWA",
  "start_url": "/Students",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0d6efd",
  "icons": [
    {
      "src": "/icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

8. Create Icons Folder

: `wwwroot` → `Add → New Folder` → Name: `icons`

Add: `icon1.png`

`icon2.png`

9. Create Service Worker

: `wwwroot` → `Add → New Item` → `JavaScript File` →

Name: `service-worker.js`

Write:

```
const CACHE_NAME = "student-pwa-cache";

const urlsToCache = [
  "/Students"
];

self.addEventListener("install", event => {
  event.waitUntil(
    caches.open(CACHE_NAME)
    .then(cache => {
      return cache.addAll(urlsToCache);
    })
  );
});

self.addEventListener("fetch", event => {
  event.respondWith(
    caches.match(event.request)
    .then(response => {
      return response || fetch(event.request);
    })
  );
});
```

10. Register Manifest

: `Views → Shared → _Layout.cshtml`

Inside `<head>` add manifest link

<link rel="manifest" href="/manifest.json">

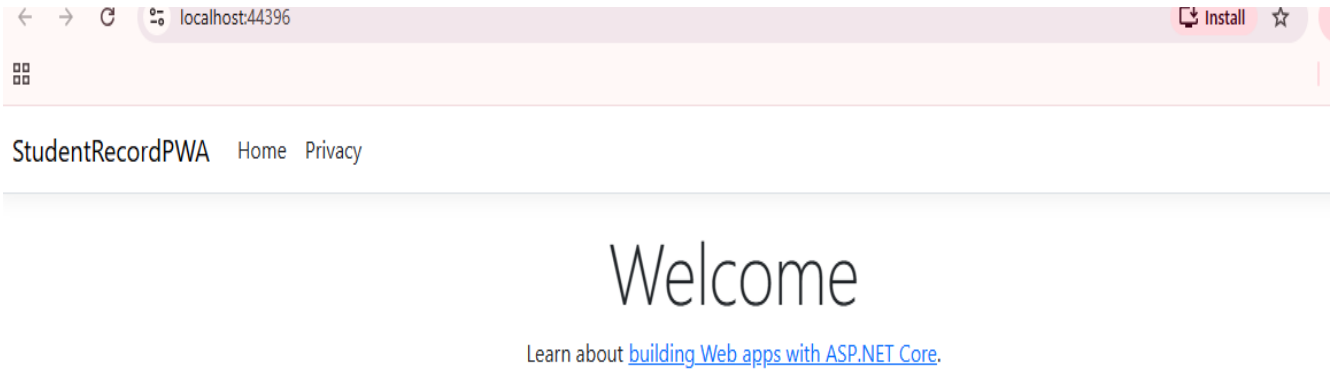
11. Register Service Worker

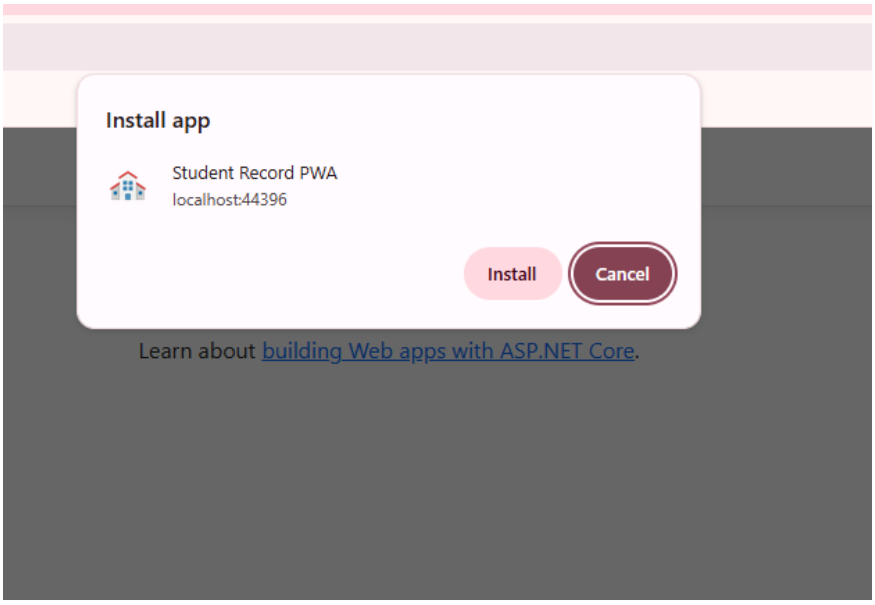
: Same `_Layout.cshtml` file, before `</body>` tag

```
<script>
  if ('serviceWorker' in navigator) {
    navigator.serviceWorker.register('/service-worker.js')
    .then(() => console.log('Service Worker Registered'));
  }
</script>
```

12. Run Application

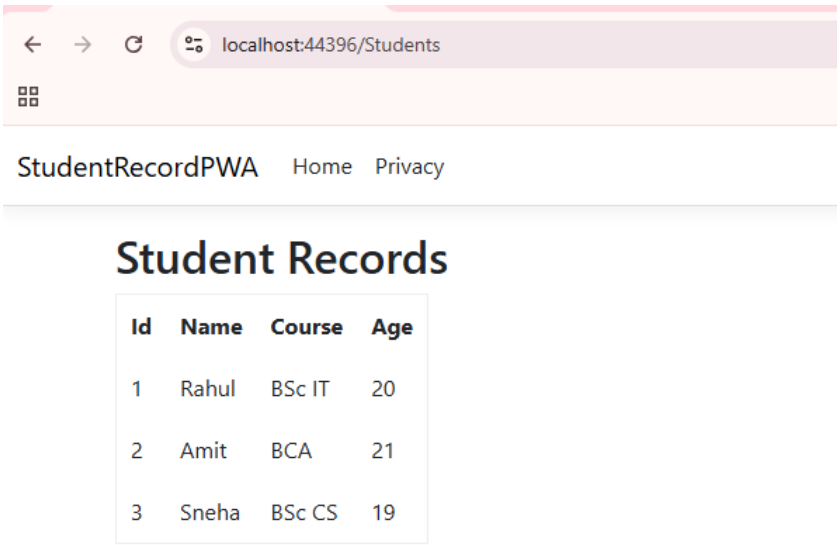
Run project and append `/Students` in URL to view student records. PWA Install button will appear.





In the Browser click on the URL and add Students

"https://localhost:7147/Students"



Important Methods/Functions Used:

1. `public IActionResult Index()` - Controller action method to return student list
2. `List<Student>` - To store collection of student objects
3. `caches.open(CACHE_NAME)` - To open cache storage
4. `cache.addAll(urlsToCache)` - To cache "/Students" page during install
5. `self.addEventListener('install', event)` - To handle service worker installation
6. `self.addEventListener('fetch', event)` - To handle fetch requests
7. `caches.match(event.request)` - To serve cached response
8. `fetch(event.request)` - To fetch from network if not cached
9. `navigator.serviceWorker.register('/service-worker.js')` - To register service worker
10. `@foreach (var item in Model)` - To iterate and display student records in View

PRACTICAL-8

(A) Create a PWA Application with an HTML page that would be the starting point of the application. This HTML will contain a link to the file named manifest.json .

Steps:

1. Create Empty Web Project

Open Visual Studio 2026

Create new project

Select `ASP.NET Core Web App (Model-View-Controller)

Click: Next

2. Configure Project

Project Name: `BasicPWA`

Framework: `.NET 10 or Latest`

Click: Create

3. Enable Static Files

Open `Program.cs

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.UseDefaultFiles();
app.UseStaticFiles();

app.Run();
```

Add middleware to enable default files and static files

4. Create Starting HTML Page

: Right-click `wwwroot` → `Add New Item` → `HTML Page`

Name: `index.html`

Write:

```
<!DOCTYPE html>
<html>
<head>
   <title>Basic PWA</title>
   <meta name="viewport" content="width=device-width, initial-scale=1">
   <link rel="manifest" href="manifest.json">
</head>
<body>
   <h1>My Progressive Web Application</h1>
   <p>This is the starting page of PWA.</p>
</body>
</html>
```

Add heading "My Progressive Web Application" and paragraph "This is the starting page of PWA."

5. Link Manifest File in HTML

Inside `<head>` tag of `index.html` add link to `manifest.json`

<link rel=”manifest” href=”manifest.json”>

This connects:

- HTML page
  With
- Manifest.json

6. Create Icons Folder

: Right-click `wwwroot` → `Add → New Folder` → Name: `icons`

Add: icon1.png

icon2.png

7. Create Manifest File

: Right-click `wwwroot` → `Add → New Item` → `JSON File`

Name: `manifest.json`

Add following code in manifest.json file

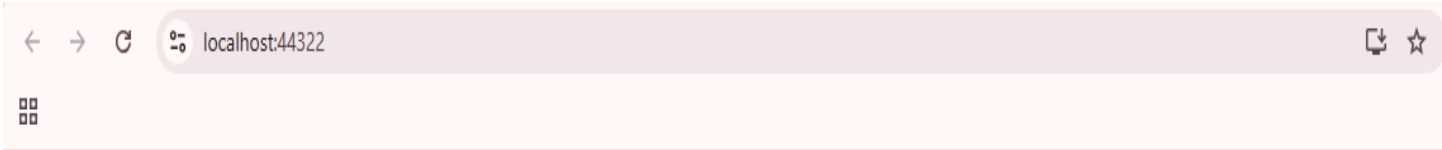
```
{
 "name": "Basic Progressive Web Application",
 "short_name": "BasicPWA",
 "description": "Simple PWA Example",
 "start_url": "/index.html",
 "display": "standalone",
 "background_color": "#ffffff",
 "theme_color": "#0d6efd",
 "orientation": "portrait",
 "icons": [
  {
   "src": "/icons/icon1.png",
   "sizes": "192x192",
   "type": "image/png"
  },
  {
   "src": "/icons/icon2.png",
   "sizes": "512x512",
   "type": "image/png"
  }
 ]
}
```

Define app name, description, start_url, display mode, theme color and icons

8. Run Application

Browser opens: `https://localhost:xxxx/index.html`

PWA Install button appears in address bar



My Progressive Web Application

This is the starting page of PWA.

Important Methods/Functions Used:

1. `WebApplication.CreateBuilder(args)` - To create web application builder
2. `builder.Build()` - To build the application
3. `app.UseDefaultFiles()` - To enable default file serving
4. `app.UseStaticFiles()` - To enable serving static files like HTML, CSS, JS
5. `app.Run()` - To start the application

(B) Create a simple Web API that returns student data in JSON format.

Algorithm / Steps:

1. Create Web API Project

Open Visual Studio 2026

Create new project

Select `ASP.NET Core Web API`

Click: Next

2. Configure Project

Project Name: `StudentAPI`

Framework: `.NET 10 or Latest`

Click: Create

3. Create Student Model

: Right-click Project → `Add → New Folder`

Name: `Models`

Inside Models: `Add → Class` → Name: `Student.cs`

Add properties: `Id`, `Name`, `Course`, `Age`

```
namespace StudentAPI.Models
{
    public class Student
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Course { get; set; }
        public int Age { get; set; }
    }
}
```

4. Create API Controller

: Right-click Project → `Add → New Folder` → Name: `Controllers`

Inside Controllers: `Add → Controller` → Choose: `API Controller - Empty`

Name: `StudentController.cs`

5. Write API code in the File

: `StudentController.cs`

Add routing attribute and create method to return list of student objects

```
using Microsoft.AspNetCore.Mvc;
using StudentAPI.Models;

namespace StudentAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class StudentController : ControllerBase
    {
        [HttpGet]
        public List<Student> GetStudents()
        {
            return new List<Student>()
            {
                new Student
                {
                    Id = 1,
                    Name = "Rahul",
                    Course = "BSc IT",
                    Age = 20
                },

                new Student
                {
                    Id = 2,
                    Name = "Sneha",
                    Course = "BCA",
                    Age = 21
                }
            };
        }
    }
}
```

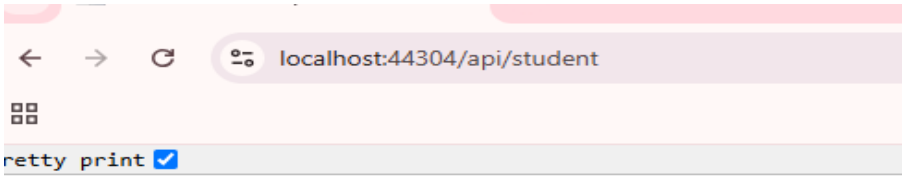
6. Run Application

Run the project

7. Test API

Open browser and go to: `https://localhost:xxxx/api/student`

Output: Student data is displayed in JSON format




```
{
  "id": 1,
  "name": "Rahul",
  "course": "BSc IT",
  "age": 20
},
{
  "id": 2,
  "name": "Sneha",
  "course": "BCA",
  "age": 21
}
```


Important Methods/Functions/Attributes Used:

1. `public class Student` - Model class with properties
2. `[ApiController]` - Attribute to enable API specific features
3. `[Route("api/[controller]")]` - Attribute to define API route
4. `public class StudentController : ControllerBase` - Base class for API controller
5. `[HttpGet]` - Attribute to handle HTTP GET requests
6. `public List<Student> GetStudents()` - Method to return student data
7. `new List<Student>()` - To create and return collection of student objects

PRACTICAL-9

(A) Develop CRUD operations using ASP.NET Core Web API and Entity Framework core.

Algorithm / Steps:

1. Create http://ASP.NET Core Web API Project

Open Visual Studio 2026 → Create New Project → Select `ASP.NET Core Web API`

Project Name: `StudentAPIWithEF` → Framework: `.NET 8`

Enable OpenAPI Support → Click: Create

2. Install Entity Framework Core Packages

: `Tools → NuGet Package Manager → Manage NuGet Packages`

Install:

- `Microsoft.EntityFrameworkCore`,
- `Microsoft.EntityFrameworkCore.SqlServer`,
- `Microsoft.EntityFrameworkCore.Tools`,
- `Swashbuckle.AspNetCore`,
- `Microsoft.EntityFrameworkCore.Design`

3. Create Model Class

: Right-click Project → `Add → New Folder` → Name: `Models`

Add Class: `Student.cs` with properties: `Id`, `Name`, `Age`, `Course`

```
namespace StudentAPIWithEF.Models
{
    public class Student
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public int Age { get; set; }
        public string Course { get; set; }
```

```
    }
}
```

4. Create DbContext Class

: Right-click Project → `Add → New Folder` → Name: `Data`

Add Class: `AppDbContext.cs`

Inherit from `DbContext` and add `DbSet<Student>` property

```
using Microsoft.EntityFrameworkCore;
using StudentAPIWithEF.Models;

namespace StudentAPIWithEF.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options) { }

        public DbSet<Student> Students { get; set; }
    }
}
```

5. Add LocalDB Connection String

: Open `appsettings.json`

Add Following code:

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=StudentDB;Trusted_Connection=True;"
```

```
  },
  "Logging": {... }
}
```

6. Configure Database in Program.cs

Replace with:

```
using Microsoft.EntityFrameworkCore;
using StudentAPIWithEF.Data;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

app.UseSwagger();
app.UseSwaggerUI();

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
```

app.Run();

7. Create Controller

: Right-click Project → `Add → New Folder` → Name: `Controllers`

Add Class: `StudentsController.cs` inheriting `ControllerBase`

Inject `AppDbContext` through constructor for database operations

8. Implement CRUD Methods in Controller

- GET ALL: To fetch all students

- GET BY ID: To fetch single student by Id

- POST: To insert new student

- PUT: To update existing student

- DELETE: To delete student by Id

```
using Microsoft.AspNetCore.Mvc;
using StudentAPIWithEF.Data;
using StudentAPIWithEF.Models;

namespace StudentAPIWithEF.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class StudentsController : ControllerBase
    {
        private readonly AppDbContext _context;

        public StudentsController(AppDbContext context)
        {
            _context = context;
        }

        // GET ALL
        [HttpGet]
        public IActionResult GetStudents()
        {
            return Ok(_context.Students.ToList());
        }

        // GET BY ID
        [HttpGet("{id}")]
        public IActionResult GetStudent(int id)
```

```
        {
            var student = _context.Students.Find(id);
            if (student == null) return NotFound();
            return Ok(student);
        }

        // INSERT
        [HttpPost]
        public IActionResult AddStudent(Student student)
        {
            _context.Students.Add(student);
            _context.SaveChanges();
            return Ok(student);
        }

        // UPDATE
        [HttpPut("{id}")]
        public IActionResult UpdateStudent(int id, Student student)
        {
            var data = _context.Students.Find(id);
            if (data == null) return NotFound();

            data.Name = student.Name;
            data.Age = student.Age;
            data.Course = student.Course;

            _context.SaveChanges();
            return Ok(data);
        }

        // DELETE
        [HttpDelete("{id}")]
        public IActionResult DeleteStudent(int id)
        {
            var student = _context.Students.Find(id);
            if (student == null) return NotFound();

            _context.Students.Remove(student);
            _context.SaveChanges();
            return Ok();
        }
    }
}
```

9. Create Database using Migration

: `Tools → NuGet Package Manager → Package Manager Console`

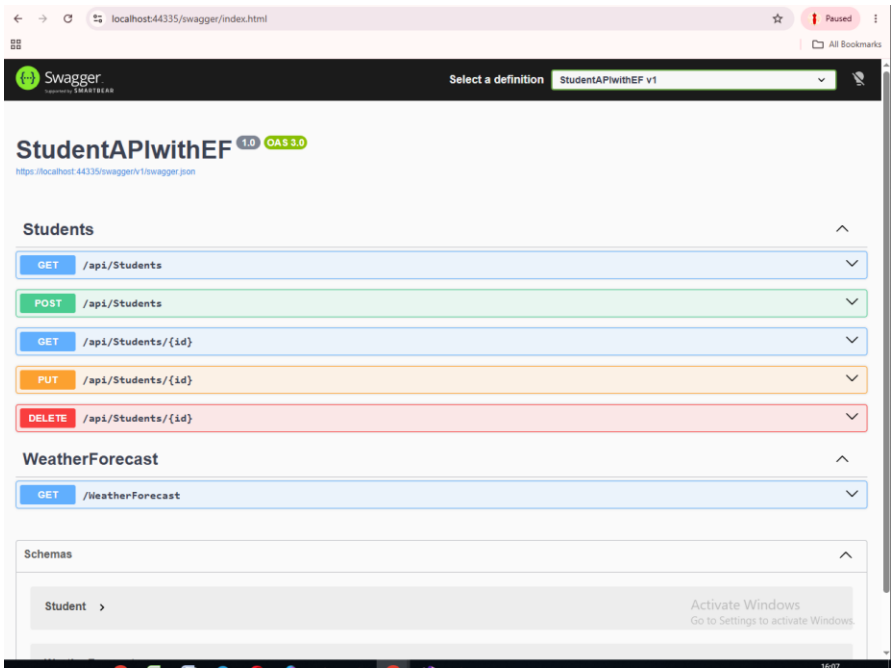
Run: `Add-Migration InitialCreate`

Then Run: `Update-Database`

This will create `StudentDB` database and `Students` table in LocalDB

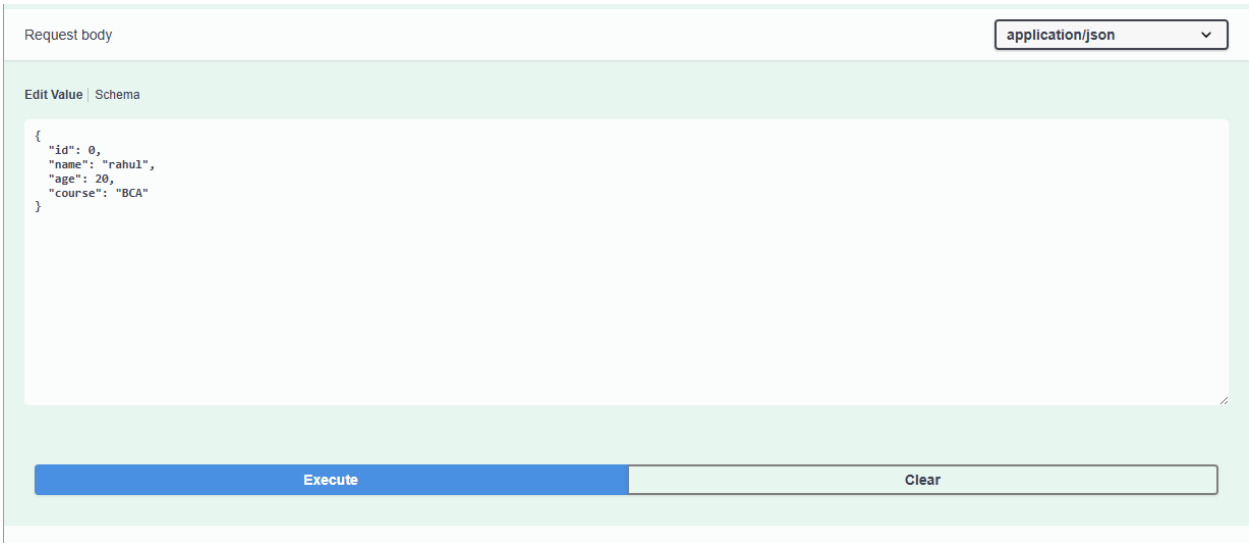
10. Run Project

Swagger UI will open automatically at `https://localhost:xxxx/swagger`



11. Test CRUD Operations

- POST: Insert student with Name, Age, Course
- GET: View all students at `/api/students`
- GET BY ID: View student at `/api/students/1`
- PUT: Update student details
- DELETE: Delete student at `/api/students/1`



{
"name": "Rahul",
"age": 21,
"course": "BCA"
}

12. View Database in Visual Studio

: `View → SQL Server Object Explorer` → Open `(localdb)\MSSQLLocalDB`

Go to: `Databases → StudentDB → Tables` to view inserted records

Curl

```
curl -X 'GET' \
  'https://localhost:44335/api/Students' \
  -H 'accept: */*'
```

Request URL

```
https://localhost:44335/api/Students
```

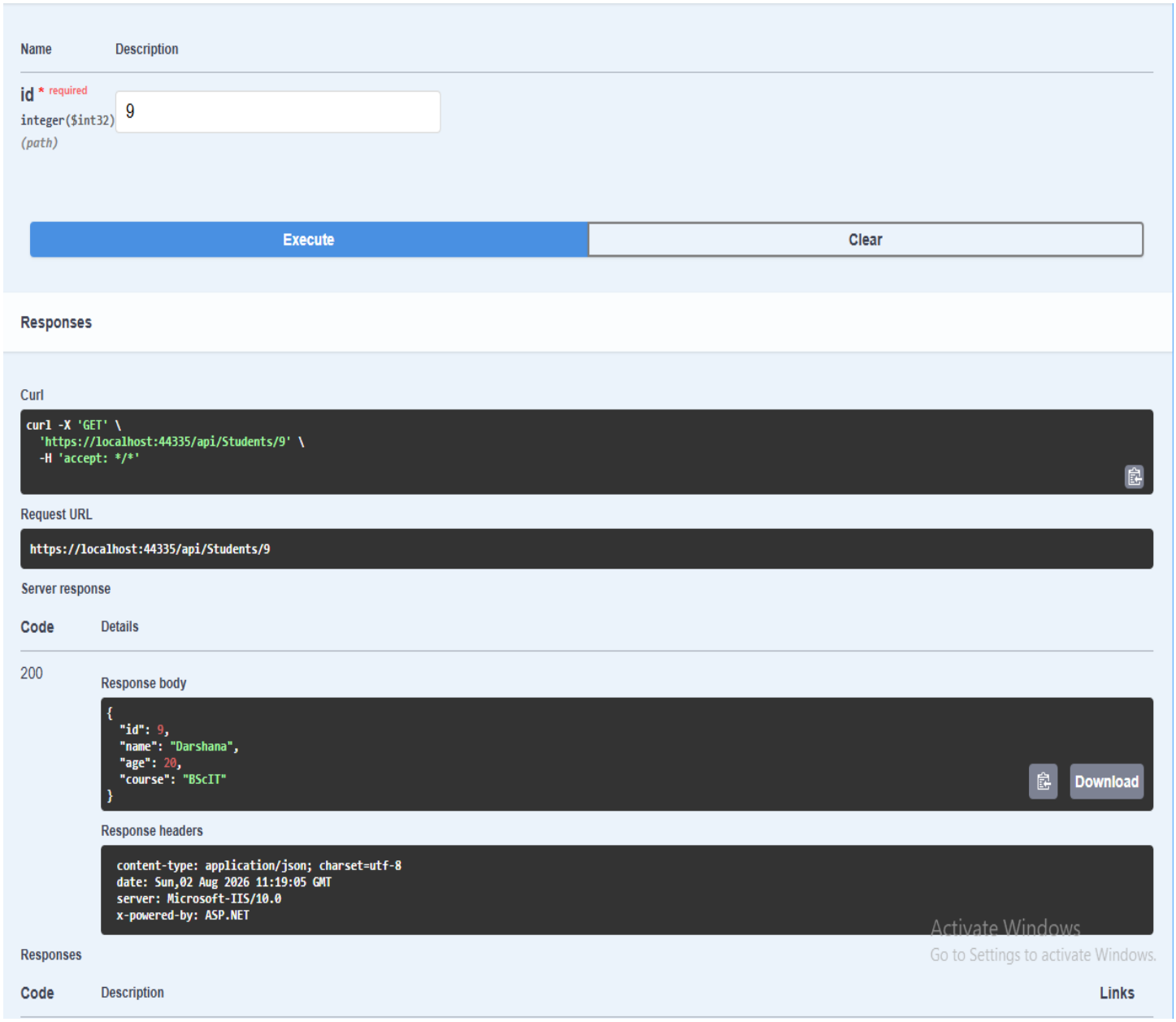
Server response

Code	Details
200	Response body

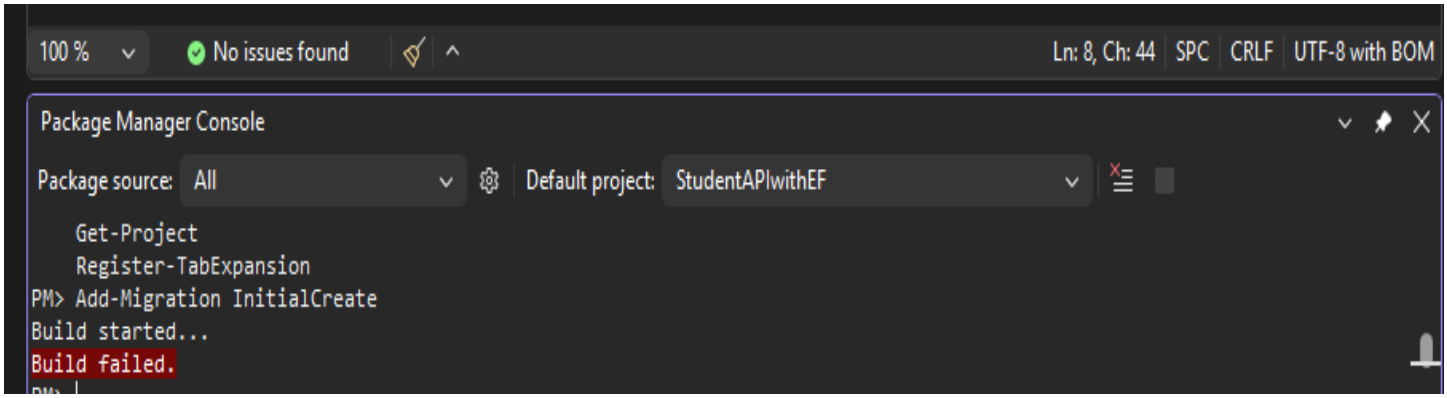

```
  {
    "id": 5,
    "name": "riya",
    "age": 20,
    "course": "BCA"
  },
  {
    "id": 6,
    "name": "darshana",
    "age": 20,
    "course": "BCsIT"
  },
  {
    "id": 7,
    "name": "Isha",
    "age": 19,
    "course": "BCsIT"
  },
  {
    "id": 8,
    "name": "Priya",
    "age": 19,
    "course": "BCsIT"
  },
  {
    "id": 9,
    "name": "Darshana",
    "age": 20,
    "course": "BScIT"
```

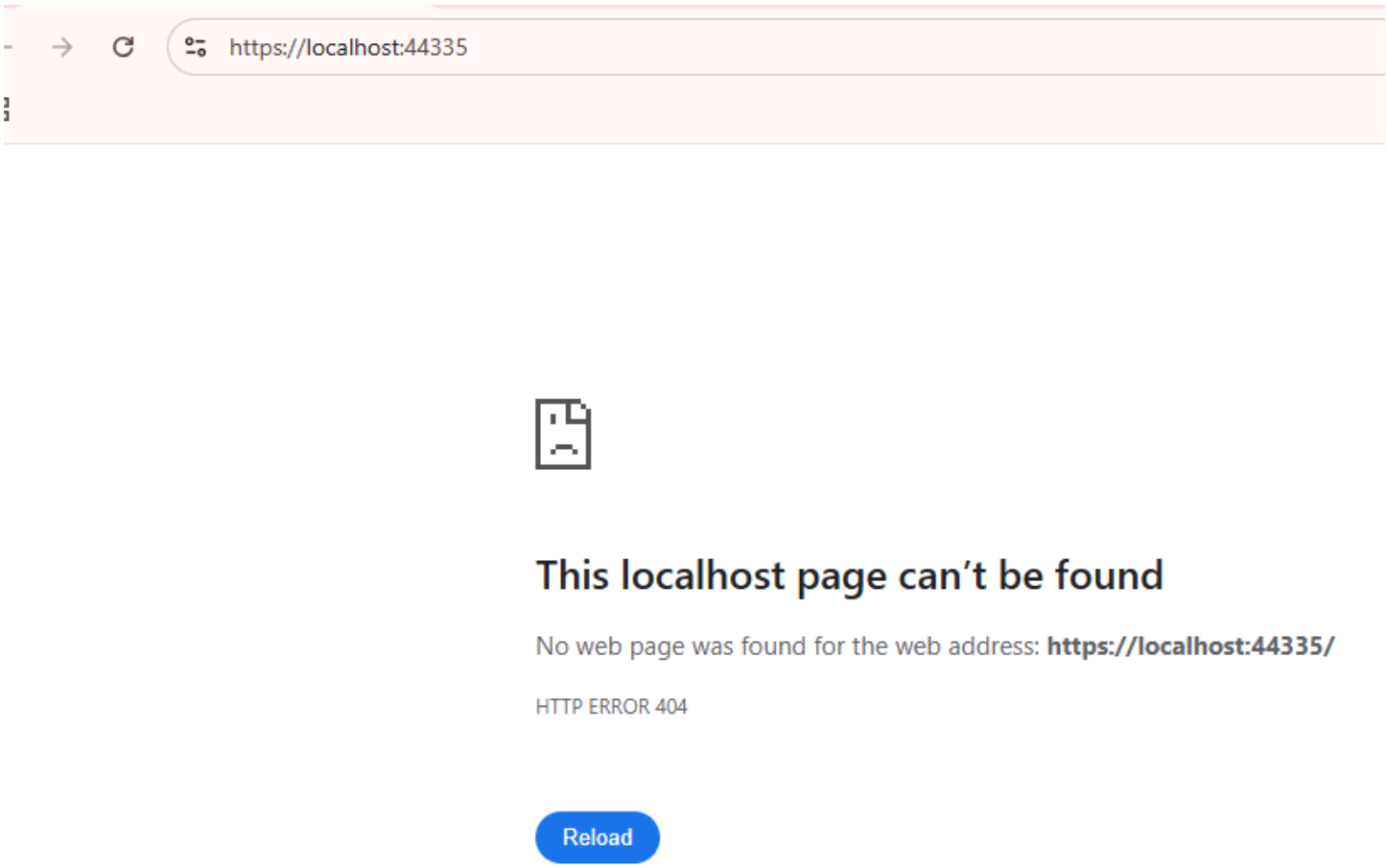

Activate Windows
Go to Settings to activate Windows.

Download


Name
Description
id * required
integer($int32)
(path)
9
Execute
Clear
Responses
Curl
curl -X 'GET' \
'https://localhost:44335/api/Students/9' \
-H 'accept: */*'
Request URL
https://localhost:44335/api/Students/9
Server response
Code
Details
200
Response body
{
"id": 9,
"name": "Darshana",
"age": 20,
"course": "BScIT"
}
Download
Response headers
content-type: application/json; charset=utf-8
date: Sun,02 Aug 2026 11:19:05 GMT
server: Microsoft-IIS/10.0
x-powered-by: ASP.NET
Activate Windows
Go to Settings to activate Windows.
Responses
Code
Description
Links

Errors you might face:





Important Methods/Functions/Attributes Used:

1. `DbContext` - Base class for database context
2. `DbSet<Student>` - To represent Students table
3. `AddDbContext<AppDbContext>()` - To register DbContext in services
4. `options.UseSqlServer()` - To configure SQL Server connection
5. `builder.Configuration.GetConnectionString()` - To read connection string
6. `[ApiController]` - To enable API specific features
7. `[Route("api/[controller]")]` - To define API route
8. `[HttpGet]` - To handle GET requests
9. `[HttpGet("{id}")]` - To handle GET request with parameter
10. `[HttpPost]` - To handle POST requests
11. `[HttpPut("{id}")]` - To handle PUT requests
12. `[HttpDelete("{id}")]` - To handle DELETE requests
13. `_context.Students.ToList()` - To fetch all records
14. `_context.Students.Find(id)` - To find record by Id
15. `_context.Students.Add()` - To add new record
16. `_context.SaveChanges()` - To save changes to database
17. `_context.Students.Remove()` - To delete record
18. `Add-Migration` and `Update-Database` - EF Core commands for database creation

(B) Create an API endpoint to calculate Factorial of a Number and test it using Postman.

Steps:

1. Create ASP.NET Core Web API Project

Open Visual Studio 2026 → Create New Project

Select `ASP.NET Core Web API`

Project Name: `FactorialAPI`

Framework: `.NET 8` → Enable OpenAPI Support → Click: Create

2. Create Controller

: `Controllers` folder → Add Class: `MathController.cs`

3. Write Factorial Logic

Create GET method with route `factorial/{number}`

Check if number is negative, return BadRequest

Use loop to calculate factorial of the given number

Return result as JSON with Number and Factorial

```
using Microsoft.AspNetCore.Mvc;

namespace FactorialAPI.Controllers

{

    [Route("api/[controller]")]

    [ApiController]

    public class MathController : ControllerBase

    {

        [HttpGet("factorial/{number}")]

        public IActionResult GetFactorial(int number)

        {

            if (number < 0)
```

```
        {
            return BadRequest("Factorial not possible for negative numbers.");
        }

        long factorial = 1;

        for (int i = 1; i <= number; i++)
        {
            factorial = i;
        }

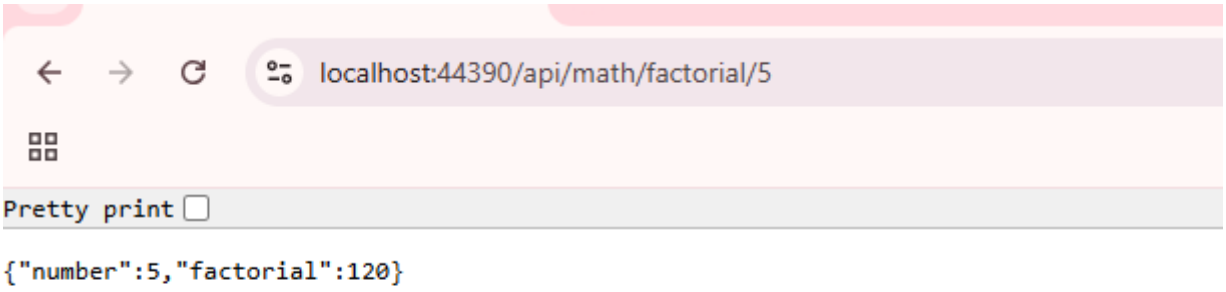
        return Ok(new
        {
            Number = number,
            Factorial = factorial
        });
    }
  }
}
```

4. Run the Project

Project will run at `https://localhost:xxxx`

5. Test API in Browser

Open URL: https://localhost:44335/api/math/factorial/5

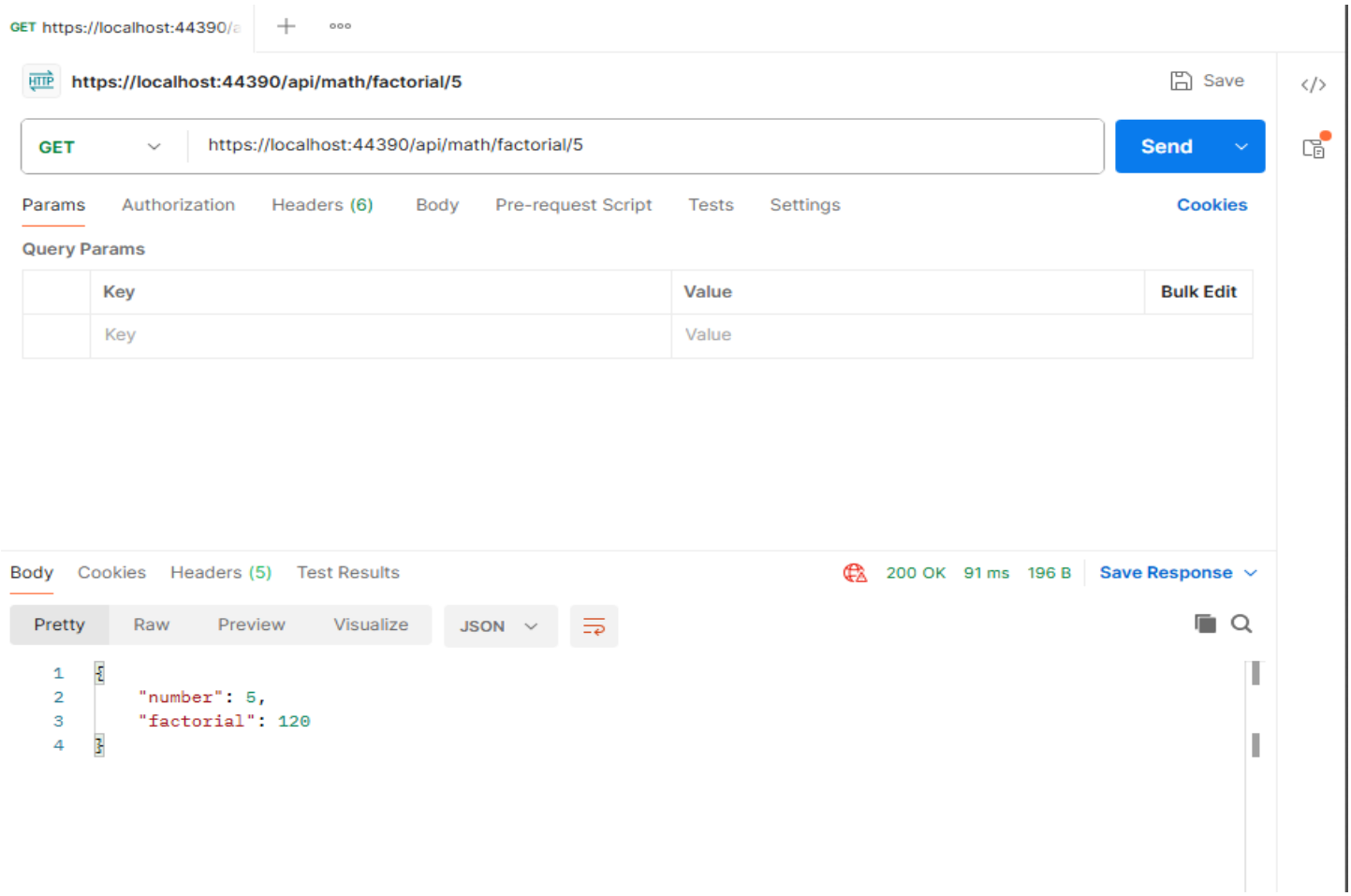


6. Test using Postman

Open Postman → Select Method: `GET`

Enter URL: `https://localhost:xxxx/api/math/factorial/5`

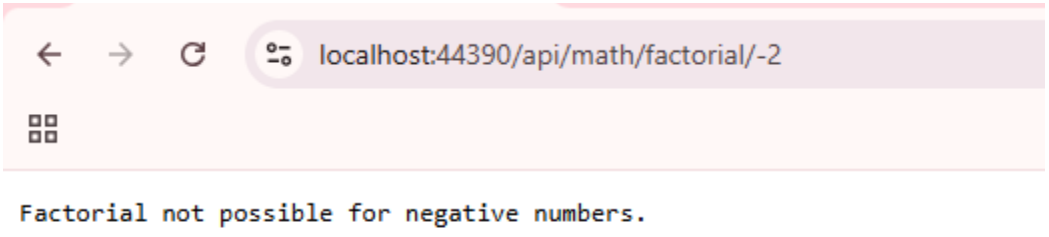
Click: Send



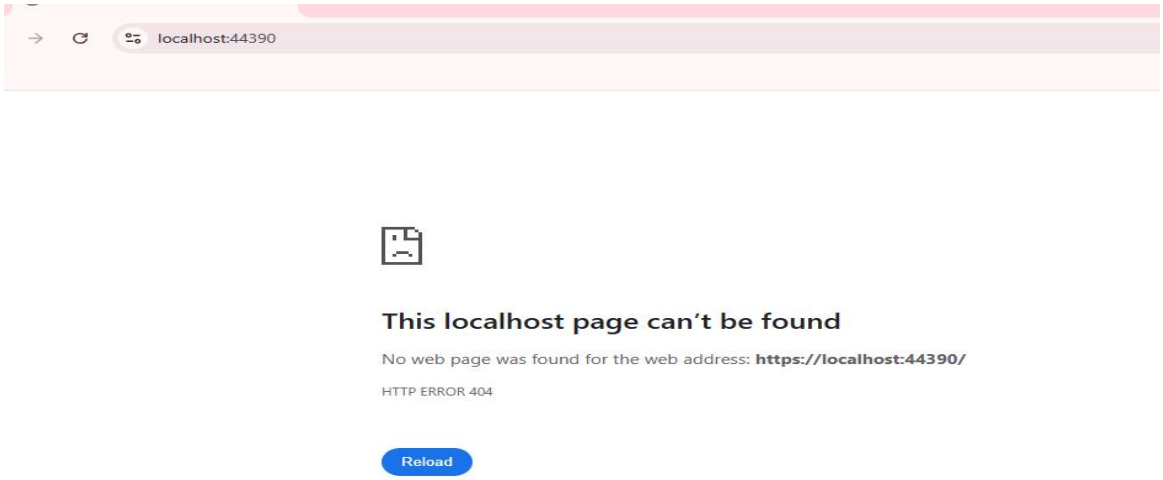
7. Test Negative Number

Enter URL: `https://localhost:xxxx/api/math/factorial/-2`

Click: Send



Errors:



Important Methods/Functions/Attributes Used

1. `[ApiController]` - To enable API specific features
2. `[Route("api/[controller]")]` - To define API route
3. `[HttpGet("factorial/{number}")]` - To handle GET request with number parameter
4. `BadRequest()` - To return 400 error for invalid input
5. `Ok()` - To return 200 success with result
6. `for loop` - To calculate factorial
7. `return Ok(new { Number = number, Factorial = factorial })` - To return JSON response

PRACTICAL-10

(A) Create a PWA with a web App Manifest file including:

- **App name**
- **Icons**
- **Start URL**
- **Display mode**
- **Implement Push Notifications in a PWA.**

Steps:

1. Create Project

Open Visual Studio 2026 → Create new project

Select `ASP.NET Core Empty Web App`

Project Name: `PushNotification`

Click: Create

2. Create Manifest File

: Right-click Project → `Add → New Folder` → Name: `wwwroot`

Inside `wwwroot` → `Add → New Item` → `JSON File`

Name: `manifest.json`

```
  {
"name": "My PWA Application",
"short_name": "MyPWA",
"start_url": "/index.html",
"display": "standalone",
"background_color": "#ffffff",
"theme_color": "#0d6efd",
"icons": [
 {
  "src": "icons/icon1.png",
  "sizes": "192x192",
  "type": "image/png"
 },
 {
  "src": "icons/icon2.png",
  "sizes": "512x512",
```

```
    "type": "image/png"
   }
  ]
}
```

3. Add Icons

Right-click `wwwroot` → `Add → New Folder` → Name: `icons`

Add: `icon1.png` 192x192

`icon2.png` 512x512

4. Create HTML Page

: `wwwroot` → `Add → New Item` → `HTML Page` → Name: `index.html`

Add heading "My Progressive Web App" and Button "Show Notification"

Link `manifest.json` in `<head>` and `app.js` in `<body>`

```
<!DOCTYPE html>
<html>
<head>
   <title>My PWA</title>
   <link rel="manifest" href="manifest.json">
   <meta name="theme-color" content="#0d6efd">
</head>
<body>
   <h1>My Progressive Web App</h1>
   <button onclick="showNotification()">
      Show Notification
   </button>

   <script src="app.js"></script>
</body>
</html>
```

5. Create JavaScript File for Notification

: `wwwroot` → `Add → New Item` → `JavaScript File` → Name: `app.js`

Register Service Worker and create `showNotification()` function

```
if ('serviceWorker' in navigator) {
   navigator.serviceWorker.register('service-worker.js')
      .then(() => console.log('Service Worker Registered'));
}

function showNotification() {
```

```
    Notification.requestPermission().then(permission => {
        if (permission === 'granted') {
            navigator.serviceWorker.ready.then(registration => {
                registration.showNotification('PWA Notification', {
                    body: 'Push Notification Working Successfully!',
                    icon: 'icons/icon1.png'
                });
            });
        }
    });
}
```

6. Create Service Worker

`wwwroot` → `Add → New Item` → `JavaScript File`

Name: `service-worker.js`

Add event listeners for `install` and `activate` events

```
 self.addEventListener('install', event => {
    console.log('Service Worker Installed');
});

self.addEventListener('activate', event => {
    console.log('Service Worker Activated');
});
```

7. Configure Program.cs

`Program.cs`

Enable `UseDefaultFiles()` and `UseStaticFiles()` for serving PWA files

```
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

app.UseDefaultFiles(); // index.html ko default banata hai
app.UseStaticFiles();  // css, js, manifest serve karta hai

app.Run();
```

8. Run Application

Browser opens at `https://localhost:7236`

PWA Install icon appears in address bar. Click to install

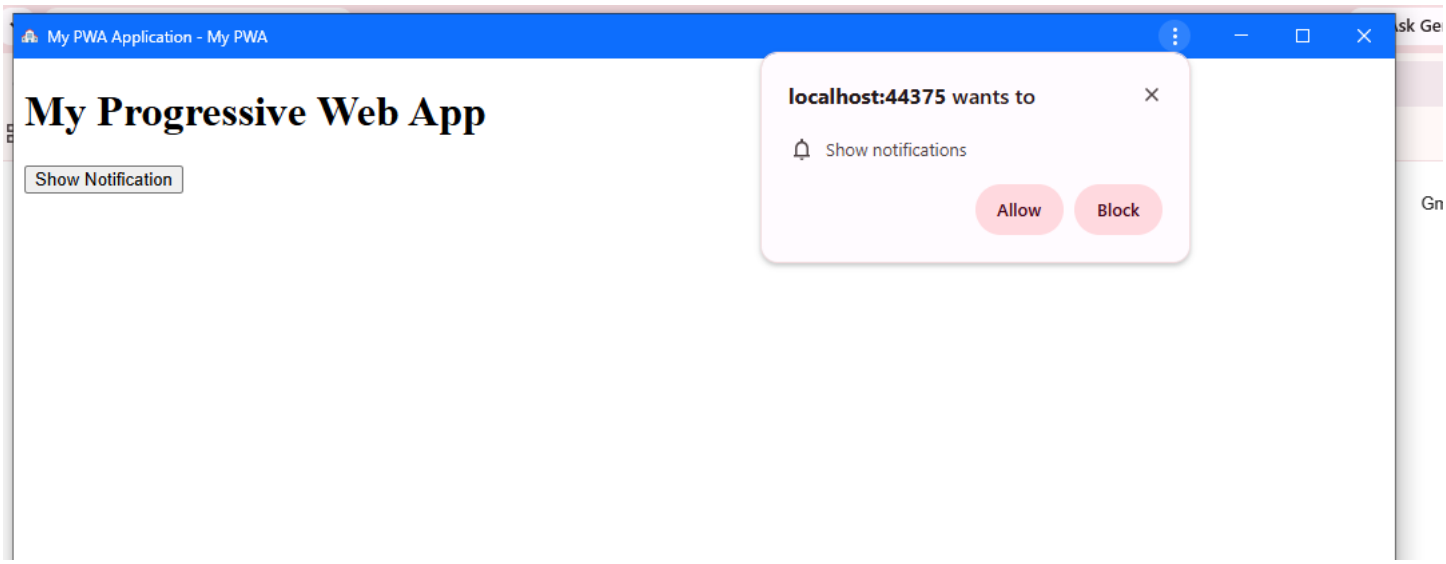
9. Install PWA

Open website in Chrome → Click Install icon → App installs like mobile app



10. Test Push Notification

Click "Show Notification" button → Browser asks permission → Click Allow → Notification appears



Important Methods/Functions Used:

1. `navigator.serviceWorker.register('service-worker.js')` - To register service worker
2. `Notification.requestPermission()` - To request permission for notifications
3. `navigator.serviceWorker.ready.then()` - To get service worker registration
4. `registration.showNotification(title, options)` - To display push notification
5. `self.addEventListener('install', event)` - To handle service worker install
6. `self.addEventListener('activate', event)` - To handle service worker activation
7. `app.UseDefaultFiles()` - To enable default file serving
8. `app.UseStaticFiles()` - To enable serving static files

(B) E-Commerce PWA with Product Listing and Offline Cart

To develop an E-Commerce Progressive Web Application with product listing and offline cart functionality using Service Worker and local storage.

Steps:

1. Create Project

Open Visual Studio 2026 → Create new project

Select `ASP.NET Core Empty Web App`

Project Name: `ECommercePWA`

Click: Create

2. Configure Program.cs

`Program.cs`

Enable `UseDefaultFiles()` and `UseStaticFiles()` for static content

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
app.UseDefaultFiles();
app.UseStaticFiles();
app.Run();
```

3. Create Manifest File

: Right-click Project → `Add → New Folder` → Name: `wwwroot`

Inside `wwwroot` → `Add → New Item` → `JSON File`

Name: `manifest.json`

```
{
"name": "E-Commerce PWA",
"short_name": "ShopPWA",
"start_url": "/index.html",
"display": "standalone",
"background_color": "#ffffff",
"theme_color": "#0d6efd",
"icons": [
 {
  "src": "icons/icon1.png",
  "sizes": "192x192",
  "type": "image/png"
```

```
    },
    {
      "src": "icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

4. Add Icons

: `wwwroot` → `Add → New Folder` → Name: `icons`

Add: `icon1.png`

`icon2.png`

5. Create HTML Page with Product Listing

: `wwwroot` → `Add → New Item` → `HTML Page` → Name: `index.html`

Design layout with 2 sections: `Cart Items` and `Products`

Products: Laptop - ₹50000, Mobile - ₹20000, Headphones - ₹3000

Each product has "Add to Cart" button

```
<!DOCTYPE html>
<html>
<head>
    <title>E-Commerce PWA</title>
    <link rel="manifest" href="manifest.json">

    <meta name="theme-color" content="#0d6efd">
    <style>
        body {
            font-family: Arial;
            margin: 20px;
        }

        .container {
            display: flex;
            gap: 30px;
        }

        .cart-section {
            width: 30%;
            border: 1px solid gray;
            padding: 15px;
            min-height: 300px;
```

```
      }

      .product-section {
         width: 70%;
      }

      .products {
         display: flex;
         flex-wrap: wrap;
         gap: 15px;
      }

      .product {
         border: 1px solid gray;
         padding: 10px;
         width: 200px;
      }

      button {
         padding: 5px 10px;
         cursor: pointer;
      }
   </style>
</head>
<body>
   <h1>E-Commerce PWA</h1>
   <div class="container">

<div class="cart-section">
         <h2>Cart Items</h2>
         <ul id="cartList"></ul>
      </div>

<div class="product-section">

<h2>Products</h2>
         <div class="products">
            <div class="product"><h3>Laptop</h3><p>Price: ₹ 50000</p><button
onclick="addToCart('Laptop')">Add to Cart</button></div>
            <div class="product"><h3>Mobile</h3><p>Price: ₹ 20000</p><button
onclick="addToCart('Mobile')">Add to Cart</button></div>
            <div class="product"><h3>Headphones</h3><p>Price: ₹ 3000</p><button
onclick="addToCart('Headphones')">Add to Cart</button></div>
         </div>
      </div>
   </div>
   <script src="app.js"></script>
</body>
</html>
```

6. Create JavaScript File for Cart Functionality

: `wwwroot` → `Add → New Item` → `JavaScript File` → Name: `app.js`

Register Service Worker

Use `localStorage` to store cart items for offline functionality

Create functions: `addToCart(product)` and `displayCart()`

```
if ('serviceWorker' in navigator) {
    navigator.serviceWorker.register('service-worker.js')
        .then(() => console.log('Service Worker Registered'));
}

let cart = JSON.parse(localStorage.getItem('cart')) || [];
displayCart();

function addToCart(product) {
    cart.push(product);
    localStorage.setItem('cart', JSON.stringify(cart));
    displayCart();
}

function displayCart() {
    let cartList = document.getElementById('cartList');
    cartList.innerHTML = "";
    cart.forEach(item => {
        let li = document.createElement('li');
        li.textContent = item;
        cartList.appendChild(li);
    });
}
```

7. Create Service Worker for Offline Support

: `wwwroot` → `Add → New Item` → `JavaScript File`

Name: `service-worker.js`

Define `CACHE_NAME` and `urlsToCache` array

Cache files during `install` event using `cache.addAll()`

Serve cached files during `fetch` event

```
const CACHE_NAME = 'ecommerce-cache-v1';
```

```
const urlsToCache = [
    '/',
    '/index.html',
    '/app.js',
    '/manifest.json'
];

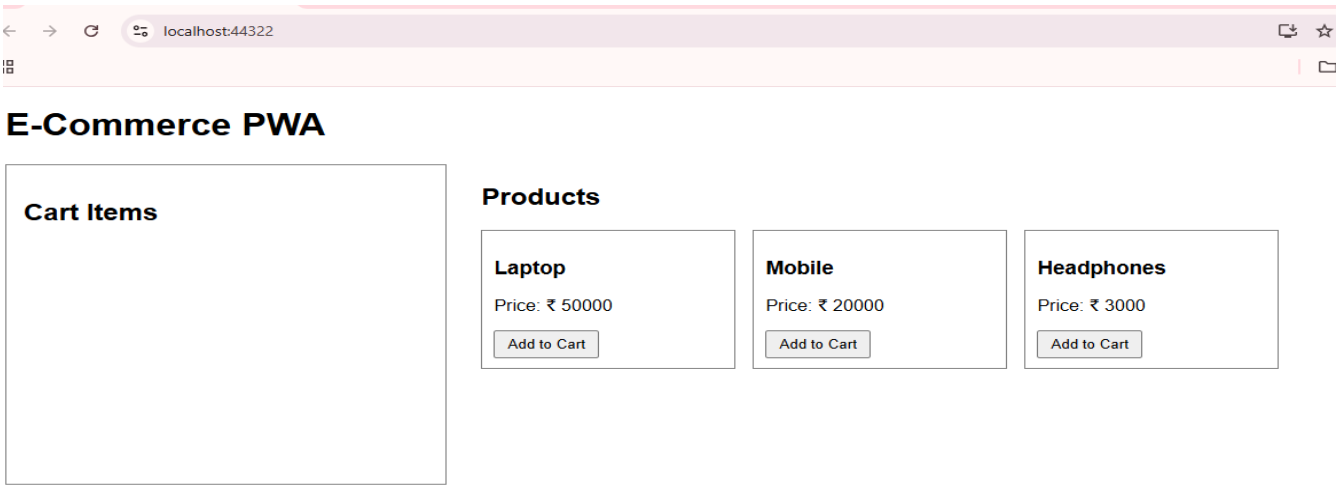

self.addEventListener('install', event => {
    event.waitUntil(
        caches.open(CACHE_NAME)
            .then(cache => {
                return cache.addAll(urlsToCache);
            })
    );
});


self.addEventListener('fetch', event => {
    event.respondWith(
        caches.match(event.request)
            .then(response => {
                return response || fetch(event.request);
            })
    );
});
```

8. Run Application

Browser opens at `https://localhost:7178`

Add products to cart. Data persists even after refresh



9. Test Offline Functionality

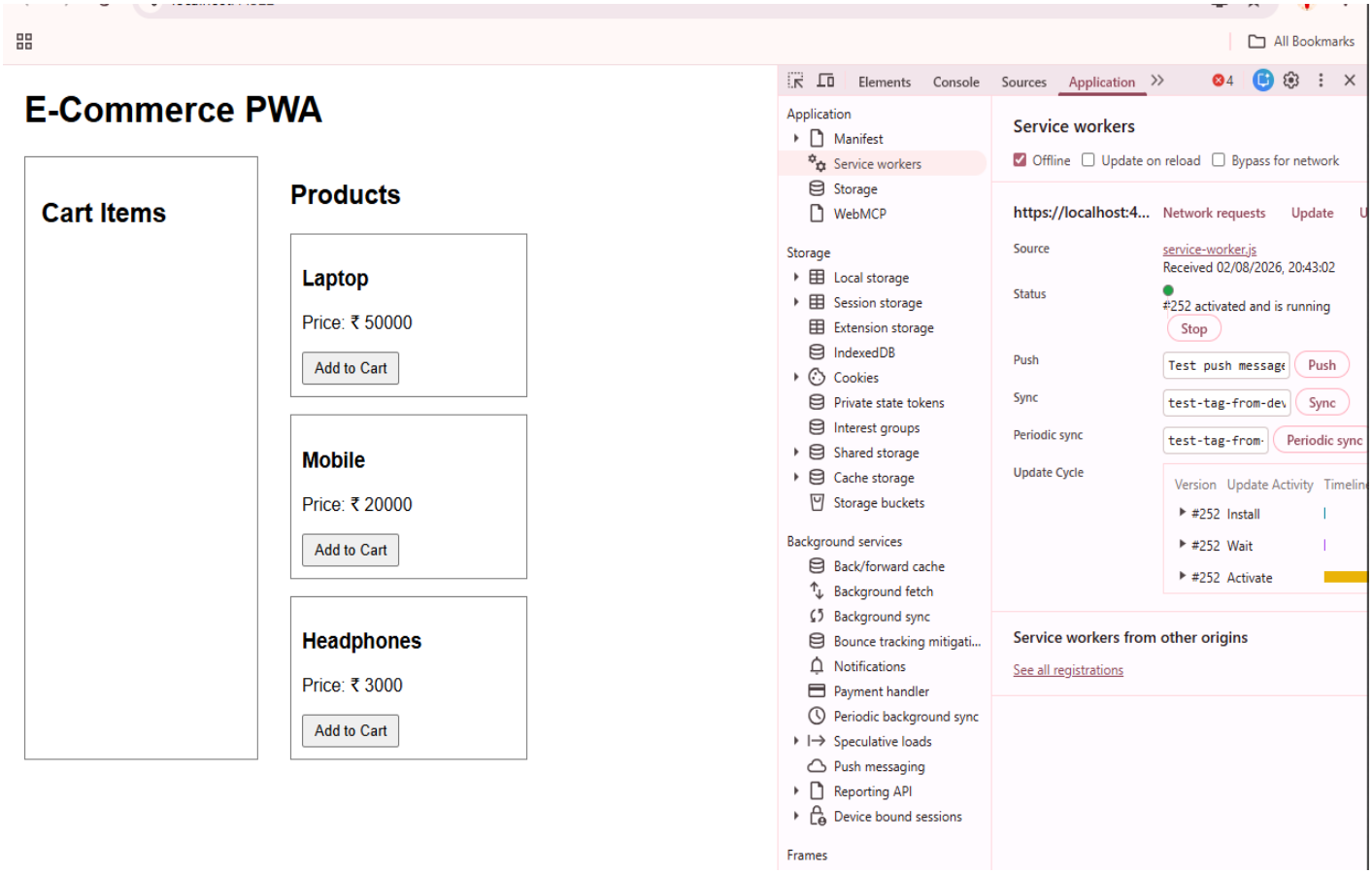
Step 1: Open Chrome DevTools `F12`

Step 2: Go to `Network` tab → Select `Offline`

Step 3: Refresh page

Application still Works because

- Files cached by Service Worker
- Cart stored



Important Methods/Functions Used:

1. `navigator.serviceWorker.register('service-worker.js')` - To register service worker
2. `JSON.parse(localStorage.getItem('cart'))` - To retrieve cart from local storage
3. `localStorage.setItem('cart', JSON.stringify(cart))` - To save cart to local storage
4. `cart.push(product)` - To add product to cart array
5. `document.getElementById('cartList')` - To get cart list element
6. `document.createElement('li')` - To create list item for cart
7. `cart.forEach(item => )` - To iterate and display cart items
8. `caches.open(CACHE_NAME)` - To open cache storage
9. `cache.addAll(urlsToCache)` - To cache files during install
10. `self.addEventListener('install', event)` - To handle install event
11. `self.addEventListener('fetch', event)` - To handle fetch requests
12. `caches.match(event.request)` - To check cache for request
13. `fetch(event.request)` - To fetch from network
14. `app.UseDefaultFiles()` - To enable default file serving
15. `app.UseStaticFiles()` - To enable serving static files